



Java Full-Stack Interview Guide

Concise Interview-Ready Answers

Core Java · Java 8 Streams · Spring Boot · JPA/Hibernate · Security & JWT ·
Microservices · Kafka · SQL · Angular · Production Scenarios · Project & HR

245 unique questions · 28 diagrams · 5 October 2026

Contents

A. Core Java

- A1. What is the use of Docker?
- A2. What are the main features introduced in Java 8, and wh...
- A3. What are the important features introduced in Java 18, a...
- A4. Can we override the main method in Java? Explain why o...
- A5. Explain the main OOP concepts in Java.
- A6. How do you achieve abstraction in Java? Write a class str...
- A7. Why were functional interfaces introduced, what is one, ...
- A8. If a functional interface already has one abstract method...
- A9. What is the use of default methods in interfaces?
- A10. What is the use of static methods in interfaces?
- A11. Can we override private methods?
- A12. Can we override static methods? Can we override a stat...
- A13. What is method hiding?
- A14. What is serialization? How do you prevent a variable fr...
- A15. Explain the volatile keyword.
- A16. What is garbage collection and how is it used?
- A17. Output of `System.out.println(Double.MIN_VALUE > 0.0...`
- A18. If you add a null to an empty Set, what is the size?
- A19. Can we call one constructor from another? How?
- A20. Explain composition and aggregation with examples, a...
- A21. What is polymorphism? Real project example.
- A22. Explain method overriding with a runtime polymorphis...
- A23. Class A has a public method; class B overrides it as priv...
- A24. Difference between HashMap and Hashtable.
- A25. Internal working of HashMap
- A26. How are HashMap collisions handled?
- A27. `hashCode()`, `equals()`, `toString()` in the context of HashM...
- A28. What happens when custom objects are used as Hash...
- A29. HashMap vs ConcurrentHashMap.
- A30. Java memory areas: Heap and Stack (plus Metaspace br...
- A31. Explain the Java Collection Framework.
- A32. Concurrent collections and concurrent access in Java.
- A33. `wait()` vs `sleep()`.
- A34. The Callable interface.
- A35. `execute()` vs `submit()`.
- A36. How do you create a thread? Extending Thread vs impl...
- A37. How did you connect Java with Python?
- A38. What is the SOLID principle?
- A39. Monolithic vs microservice architecture.
- A40. Which declarations are correct and why?
- A41. Which array declarations are valid and why?
- A42. Thread `start()` vs `run()`: which method do you call and w...

B. Java 8, Streams, Optional, Lambdas & Coding Problems

- B1. Explain Java 8 features.
- B2. What is a lambda expression?
- B3. Functional interfaces with respect to lambdas.
- B4. What is a method reference? (4 kinds)
- B5. Stream API methods: intermediate vs terminal.
- B6. `map()` vs `flatMap()`
- B7. How do you debug a Java Stream?
- B8. Optional for null-safe access; chaining with streams.
- B9. Sequential vs parallel streams.
- B10. Count duplicate elements in an array.
- B11. Find even numbers from a list.
- B12. Occurrence of a character in a string.
- B13. Count first-character occurrences in a list of strings.
- B14. Remove duplicates while maintaining order.
- B15. Remove null and empty strings from a list.
- B16. Convert list of strings to uppercase.
- B17. Join strings into a comma-separated value.
- B18. Sum all numbers using `reduce()`.
- B19. `List<String> → Map<String,Integer>` (`string → length`).
- B20. Group employees by department.
- B21. Sort employees by salary.
- B22. Second-highest number in a list (stream).
- B23. Second-largest distinct element, single pass $O(n)$.
- B24. Flatten a list of lists with `flatMap`.
- B25. Count elements by category (`groupingBy` + counting).
- B26. Sort a map by values.
- B27. Filter a list based on multiple conditions.
- B28. Partition into even/odd with `partitioningBy`.
- B29. Handle duplicate keys in `Collectors.toMap()`.
- B30. Caching with `ConcurrentHashMap.computeIfAbsent()`.

B31. Custom collector to join strings in uppercase.
 B32. Stream a large file line by line efficiently.
 B33. Generic method using functional interfaces.
 B34. Stream pipeline: CSV → filter → map → collect to a reco...
 B35. Longest string.
 B36. Convert "this is test" into "THIS IS TEST" using Stream A...
 B37. "Get the highest substring"
 B38. First non-repeating character in a string.

B39. Remove duplicate letters, smallest lexicographic result (...
 B40. Search in a rotated sorted array (4,5,6,7,0,1,2,3).
 B41. Merge Sort, Quick Sort, Bubble Sort, Linear Search, Bina...
 B42. Count word frequency in a given string.
 B43. Merge two arrays without duplicates.
 B44. Rotate an $N \times N$ matrix by $K \times 90^\circ$ clockwise.
 B45. Check whether brackets {}, [], () are balanced.

C. Spring Boot, Spring Framework, JPA/Hibernate & Testing

C1. What is @SpringBootApplication?
 C2. What happens if @EnableAutoConfiguration (or equival...
 C3. @Configuration vs @Bean; why do we need configuratio...
 C4. What is a Bean? How is a Bean created internally?
 C5. Bean life cycle.
 C6. Bean scopes; @Scope annotation.
 C7. @Lazy.
 C8. @Component; component scanning.
 C9. Dependency injection in Spring Boot; types of DI.
 C10. Multiple beans of the same type: how to specify which ...
 C11. What is JPA? (vs Hibernate vs Spring Data JPA)
 C12. @GeneratedValue (strategies).
 C13. @RequestParam, @RequestBody, @PathVariable; handl...
 C14. What are REST controllers used for? (@RestController v...
 C15. Skeleton for getEmployeeById.
 C16. PUT vs PATCH.
 C17. HTTP 204.

C18. Spring Boot Actuator.
 C19. Front Controller design pattern (DispatcherServlet).
 C20. Exception handling in Spring Boot.
 C21. @ConfigurationProperties; creating custom configurati...
 C22. Configuring different environments.
 C23. SessionFactory: what it is and how to configure it.
 C24. Configuring multiple data sources.
 C25. Lazy vs eager loading in Hibernate.
 C26. One-to-Many relationship across multiple tables.
 C27. Implementing audit logs.
 C28. URL shortener in Java/Spring Boot.
 C29. What does LOC mean?
 C30. @Transactional and why transaction management is us...
 C31. @Cacheable, caching for DB performance, and cache c...
 C32. JUnit annotations.
 C33. Writing test cases for Spring Boot applications.
 C34. Testing REST APIs.
 C35. Handling exceptions in test cases.

D. Security, Microservices & System Design

D1. Authentication vs authorization in Java/Spring Boot.
 D2. How do you implement authentication & authorization i...
 D3. Handling exceptions in authentication and authorization.
 D4. Pre-authorization (@PreAuthorize).
 D5. How does JWT work?
 D6. Explain the authentication layer in your project (templat...
 D7. Security layers for REST APIs; best practices; securing int...
 D8. What are microservices; key things to take care of.
 D9. Explain the microservices architecture used in your proje...
 D10. How do microservices communicate?
 D11. Circuit breaker.
 D12. Retry mechanism.
 D13. If Microservice B is down, what should A do?

D14. API Gateway (Spring Cloud Gateway).
 D15. Load Balancer; Application Load Balancer.
 D16. Distributed caching and distributed tracing.
 D17. What is Kafka and its working flow.
 D18. Explain one microservice you worked on (template: Inci...
 D19. When integrating 2–3 microservices, do they connect d...
 D20. Complete API request flow through microservices.
 D21. API works internally but not for the client: primary inve...
 D22. API response is slow: primary investigation.
 D23. Delay caused by a stored procedure / SQL query: identi...
 D24. Design patterns used in microservices, and in code.
 D25. Cloud experience: which cloud services and tools have ...
 D26. Stateless vs stateful services and authentication.

E. SQL, Angular/JavaScript, Project & HR

- E1. Find the second-highest salary (3 ways) + employees wh...
- E2. Highest-paid employee(s) in each department (ties inclu...
- E3. Name and salary of employees in each department
- E4. Window functions
- E5. ACID properties
- E6. Stored procedures: what, why, "hierarchy"
- E7. Table A and Table B hold the same records: what does IN...
- E8. 20 records starting from the 5th row (first and last name)
- E9. INNER JOIN between two tables
- E10. Highest receiver amount
- E11. Explain Angular
- E12. Your Angular work / roles / how you introduce yourself ...
- E13. Basic JavaScript concepts

F. Production & Troubleshooting Scenarios

- F1. Your application suddenly throws OutOfMemoryError in ...
- F2. Multiple users are updating the same data simultaneousl...
- F3. One API is taking 20 seconds response time only in prod...
- F4. CPU usage suddenly increased to 95% in production serv...
- F5. Your application is creating too many threads and server ...
- F6. One scheduled job is processing duplicate records multi...
- F7. Garbage collection time suddenly increased and applicat...
- F8. Large Excel/file processing causes application crash due t...
- F9. A production issue occurs randomly and logs are not cle...
- F10. One API works for small data but fails for huge data. W...
- F11. Two microservices are communicating and one depend...
- F12. Your API is returning 401 Unauthorized randomly for va...
- F13. After deployment application is starting very slowly. Wh...
- F14. One API is receiving too many requests and server beco...
- F15. Your Spring Boot application works locally but fails in Q...
- F16. A REST API is exposing sensitive data accidentally. What...
- F17. One external API dependency is timing out frequently. ...
- F18. Application configuration differs between DEV, QA, and...
- F19. Your API is throwing exceptions but frontend only sees ...
- F20. A scheduled task is running multiple times in clustered ...
- F21. Your application suddenly becomes slow because of dat...
- F22. Hibernate lazy loading exception occurs only in product...
- F23. Database connections are exhausted and new users can...
- F24. One query works fast initially but becomes slow after d...
- F25. Duplicate records are inserted during concurrent transa...

- E14. var, let, const and hoisting (TDZ)
- E15. Tell me about yourself (TEMPLATE, ~60–90 s)
- E16. Roles and responsibilities (TEMPLATE)
- E17. Explain your project (TEMPLATE)
- E18. Architecture and codebase structure (TEMPLATE + typic...
- E19. Technologies and tools (TEMPLATE: pick what you actua...
- E20. Coding standards, best practices and precautions
- E21. Challenges faced (STAR template + examples)
- E22. How do you handle conflicts? (STAR template)
- E23. How do you integrate third-party data?
- E24. Microservices in your project / how you handle microse...
- E25. "Given a list of strings, write..." (question incomplete)
- E26. Puzzle: 12 balls, one odd (heavier or lighter), find it in 3 ...
- E27. Puzzle: remove exactly half the water from a full cylindri...

- F26. A transaction partially updates data and application cra...
- F27. Deadlock occurs frequently in database during peak tra...
- F28. Pagination API becomes slow for huge tables. How will ...
- F29. Users report inconsistent data between UI and databas...
- F30. Batch insert operation is taking too much time. How wil...
- F31. One microservice is down and another service continuo...
- F32. Service-to-service communication is failing intermittently...
- F33. One microservice deployment breaks communication w...
- F34. Distributed transaction failed between payment service ...
- F35. Users are seeing old data because services are not sync...
- F36. API gateway is routing traffic incorrectly after deployme...
- F37. One microservice scales properly but another becomes ...
- F38. Logs are distributed across multiple services and difficul...
- F39. A dependent service becomes slow and entire system r...
- F40. Your microservice is healthy but Kubernetes/load balan...
- F41. Kafka consumers are processing same message multipl...
- F42. Consumer lag is increasing continuously in production. ...
- F43. Messages are not reaching consumer even though pro...
- F44. Kafka broker goes down during high traffic. How will yo...
- F45. Events are processed out of order causing wrong busin...
- F46. One failed message blocks entire consumer processing....
- F47. Retry mechanism causes duplicate event processing. Ho...
- F48. High volume event traffic slows down consumers. How ...
- F49. Offsets are committed incorrectly and messages are los...
- F50. Different services consume same event differently causi...

F51. Payment updated successfully but notification service fa...

F52. Distributed cache contains stale data and users see out...

F53. One server has updated data but another server still sh...

F54. Network partition occurs between services in distribute...

F55. Multiple services update same resource causing inconsi...

F56. One region/server goes down completely. How will hig...

F57. Traffic suddenly increases 10x during sale/event. How w...

F58. Data synchronization failed between microservices and ...

F59. A service continuously retries failed requests and create...

F60. Distributed tracing is missing for some requests. How w...

F61. Production is down and users cannot login. What will b...

F62. After deployment users report major issue. How will yo...

F63. Logs are huge and issue is not reproducible locally. Ho...

F64. One API fails only during peak traffic hours. How will yo...

F65. Application works fine from Postman but frontend user...

F66. Monitoring alerts show high memory usage but applica...

F67. SSL certificate expired in production. What impact can ...

F68. A batch job failed in middle after processing lakhs of re...

F69. Server restarted automatically at midnight. How will yo...

F70. Business team says reports show wrong data in product...

Short answers to say out loud. For internals, edge cases and follow-ups, see [Interview-Guide-InDepth.md](#).

How to use this guide

The original question bank had ~330 questions, many repeated across the topic and company sections. They have been **deduplicated into 245 unique questions** grouped into six sections:

Section	Topic
A	Core Java (OOP, collections, HashMap internals, multithreading, memory, tricky output questions)
B	Java 8+, Streams, Optional, Lambdas, and coding/DSA programs
C	Spring Boot, Spring Framework, JPA/Hibernate, caching, transactions, JUnit/testing
D	Security (JWT, Spring Security), Microservices, Kafka, System Design, troubleshooting
E	SQL, Angular/JavaScript, Project/Architecture, HR questions and puzzles
F	Production and troubleshooting scenarios: Java, Spring Boot, Hibernate/DB, Microservices, Kafka, Distributed systems, Production support

Answers to "your project" and HR questions are **templates with [placeholders]** . Replace them with your real work before the interview.

Company question index

Each company question maps to its answer ID, e.g. [C15](#) is section C, question 15.

TCS

#	Question	Answer
1	Configure different environments in Spring Boot	C22
2	Skeleton for getEmployeeById	C15
3	What are REST controllers used for	C14
4	Occurrence of a character using Stream API	B12
5	Java 8 features	A2 , B1
6	Stream API methods	B5
7	Functional interfaces w.r.t. lambda	B3
8	Java memory: Heap and Stack	A30
9	Collection Framework	A31
10	Search target in rotated list 4,5,6,7,0,1,2,3	B40
11	Merge/Quick/Bubble Sort, Linear/Binary Search	B41
12	What is a Bean	C4
13	Two beans with the same name	C10
14	No @EnableAutoConfiguration	C2
15	map() vs flatMap()	B6

Company 2

#	Question	Answer
1	Internal working of HashMap	A25
2	HashMap collisions	A26
3	hashCode>equals/toString in HashMap	A27
4	Stack and Heap	A30
5	Callable interface	A34
6	ACID	E5
7	Three beans of same type, use one	C10
8	Second-highest salary SQL	E1

#	Question	Answer
9	Constructor calling another constructor	A19
10	Which declarations are correct (I/A/B/C)	A40
11	First-character count (guava, grapes...)	B13
12	HashMap Employee("Anshul") output	A28
13	Valid array declarations	A41
14	Composition & aggregation	A20
15	Java 8 features	A2
16	Method reference	B4
17	Default methods	A9
18	Intermediate operations	B5
19	HTTP 204	C17
20	Microservices in your project	D9 , E24
21	HashMap vs Hashtable	A24
22	Authentication layer in your project	D6
23	How JWT works	D5
24	Garbage collection	A16
25	Method hiding	A13
26	SOLID	A38
27	@Lazy	C7
28–29	@Component, component scanning	C8
30	Exception handling in Spring Boot	C20
31	JPA	C11
32	Bean scope	C6
33–35	@RequestParam / @RequestBody / @PathVariable	C13
36	@GeneratedValue	C12
37	20 records from 5th row	E8
38	INNER JOIN query	E9

Hexaware

#	Question	Answer
1	Authentication vs authorization	D1

#	Question	Answer
2	Exceptions in authn/authz	D3
3	INNER JOIN on identical tables	E7
4	Microservices, circuit breaker, communication	D8 , D11 , D10
5	Kafka and its flow	D17
6	Pre-authorization	D4
7	Stored procedures	E6
8	@Transactional	C30
9	Distributed caching & tracing	D16
10	Slow API investigation	D22
11	API Gateway	D14
12	API works internally, not for client	D21
13	One microservice you worked on	D18
14	Retry mechanism	D12
15	Microservice B is down	D13
16	Debugging a stream	B7
17	flatMap() example	B6
18	Architecture walkthrough	D9 , E18
19–20	Load Balancer / ALB	D15
21	Second-largest distinct element	B23
22	Composition in a real project	A20
23	Polymorphism real example	A21
24	Overriding & runtime polymorphism	A22
25	Authn/authz implementation	D2
26–27	@Cacheable, caching & DB performance	C31
28	Microservices architecture implementation	D9
29	How two microservices communicate	D10
30	Cache consistency across instances	C31
31	Highest receiver amount SQL	E10
32	Complete API request flow	D20
33	Direct vs shared-service integration	D19

G2Healthcare

#	Question	Answer
1	OOP concepts	A5
2–3	Abstraction + program	A6
4	volatile	A15
5	Concurrent collections / access	A32
6–7	wait() vs sleep()	A33
8	Spring Boot scopes	C6
9–11	@ConfigurationProperties (custom)	C21
12	Types of DI	C9
13	Request parameters	C13
14	Exception handling	C20
15	Handling microservices architecture	D9 , E24
16	Design patterns in microservices	D24
17–18	Angular introduction, roles	E12
19–20	Project architecture, codebase structure	E18
21, 23	Technologies & tools	E19
22	Precautions / best practices	E20
24	Roles and responsibilities	E16
25	Tell me about yourself	E15
26	Remove duplicates from a list of strings, keeping order	B14
27–30	Balanced brackets: approach, closing order, data structure	B45

Other rounds (Question List)

#	Question	Answer
1	Merge two arrays without duplication	B43
2	Even numbers using Java 8 streams	B11
3	Microservices vs monolithic	A39
4–6	Creating a thread; which method you call; Thread vs Runnable	A36 , A42
7	Bean life cycle	C5
8–9	REST API security layers; Spring Boot security	D7
10	execute() vs submit()	A35

#	Question	Answer
11	API delay from a stored procedure or SQL	D23
12	Bean creation internals; HashMap internals	C4 , A25
13	HashMap vs ConcurrentHashMap	A29
14	volatile	A15
15–16	Functional interfaces; adding a second method	A7 , A8
17–19	Stream API; intermediate and terminal operations	B5
20	Cloud services and tools	D25
—	Rotate an $N \times N$ matrix by $K \times 90^\circ$	B44
—	Count word frequency in a string	B42
—	Puzzle: 12 balls in 3 weighings	E26
—	Puzzle: remove half the water from a glass	E27
—	Stateless vs stateful (garbled line in source)	D26

Section F holds all 70 production scenarios in their original order (F1–F10 Java, F11–F20 Spring Boot, F21–F30 Hibernate/DB, F31–F40 Microservices, F41–F50 Kafka, F51–F60 Distributed systems, F61–F70 Production support).

A. Core Java

A1. What is the use of Docker?

- Docker packages an app with its runtime, libraries and config into an **image**. You run that image as a lightweight, isolated **container**. The goal is "build once, run anywhere".
- It removes "works on my machine" problems because dev, QA and prod all run the same image.
- Containers share the host OS kernel, so they start in seconds and use far fewer resources than VMs.
- In our microservices setup, each Spring Boot service has its own Dockerfile. We run local dependencies (Postgres, Kafka, Redis) with Docker Compose, and in production Kubernetes orchestrates the containers.

```
FROM eclipse-temurin:17-jre
COPY target/alert-service.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

A2. What are the main features introduced in Java 8, and why were they introduced?

Feature	Why
Lambda expressions	Pass behaviour as data, with far less anonymous-class boilerplate
Functional interfaces (<code>Function</code> , <code>Predicate</code> , <code>Supplier</code> , <code>Consumer</code>)	Give lambdas a target type
Stream API	Declarative filter/map/reduce over collections; easy to run in parallel
Default and static methods in interfaces	Add new methods to existing interfaces (e.g. <code>Collection.stream()</code>) without breaking implementors
<code>Optional</code>	Make "may be absent" explicit and reduce NPEs
<code>java.time</code> (JSR-310)	Replace the mutable, non-thread-safe <code>Date</code> / <code>Calendar</code>
Method references, <code>CompletableFuture</code> , <code>Metaspace</code> (replaced PermGen)	Shorter code, async composition, better memory handling

The overall reason was to bring functional-style programming to Java and make code more concise and easier to parallelise on multi-core machines.

A3. What are the important features introduced in Java 18, and how are they used?

Java 18 (March 2022) is a **non-LTS** release. In production we use the LTS versions, **17** or **21** (21 added virtual threads, record patterns and pattern matching for switch).

- **JEP 400, UTF-8 by default:** `Charset.defaultCharset()` is now UTF-8 on every OS, so `FileReader` and similar no longer behave differently on Windows.
- **JEP 408, Simple Web Server:** run `jwebserver -p 8000` to serve static files for quick testing (also available via the `SimpleFileServer` API).

- **JEP 413, @snippet in Javadoc:** code examples in Javadoc that can be validated.
- **JEP 418, internet-address resolution SPI:** plug in a custom DNS resolver in place of the built-in one used by `InetAddress`.
- **JEP 421, finalization deprecated for removal:** use `try-with-resources` or `Cleaner` instead of `finalize()`.
- Others: JEP 416 (core reflection reimplemented on method handles), Vector API (3rd incubator), Foreign Function and Memory API (2nd incubator), pattern matching for switch (2nd preview).

```
/**
 * {@snippet :
 *   var list = List.of(1, 2, 3);
 * }
 */
```

A4. Can we override the main method in Java? Explain why or why not.

- **No.** `main` is `static`, and static methods belong to the class, so they can't be overridden. A subclass can only declare its own `static main`, which **hides** the parent's version.
- We **can overload** `main` (e.g. `main(int x)`), but the JVM only calls `public static void main(String[] args)`.
- (Java 21+ preview features allow instance `main` methods, but that is a different topic.)

A5. Explain the main OOP concepts in Java.

The Four Pillars of Object-Oriented Programming

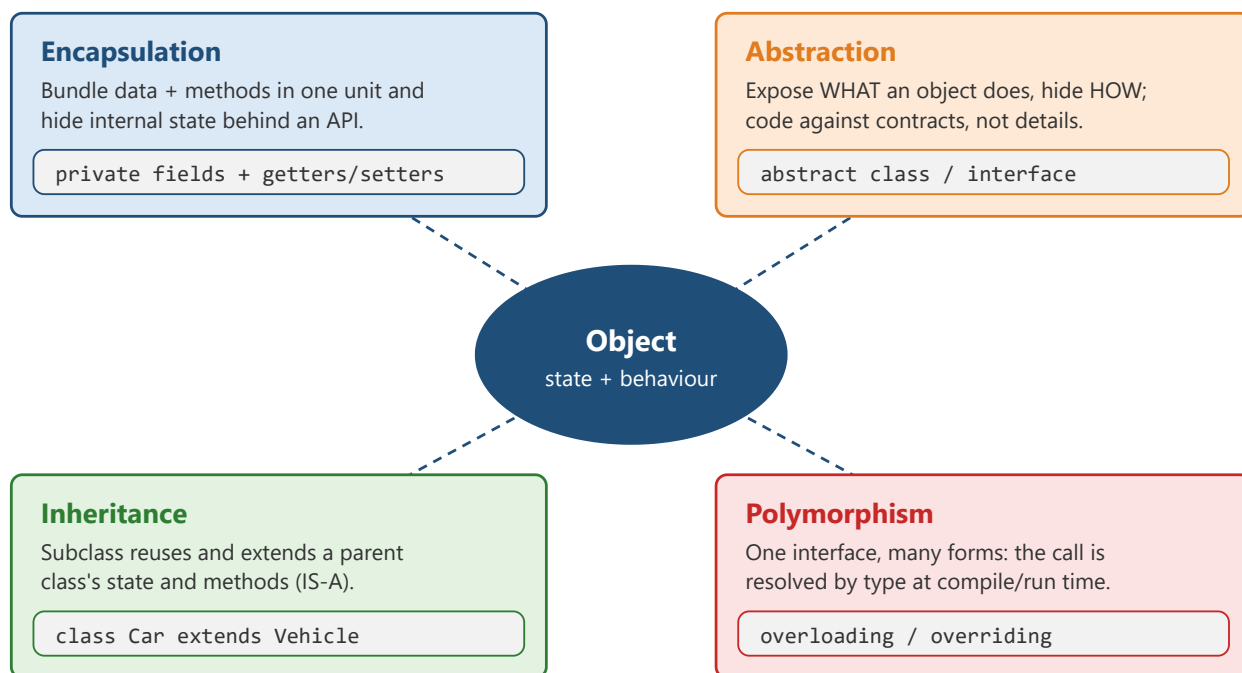


Figure 1: Four pillars of OOP

- **Encapsulation:** private fields with public getters and setters; the object controls its own state.
- **Abstraction:** expose *what* an object does and hide *how*, using abstract classes or interfaces.
- **Inheritance:** reuse and extend behaviour with `extends`, giving an IS-A relationship.
- **Polymorphism:** one interface, many forms. Overloading is resolved at compile time, overriding at runtime.
- Supporting ideas: association, aggregation and composition (HAS-A relationships).

A6. How do you achieve abstraction in Java? Write a class structure/program to demonstrate it.

- Use **abstract classes** for partial abstraction (they can have state, constructors and concrete methods).
- Use **interfaces** for full contracts (they can also have default and static methods). A class can implement many interfaces.

```
interface Notifier { void send(String msg); }

abstract class BaseNotifier implements Notifier {
    protected final String recipient;
    BaseNotifier(String recipient) { this.recipient = recipient; }
    public void log(String msg) { System.out.println("Sending to " + recipient + ": " + msg); }
}

class SmsNotifier extends BaseNotifier {
    SmsNotifier(String phone) { super(phone); }
    @Override public void send(String msg) { log(msg); /* call SMS gateway */ }
}

class EmailNotifier extends BaseNotifier {
    EmailNotifier(String email) { super(email); }
    @Override public void send(String msg) { log(msg); /* call SMTP */ }
}

// Client code depends only on the abstraction:
// Notifier n = new SmsNotifier("98xxxx"); n.send("Flood alert in Zone 3");
```

A7. Why were functional interfaces introduced, what is one, and how do you define one?

- A **functional interface** has **exactly one abstract method** (SAM). It can also have any number of default and static methods.
- They were introduced in Java 8 to give **lambdas and method references a target type**, so behaviour can be passed around without anonymous classes.
- Mark one with `@FunctionalInterface` (optional, but the compiler then enforces the single-abstract-method rule). Built-in examples: `Runnable`, `Comparator`, `Function`, `Predicate`.

```
@FunctionalInterface
interface Validator<T> {
    boolean validate(T t);
    default Validator<T> and(Validator<T> other) { return t -> validate(t) && other.validate(t); }
}

Validator<String> notEmpty = s -> !s.isBlank();
```

A8. If a functional interface already has one abstract method and another abstract method is added, what happens?

- With `@FunctionalInterface`: **compile error** ("multiple non-overriding abstract methods found").
- Without the annotation: the interface compiles but is **no longer functional**, so every lambda or method reference targeting it fails to compile.
- Exceptions: adding **default/static** methods is fine. Redefining a public `Object` method (e.g. `boolean equals(Object o)`) does not count as an abstract method.

A9. What is the use of default methods in interfaces?

- They let you **add new methods to an interface without breaking existing implementations**. This is backward compatibility, for example `List.forEach` and `Collection.stream()`.
- They provide a common implementation that classes can optionally override.
- Diamond rule: if two interfaces provide the same default method, the class **must override it** and can pick one with `A.super.method()`.

A10. What is the use of static methods in interfaces?

- They hold **utility/helper methods that belong to the interface**, so you don't need a separate `*Utils` class. Examples: `Comparator.comparing()`, `List.of()`.
- They are **not inherited** by implementing classes and must be called as `InterfaceName.method()`.

A11. Can we override private methods?

- **No**. Private methods aren't visible to subclasses, so they aren't inherited. A method with the same signature in a subclass is a **new, unrelated method**: no polymorphism, and `@Override` gives a compile error.

A12. Can we override static methods? Can we override a static method in an interface?

- **Class static methods**: no. A subclass static method with the same signature **hides** the parent's (method hiding). Which one runs is decided by the **reference type at compile time**.
- **Interface static methods**: no. They aren't even inherited by implementing classes. A class can declare a static method with the same signature, but it is completely unrelated, and you still call `I.m()`.
- Declaring an *instance* method that matches a parent's *static* method, or the other way round, is a compile error.

A13. What is method hiding?

When a subclass defines a **static** method with the same signature as a static method in its parent, the subclass method **hides** the parent's. The call is resolved by the **reference type**, not the object type.

```
class P { static void show() { System.out.println("P"); } }
class C extends P { static void show() { System.out.println("C"); } }
```



```
P p = new C();
p.show();    // prints "P" (static binding by reference type)
```

A14. What is serialization? How do you prevent a variable from being serialized?

- **Serialization** converts an object's state into a byte stream (`ObjectOutputStream`) so it can be stored or sent over the network. **Deserialization** rebuilds the object from the bytes.
- The class must implement `Serializable` (a marker interface). Declare a `serialVersionUID` for version control.
- To exclude a field, mark it `transient`. On deserialization it gets its default value (null/0/false). `static` fields are also not serialized, because they belong to the class.
- In microservices we usually serialize to JSON (Jackson) and exclude fields with `@JsonIgnore`.

```
class User implements Serializable {
    private static final long serialVersionUID = 1L;
    private String username;
    private transient String password;    // not serialized
}
```

A15. Explain the volatile keyword.

- `volatile` guarantees **visibility**: reads and writes go to main memory, so every thread sees the latest value. It also prevents reordering through a **happens-before** guarantee.
- It does **not** give atomicity. `count++` on a volatile field is still a race, so use `AtomicInteger` or `synchronized` for that.
- Typical uses: stop flags, and the `instance` field in double-checked locking singletons.

```
private volatile boolean running = true;
public void stop() { running = false; }    // other thread sees it immediately
```

A16. What is garbage collection and how is it used?

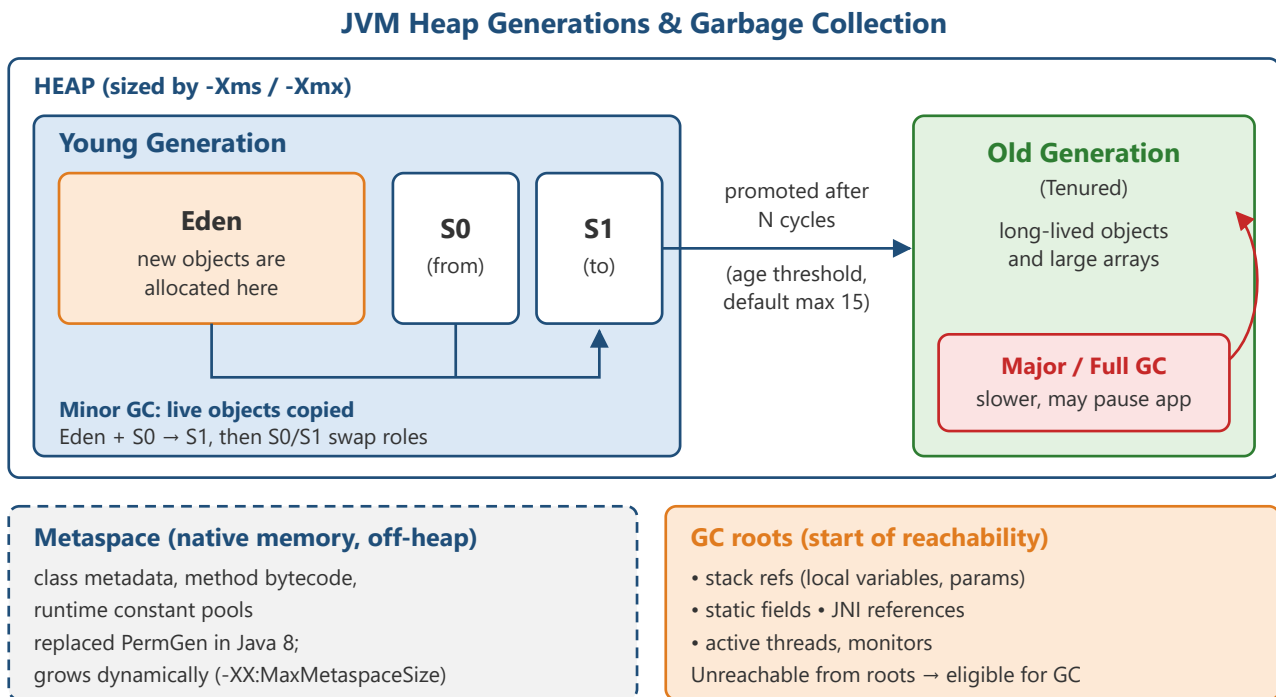


Figure 2: JVM heap generations and garbage collection

- The GC is the JVM's **automatic memory management**. It frees heap objects that are **no longer reachable** from GC roots (stack references, static fields, active threads).
- The heap is generational: **Young** (Eden plus survivor spaces, collected often by minor GC) and **Old** (long-lived objects, collected by major/full GC).
- Collectors: Serial, Parallel, **G1** (default since Java 9), and ZGC/Shenandoah for low pause times.
- We don't control when GC runs. `System.gc()` is only a hint. Our job is to avoid leaks (static collections, unclosed resources, listeners) and tune with flags like `-Xms`, `-Xmx` and `-XX:+UseG1GC`.

A17. Output of `System.out.println(Double.MIN_VALUE > 0.0d)` and why?

- It prints `true`.
- `Double.MIN_VALUE` is the **smallest positive** double (about `4.9E-324`), not the most negative one. The most negative double is `-Double.MAX_VALUE`.
- This differs from `Integer.MIN_VALUE`, which is negative, and that is the trap in the question.

A18. If you add a null to an empty Set, what is the size?

Set	Result
HashSet / LinkedHashSet	Allows one null, so size = 1

Set	Result
<code>TreeSet</code> (natural ordering)	<code>NullPointerException</code> , even on an empty set (Java 7+), because it calls <code>compareTo</code>
<code>TreeSet</code> with <code>Comparator.nullsFirst(...)</code>	Null is allowed, size = 1
<code>Set.of(null)</code> / <code>ConcurrentSkipListSet</code>	NPE, because nulls aren't allowed

A19. Can we call one constructor from another? How?

- **Yes:** `this(...)` calls another constructor in the **same class** (constructor chaining), and `super(...)` calls a **parent** constructor.
- The call **must be the first statement** in the constructor, so you can't use both `this()` and `super()` in the same constructor. Java 22+ previews relax this with "statements before super".
- If you write neither, the compiler inserts `super()` automatically.

```
class Sensor {
    private final String id; private final int interval;
    Sensor(String id) { this(id, 60); } // chains to the 2-arg constructor
    Sensor(String id, int interval) { super(); this.id = id; this.interval = interval; }
}
```

A20. Explain composition and aggregation with examples, and how you'd use composition in a real project.

	Aggregation	Composition
Relationship	Weak HAS-A	Strong HAS-A, "part-of"
Lifecycle	The part can live independently	The part dies with the owner
Example	<code>Department</code> has <code>Employee</code> s	<code>Car</code> has an <code>Engine</code> ; <code>Order</code> has <code>OrderLine</code> s

- In a real project I prefer **composition over inheritance**. Example: an `IncidentService` is *composed of* a `NotificationService`, an `IncidentRepository` and a `GeoService`, injected through the constructor. Spring wires them, I can swap implementations, and I can mock them in tests.

```
@Service
class IncidentService {
    private final IncidentRepository repo;
    private final NotificationService notifier;
    IncidentService(IncidentRepository repo, NotificationService notifier) {
        this.repo = repo; this.notifier = notifier;
    }
}
```

A21. What is polymorphism? Real project example.

- Polymorphism means one interface with many implementations, where the object decides the behaviour.
 - Compile-time:** method overloading.
 - Runtime:** method overriding plus dynamic dispatch.
- Project example: a `PaymentGateway` or `AlertChannel` interface with `SmsChannel`, `EmailChannel` and `PushChannel` implementations. Spring injects a `List<AlertChannel>`, and the code calls `channel.send(alert)` without knowing the concrete type. Adding a WhatsApp channel needs a new class but no change to the existing code (open/closed principle).

A22. Explain method overriding with a runtime polymorphism example.

- A subclass provides its own implementation of an inherited instance method. The signature must be the same, the return type the same or covariant, access the same or wider, and it can't throw broader checked exceptions.
- The JVM picks the method from the **actual object type at runtime** (dynamic method dispatch).

```
class Vehicle { void fare() { System.out.println("Base fare"); } }
class Bus extends Vehicle { @Override void fare() { System.out.println("Bus fare: 20"); } }
class Metro extends Vehicle { @Override void fare() { System.out.println("Metro fare: 40"); } }

Vehicle v = new Metro();
v.fare(); // "Metro fare: 40" (decided at runtime)
```

A23. Class A has a public method; class B overrides it as private. Will it work? Why?

- No, it's a compile error:** "attempting to assign weaker access privileges; was public".
- An overriding method **can't reduce visibility**. Otherwise `A a = new B(); a.m();` would call a private method through a public reference, which breaks the Liskov substitution principle. Widening access (protected to public) is allowed.

A24. Difference between HashMap and Hashtable.

	HashMap	Hashtable
Thread safety	Not synchronized	Synchronized (every method locks the whole table)
Performance	Faster	Slower
Null	1 null key, many null values	No null key or value (NPE)
Iteration	Fail-fast iterator	Enumerator (legacy) plus iterator
Since	Java 1.2, Collections Framework	Java 1.0, legacy

Use `HashMap` single-threaded and `ConcurrentHashMap` for concurrent access. `Hashtable` is effectively obsolete.

A25. Internal working of HashMap

HashMap Internals (Java 8+): put("Aa", 1)

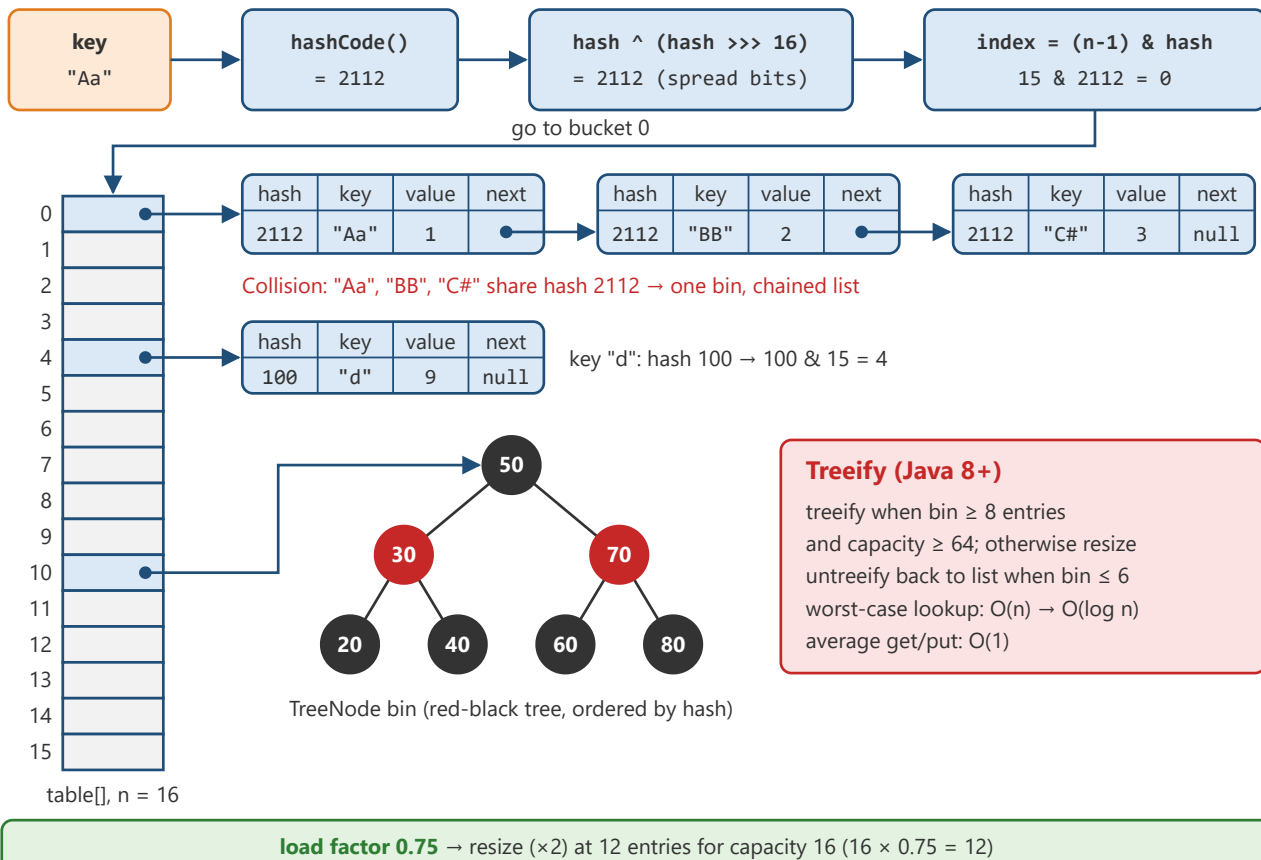


Figure 3: HashMap internals: hashing, buckets, chaining and treeification

- It is backed by an array of **buckets** (`Node<K,V>[] table`). The default capacity is **16** and is always a power of 2.
- `put(k, v)`:
 - $\text{hash} = h \wedge (h \ggg 16)$. This **hash spreading** mixes the high bits into the low bits.
 - Bucket index = $(n - 1) \& \text{hash}$.
 - Empty bucket: insert the node. Otherwise walk the bucket: if a key matches by `hash ==` and `equals()`, **replace the value**; if not, append a new node.
- Load factor 0.75**: when `size > capacity * 0.75` (e.g. 12 for capacity 16), it **resizes to double capacity**. Each node either stays at its index or moves to `index + oldCap`.
- Treeify**: a bucket's list becomes a **red-black tree** when it reaches **8 nodes** and the table has **at least 64** buckets. With a smaller table it resizes instead. A tree turns back into a list at 6 nodes. Worst-case lookup goes from $O(n)$ to $O(\log n)$.
- `get(k)` follows the same hash, then index, then `equals` path. Average time is $O(1)$.

A26. How are HashMap collisions handled?

- A collision means different keys land in the same bucket. HashMap uses **separate chaining**: the bucket holds a linked list of nodes, with new nodes appended at the tail since Java 8.
- Lookup inside a bucket compares the hash and then `equals()`.
- Since Java 8, a long chain (8 or more nodes, table of at least 64) is **converted to a red-black tree**. This keeps worst-case lookup at $O(\log n)$ and protects against hash-flooding attacks.
- A good `hashCode()` and resizing keep chains short in the first place.

A27. hashCode(), equals(), toString() in the context of HashMap.

- `hashCode()` decides **which bucket** a key goes to.
- `equals()` decides whether two keys in the same bucket are **the same key**, which means replace rather than add.
- Contract: if `a.equals(b)` then `a.hashCode() == b.hashCode()`. The reverse need not hold, and that is what a collision is. Always override both together, using the same fields.
- `toString()` has no effect on HashMap behaviour. It only controls how keys and values print (e.g. `{Employee[name=Anshul]=103}`). Without it you see `Employee@1b6d3586`.

A28. What happens when custom objects are used as HashMap keys?

```
Employee e1 = new Employee("Anshul");
Employee e2 = new Employee("Ravi");
Employee e3 = new Employee("Anshul");
HashMap<Employee, Integer> empM = new HashMap<>();
empM.put(e1, 101);
empM.put(e2, 102);
empM.put(e3, 103);
System.out.println(empM.size());
```

- **WITHOUT overriding equals / hashCode**: the `Object` versions use identity, so `e1` and `e3` are different keys. **Size = 3** (Anshul→101, Ravi→102, Anshul→103). Also, `empM.get(new Employee("Anshul"))` returns `null`.
- **WITH both overridden on name**: `e3` has the same hash and `equals(e1)` is true, so its value **replaces** `e1`'s. **Size = 2**: Anshul→103 (the original key object `e1` is kept, only the value changes) and Ravi→102.
- Overriding only one of the two breaks the contract and behaviour becomes unpredictable.

```
record Employee(String name) { } // a record generates equals/hashCode/toString from 'name'
```

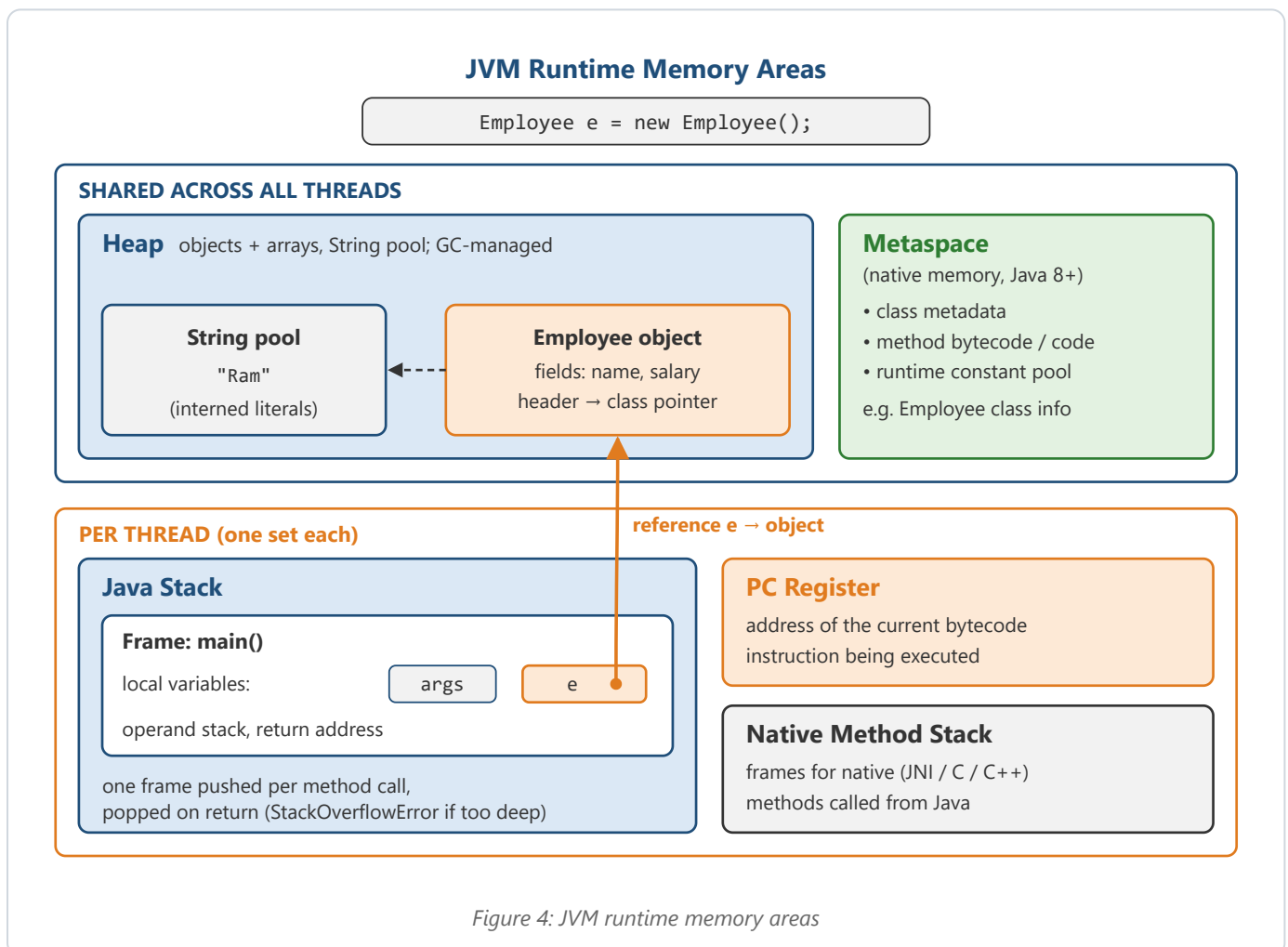
- **Mutable key pitfall**: if a key's `name` changes after `put`, its hash changes. The entry stays in the old bucket, so `get()` and `remove()` can no longer find it (an effective memory leak). Use **immutable keys** such as `String`, wrapper types or records.

A29. HashMap vs ConcurrentHashMap.

	HashMap	ConcurrentHashMap
Thread safety	No (concurrent puts can lose updates)	Yes
Locking	None	Java 8+: CAS for an empty bucket, synchronized on the bucket's first node otherwise. Reads are lock-free. Java 7 used segments.
Null	1 null key and null values allowed	No null keys or values (a null would be ambiguous with "absent")
Iterator	Fail-fast (<code>ConcurrentModificationException</code>)	Weakly consistent, never throws CME
Atomic operations	None	<code>putIfAbsent</code> , <code>computeIfAbsent</code> , <code>merge</code>

Use `ConcurrentHashMap` for shared caches and counters, e.g. `map.merge(sensorId, 1, Integer::sum)`.

A30. Java memory areas: Heap and Stack (plus Metaspace briefly).

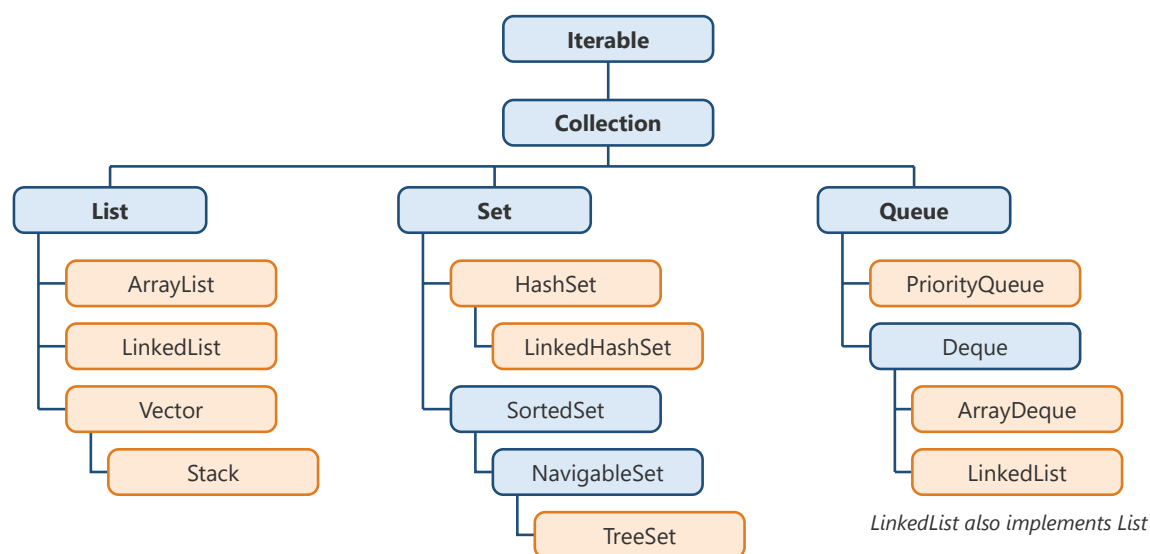


	Heap	Stack
Stores	Objects, instance variables, arrays, the String pool	Method frames: local variables, primitives, object references , partial results
Scope	Shared by all threads	One per thread
Managed by	Garbage collector (Young/Old generations)	Pushed and popped automatically per method call (LIFO)
Error	<code>OutOfMemoryError</code> : Java heap space	<code>StackOverflowError</code> (e.g. deep recursion)
Size flags	<code>-Xms</code> , <code>-Xmx</code>	<code>-Xss</code>

- **Metaspace** (Java 8+, replaced PermGen) holds class metadata in native memory. It is limited with `XX:MaxMetaspaceSize`.
- Other areas: the PC register and the native method stack.

A31. Explain the Java Collection Framework.

Java Collections Framework Hierarchy



Map hierarchy

(Map does not extend Collection)

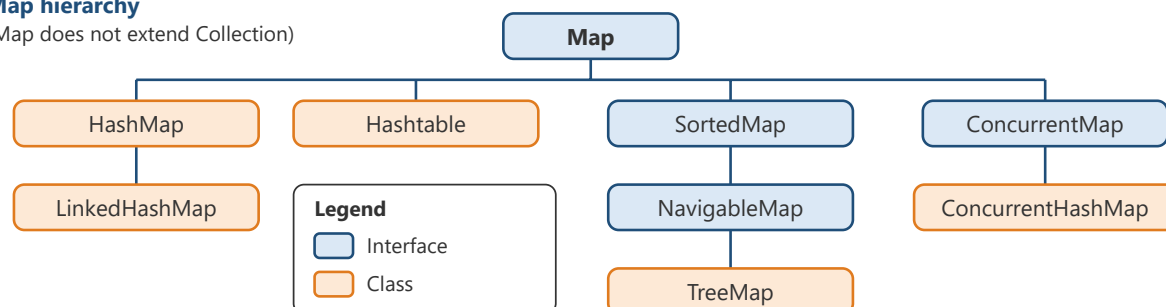


Figure 5: Java Collections Framework hierarchy

- It is a unified set of **interfaces, implementations and algorithms** for storing and processing groups of objects (in `java.util`).
- `Collection` branches into:
 - `List` (ordered, duplicates allowed): `ArrayList`, `LinkedList`
 - `Set` (unique elements): `HashSet`, `LinkedHashSet`, `TreeSet`
 - `Queue / Deque`: `PriorityQueue`, `ArrayDeque`
- `Map` (key-value) is a separate hierarchy: `HashMap`, `LinkedHashMap`, `TreeMap`.
- Utility classes: `Collections` (sort, unmodifiable, synchronized wrappers) and `Arrays`. `Iterator / ListIterator` handle traversal, `Comparable / Comparator` handle ordering.
- Concurrent implementations live in `java.util.concurrent`.

A32. Concurrent collections and concurrent access in Java.

- Plain collections aren't thread-safe. `Collections.synchronizedList/Map` locks the whole collection, which makes it a bottleneck.
- `java.util.concurrent` offers better options:
 - `ConcurrentHashMap`: fine-grained locking and atomic operations.
 - `CopyOnWriteArrayList / Set`: each write copies the array. Good when reads far outnumber writes (listener lists, config).
 - `BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`): producer-consumer with `put / take`.
 - `ConcurrentLinkedQueue`: lock-free, non-blocking.
 - `ConcurrentSkipListMap / Set`: sorted and concurrent.
- Their iterators are **weakly consistent** (no `ConcurrentModificationException`).
- For other shared state use `synchronized`, `ReentrantLock`, `atomics` (`AtomicInteger`, `LongAdder`) or `volatile`.

A33. `wait()` vs `sleep()`.

	<code>wait()</code>	<code>sleep()</code>
Class	<code>Object</code>	<code>Thread</code> (static)
Lock	Releases the monitor	Keeps any locks it holds
Where	Must be inside <code>synchronized</code> , otherwise <code>IllegalMonitorStateException</code>	Anywhere
Wakes up	<code>notify()</code> / <code>notifyAll()</code> , a timeout, or an interrupt	When the time elapses, or on interrupt
Purpose	Communication between threads	Pausing execution

Always call `wait()` in a `while (condition)` loop because of spurious wakeups.

A34. The Callable interface.

- Introduced in Java 5 (`java.util.concurrent`). It is like `Runnable` , but `call()` **returns a value** and **can throw checked exceptions**.
- Submit it to an `ExecutorService` and get back a `Future<V>` . Then use `get()` (blocking), `isDone()` and `cancel()` .

```
ExecutorService pool = Executors.newFixedThreadPool(4);
Callable<Integer> task = () -> sensorRepo.countActive();
Future<Integer> f = pool.submit(task);
Integer count = f.get();    // blocks; throws ExecutionException if the task failed
pool.shutdown();
```

A35. execute() vs submit().

	<code>execute(Runnable)</code>	<code>submit(Runnable/Callable)</code>
Defined in	<code>Executor</code>	<code>ExecutorService</code>
Returns	<code>void</code>	<code>Future<?></code> / <code>Future<T></code>
Accepts	Only <code>Runnable</code>	<code>Runnable</code> or <code>Callable</code>
Exceptions	Thrown on the worker thread (goes to the <code>UncaughtExceptionHandler</code>)	Captured in the Future and only rethrown as <code>ExecutionException</code> on <code>get()</code> , so they are silently lost if you never call <code>get()</code>

A36. How do you create a thread? Extending Thread vs implementing Runnable.

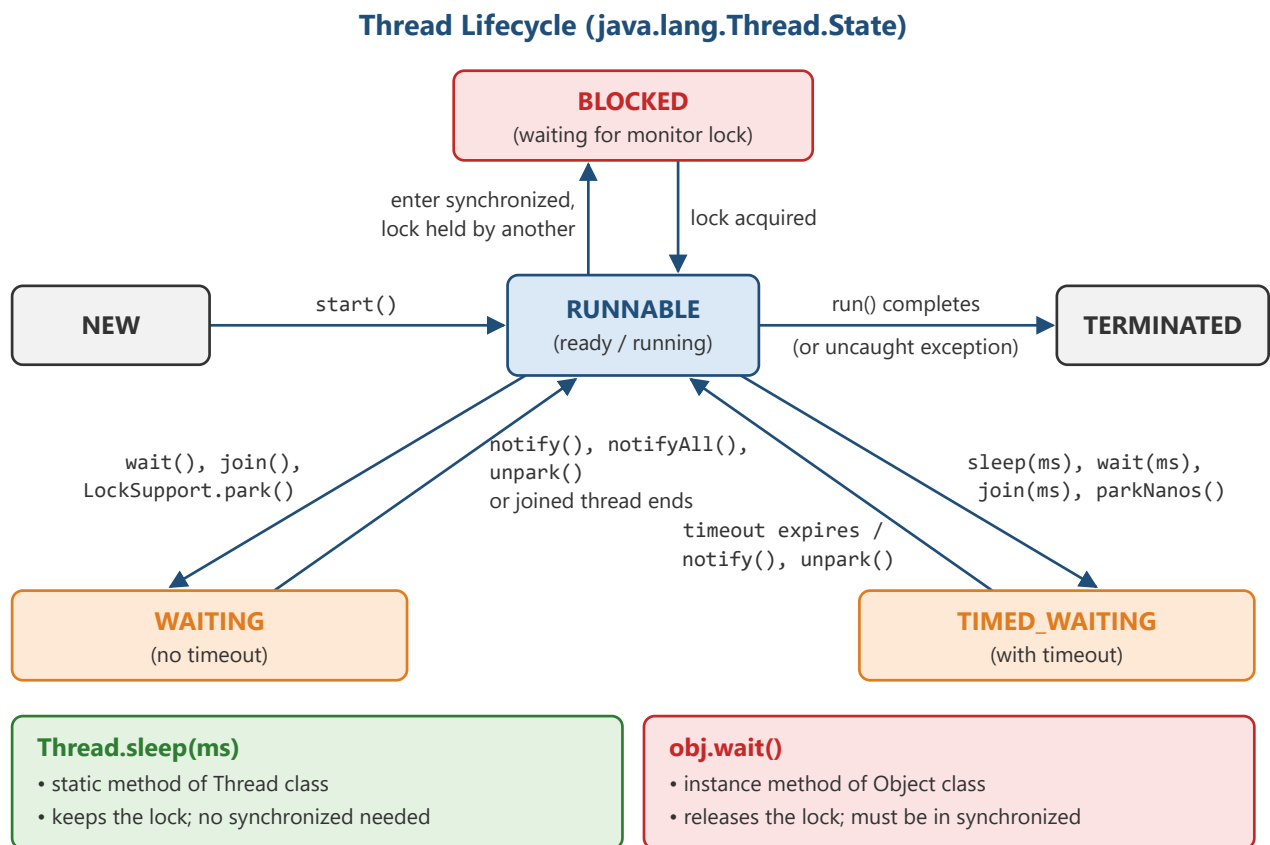


Figure 6: Java thread lifecycle states

- Ways to create one: extend `Thread`, implement `Runnable` or `Callable`, use an `ExecutorService` (preferred in real code), use `CompletableFuture`, or in Java 21 use virtual threads (`Thread.ofVirtual()`).

Extend <code>Thread</code>	Implement <code>Runnable</code>
Uses up your only superclass	Class can still extend something else
Task and thread are tightly coupled	Task is separate from the execution mechanism, so it can be reused with executors
Each object is its own thread	One <code>Runnable</code> can be shared by many threads

```
Runnable task = () -> System.out.println("Running in " + Thread.currentThread().getName());
new Thread(task).start();           // start(), not run(); run() executes on the current thread
```

A37. How did you connect Java with Python?

- **Main answer (REST):** "The Python ML model (e.g. traffic or crowd prediction) ran as a **Flask/FastAPI microservice**. Our Spring Boot service called it over REST using `WebClient` / `RestTemplate` (or `Feign`), with JSON request and response, timeouts, and retry/circuit breaker (`Resilience4j`). The two sides were decoupled and deployed as separate Docker containers."
- Other options:

- **Kafka / message queue:** Java publishes events and the Python consumer processes them and publishes results. Good for asynchronous, high-volume streams such as sensor data.
- **ProcessBuilder :** run a Python script from Java and read its stdout. Fine for batch jobs, but fragile and not scalable.
- **gRPC:** a typed, fast RPC contract between the two services.
- **Jython / GraalPy:** run Python inside the JVM. Jython only supports Python 2.7, so it is rarely used.

```
PredictionResponse res = webClient.post().uri("http://ml-service:8000/predict")
    .bodyValue(request).retrieve().bodyToMono(PredictionResponse.class).block();
```

A38. What is the SOLID principle?

- **S, Single Responsibility:** a class should have one reason to change. For example, `ReportService` shouldn't also send emails.
- **O, Open/Closed:** open for extension, closed for modification. Add a new `AlertChannel` implementation instead of editing a big if/else.
- **L, Liskov Substitution:** subtypes must work wherever their base type is used, without breaking behaviour.
- **I, Interface Segregation:** prefer many small, focused interfaces over one large interface.
- **D, Dependency Inversion:** depend on abstractions, not concrete classes. Spring dependency injection with interface-typed constructor parameters is a direct example.

A39. Monolithic vs microservice architecture.

- **Monolith:** the whole application (UI, business logic, data access) is built and deployed as **one unit** with one shared database.
- **Microservices:** the application is split into **small, independently deployable services**, each built around a business capability with its own database. They communicate over REST, gRPC or messaging, typically behind an API gateway with service discovery.

	Monolith	Microservices
Deployment	Everything redeployed together	Each service deployed independently
Scaling	Scale the whole app	Scale only the busy service
Tech stack	One stack	Can differ per service
Failure	One bug can bring everything down	Faults isolated (circuit breakers)
Data	One shared database	Database per service, eventual consistency
Complexity	Simple to start, test and debug	Distributed-system overhead: network, tracing, transactions (Saga)
Best for	Small teams, MVPs	Large, evolving systems (e.g. Smart City modules)

A40. Which declarations are correct and why?

```
interface I { } class A implements I { } class B extends A { } class C extends B { }
```

Statement	Result	Reason
<code>A a = new I();</code>	Compile error	An interface can't be instantiated
<code>I b = new B();</code>	OK	B IS-A A IS-A I (upcast)
<code>C c = new B();</code>	Compile error	A parent object can't be assigned to a child reference without a cast. <code>(C) new B()</code> would compile but throw <code>ClassCastException</code> at runtime
<code>C c2 = new A();</code>	Compile error	Same reason: A is not a C
<code>B b = new C();</code>	Valid type-wise (C IS-A B, upcast), but	<code>b</code> was already declared as <code>I b</code> in the same scope, so this is a duplicate variable compile error . Renamed (e.g. <code>B b2 = new C();</code>) it is fine

Rule: a reference of a parent type can hold a child object (upcast). The reverse needs an explicit downcast, which is only safe if the actual object really is that type.

A41. Which array declarations are valid and why?

Declaration	Result	Reason
<code>int[] n1 = new int[0];</code>	Valid	A zero-length array is legal (length 0). Useful for returning "empty" instead of null
<code>boolean[] n2 = new boolean[-200];</code>	Compiles , but throws NegativeArraySizeException at runtime	The size is only checked at runtime
<code>double[] n3 = new double[2241423798];</code>	Compile error : "integer number too large"	The literal exceeds <code>Integer.MAX_VALUE</code> (2147483647). Adding <code>L</code> still fails, because the array size must be an <code>int</code> (lossy conversion from long)
<code>char[] ch = new char[20];</code>	Valid	20 elements, each defaulting to <code>'\u0000'</code>

A42. Thread start() vs run(): which method do you call and why?

- Call `start()`. It asks the JVM to create a new thread, and the JVM then calls `run()` on that thread.
- If you call `run()` yourself, it is just a normal method call. It runs on the **current** thread, so nothing happens in parallel.
- If you call `start()` twice on the same `Thread`, you get `IllegalThreadStateException`. A thread can be started only once.

```
public class StartVsRun {
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(() -> System.out.println("Running in: " +
            Thread.currentThread().getName()));
        t.run();    // plain method call on main
    }
}
```

```
        t.start(); // new thread
        t.join();
        // t.start(); // would throw IllegalStateException
    }
}
// Output:
// Running in: main
// Running in: Thread-0
```

B. Java 8, Streams, Optional, Lambdas & Coding Problems

B1. Explain Java 8 features.

Java 8 added **lambdas**, **functional interfaces** (`java.util.function`), **method references**, the **Stream API**, **Optional**, **default and static methods in interfaces**, the **new Date/Time API** (`java.time`), `CompletableFuture`, `Collectors`, `forEach` / `removeIf` / `computeIfAbsent` on collections, `StringJoiner`, and Nashorn. Together they let you write Java in a declarative, functional style. (Section A covers each one in more detail.)

B2. What is a lambda expression?

A lambda is an anonymous function, `(params) -> body`, that provides the implementation of a functional interface's single abstract method. It replaces boilerplate anonymous inner classes. It can only capture local variables that are effectively final. Unlike an anonymous class, `this` inside a lambda refers to the enclosing instance.

```
Comparator<String> byLen = (a, b) -> Integer.compare(a.length(), b.length());
Runnable r = () -> System.out.println("run");
```

B3. Functional interfaces with respect to lambdas.

A functional interface has exactly one abstract method (`@FunctionalInterface` is optional and makes the compiler enforce it). A lambda is an instance of one.

Interface	Method	Shape
<code>Predicate<T></code>	<code>test(T)</code>	$T \rightarrow \text{boolean}$
<code>Function<T, R></code>	<code>apply(T)</code>	$T \rightarrow R$
<code>Consumer<T></code>	<code>accept(T)</code>	$T \rightarrow \text{void}$
<code>Supplier<T></code>	<code>get()</code>	$() \rightarrow T$
<code>BiFunction<T, U, R></code>	<code>apply(T, U)</code>	$(T, U) \rightarrow R$
<code>UnaryOperator<T></code>	<code>apply(T)</code>	$T \rightarrow T$ (extends <code>Function</code>)

```
Predicate<Integer> isEven = n -> n % 2 == 0;
Function<String, Integer> len = String::length;
Consumer<String> print = System.out::println;
Supplier<List<String>> sup = ArrayList::new;
BiFunction<Integer, Integer, Integer> add = Integer::sum;
UnaryOperator<String> upper = String::toUpperCase;
```

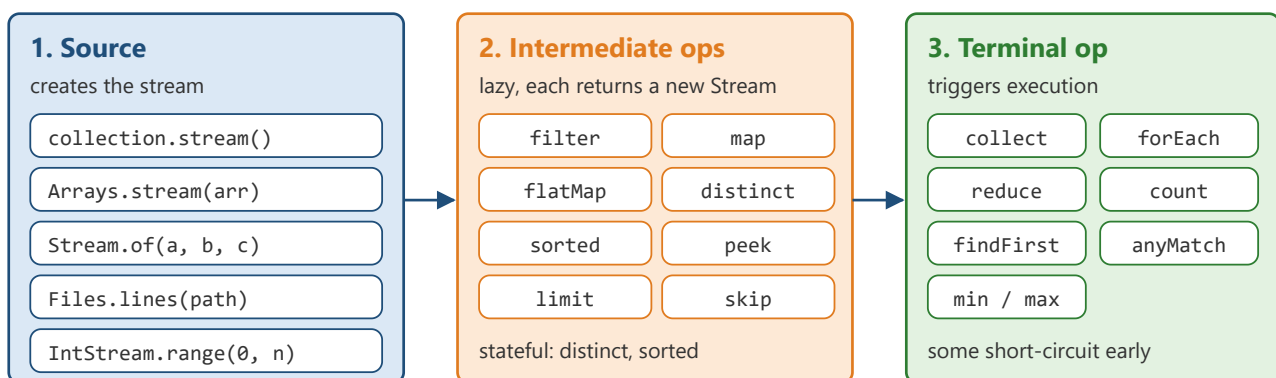
B4. What is a method reference? (4 kinds)

A method reference is shorthand for a lambda that only calls an existing method.

1. Static method: `Integer::parseInt ( s -> Integer.parseInt(s) )`
2. Instance method of a particular object: `System.out::println`
3. Instance method of an arbitrary object of a type: `String::toUpperCase ( s -> s.toUpperCase() )`
4. Constructor: `ArrayList::new`

B5. Stream API methods: intermediate vs terminal.

Stream Pipeline: Source → Intermediate (lazy) → Terminal



Nothing runs until the terminal op is called; a stream can be consumed only once.

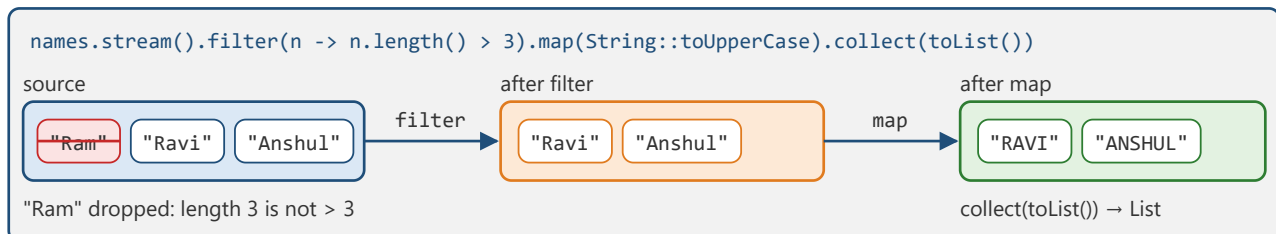


Figure 7: Java Stream pipeline

- **Intermediate** operations are lazy and return a Stream: `filter`, `map`, `flatMap`, `distinct`, `sorted`, `peek`, `limit`, `skip`, `mapToInt/Obj`, `boxed`, `takeWhile`, `dropWhile`.
- **Terminal** operations trigger execution and close the stream: `forEach`, `collect`, `reduce`, `count`, `min`, `max`, `findFirst`, `findAny`, `anyMatch`, `allMatch`, `noneMatch`, `toArray`, `toList` (16+).
- **Laziness**: nothing runs until a terminal operation is called. Elements then flow through the pipeline one at a time (vertical processing).
- **Short-circuiting** operations can stop early: `limit`, `findFirst`, `findAny`, `anyMatch`, `allMatch`, `noneMatch`, `takeWhile`. These make infinite streams usable.

B6. map() vs flatMap()

map() vs flatMap()

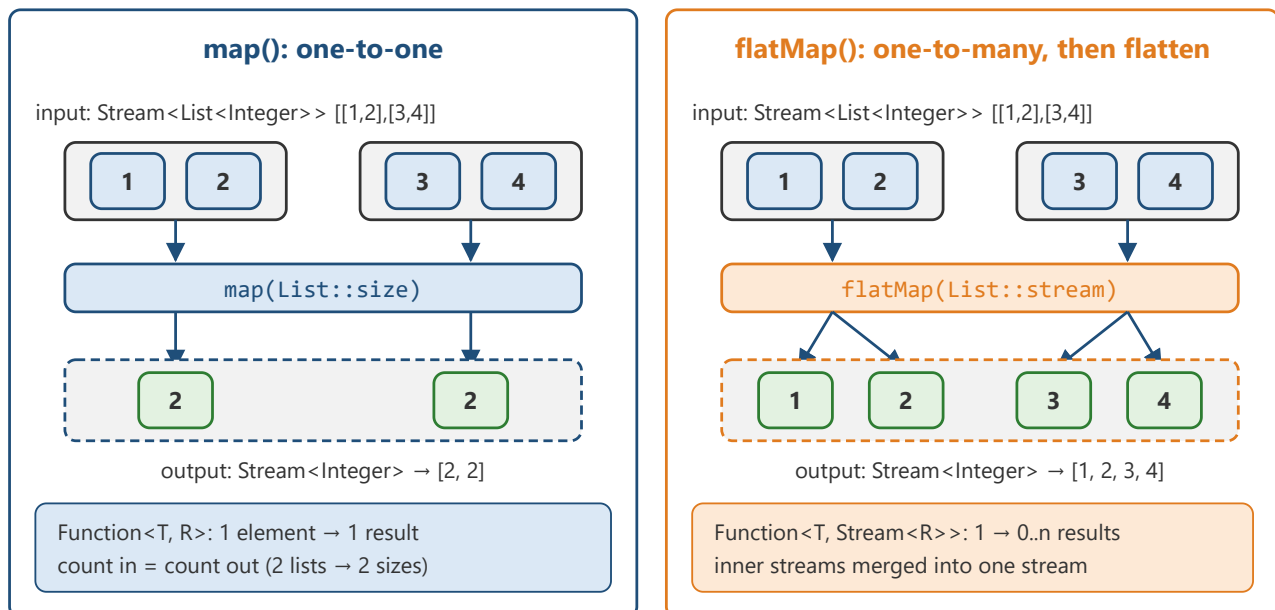


Figure 8: map vs flatMap in Java Streams

`map` is a one-to-one transform (`Stream<T> → Stream<R>`). `flatMap` is one-to-many: each element maps to a `Stream`, and those streams are flattened into one (`Stream<List<T>> → Stream<T>`).

```
List<String> words = List.of("hi", "yo");
words.stream().map(String::length).toList(); // [2, 2]
words.stream().flatMap(w -> w.chars().mapToObj(c -> (char) c)).toList(); // [h, i, y, o]
```

B7. How do you debug a Java Stream?

Use `peek()` to log elements between stages, or break the pipeline into variables and set an IDE breakpoint inside the lambda. IntelliJ's **Stream Trace** debugger also works. Don't put business logic in `peek`, because it may not run at all if the terminal operation can skip elements (e.g. `count()` on a sized stream).

```
List.of(1, 2, 3, 4).stream()
    .peek(n -> System.out.println("in: " + n))
    .filter(n -> n % 2 == 0)
    .peek(n -> System.out.println("even: " + n))
    .map(n -> n * 10)
    .toList();
// Output: in: 1, in: 2, even: 2, in: 3, in: 4, even: 4 -> [20, 40]
```

B8. Optional for null-safe access; chaining with streams.

`Optional<T>` is a container that may or may not hold a value. Use it as a return type to force callers to handle the "absent" case, but not for fields or parameters. `map` transforms the value, `flatMap` is for functions that

return an Optional, and `filter`, `orElse(default)` (always evaluated), `orElseGet(supplier)` (lazy), `orElseThrow()`, `ifPresentOrElse`, and `Optional.stream()` (Java 9; gives 0 or 1 elements) cover the rest.

```
String city = Optional.ofNullable(user)
    .map(User::getAddress)
    .map(Address::getCity)
    .orElseGet(() -> "UNKNOWN");

List<String> emails = ids.stream()
    .map(repo::findEmail)           // Stream<Optional<String>>
    .flatMap(Optional::stream)     // keep only present values
    .toList();
```

B9. Sequential vs parallel streams.

A sequential stream runs on a single thread. A parallel stream (`parallelStream()` / `.parallel()`) splits the work across the common `ForkJoinPool` (roughly cores – 1 threads). Parallel helps for large, CPU-bound, stateless work on sources that split easily (`ArrayList`, arrays). It hurts for small data, I/O, `LinkedList`, or anything order-dependent or synchronized. Lambdas must be stateless and side-effect free. Use `forEachOrdered` if output order matters. `findAny` is cheaper than `findFirst` in parallel.

B10. Count duplicate elements in an array.

```
int[] arr = {1, 2, 3, 2, 4, 1, 2};
Map<Integer, Long> dups = Arrays.stream(arr).boxed()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream().filter(e -> e.getValue() > 1)
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
System.out.println(dups);
// Output: {1=2, 2=3}
```

B11. Find even numbers from a list.

```
List<Integer> evens = List.of(1, 2, 3, 4, 5, 6).stream().filter(n -> n % 2 == 0).toList();
// Output: [2, 4, 6]
```

B12. Occurrence of a character in a string.

```
String s = "programming";
long g = s.chars().filter(c -> c == 'g').count();           // 2
Map<Character, Long> freq = s.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new,
    Collectors.counting()));
// Output: 2, {p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}
```

B13. Count first-character occurrences in a list of strings.

```
Map<Character, Long> m = List.of("guava", "grapes", "apple", "orange").stream()
    .collect(Collectors.groupingBy(s -> s.charAt(0), LinkedHashMap::new, Collectors.counting()));
// Output: {g=2, a=1, o=1}
```

B14. Remove duplicates while maintaining order.

```
List<Integer> r = List.of(3, 1, 3, 2, 1).stream().distinct().toList();
// or: new ArrayList<>(new LinkedHashSet<>(list))
// Output: [3, 1, 2]
```

B15. Remove null and empty strings from a list.

```
List<String> in = Arrays.asList("a", null, "", "b", " ");
List<String> out = in.stream().filter(s -> s != null && !s.isBlank()).toList();
// Output: [a, b]
```

B16. Convert list of strings to uppercase.

```
List<String> up = List.of("java", "spring").stream().map(String::toUpperCase).toList();
// Output: [JAVA, SPRING]
```

B17. Join strings into a comma-separated value.

```
String csv = List.of("a", "b", "c").stream().collect(Collectors.joining(","));
// Collectors.joining(" ", "[", "]") adds a delimiter, prefix and suffix; String.join(",", list) also
works
// Output: a,b,c
```

B18. Sum all numbers using reduce().

```
int sum = List.of(1, 2, 3, 4, 5).stream().reduce(0, Integer::sum);
// alternative: mapToInt(Integer::intValue).sum()
// Output: 15
```

B19. List → Map<String,Integer> (string → length).

```
Map<String, Integer> m = List.of("java", "spring").stream()
    .collect(Collectors.toMap(Function.identity(), String::length));
// Output: {java=4, spring=6} (HashMap: order not guaranteed)
```

B20. Group employees by department.

```
record Employee(String name, String dept, double salary) {}
List<Employee> emps = List.of(new Employee("A", "IT", 50000), new Employee("B", "HR", 40000),
                             new Employee("C", "IT", 70000));
Map<String, List<String>> byDept = emps.stream()
    .collect(Collectors.groupingBy(Employee::dept,
        Collectors.mapping(Employee::name, Collectors.toList())));
// Output: {HR=[B], IT=[A, C]}
```

B21. Sort employees by salary.

```
emps.stream().sorted(Comparator.comparingDouble(Employee::salary)).toList();           // asc
emps.stream().sorted(Comparator.comparingDouble(Employee::salary).reversed()).toList(); // desc
emps.stream().sorted(Comparator.comparingDouble(Employee::salary).reversed()
    .thenComparing(Employee::name)).toList();                                           // desc, then
name
// Output (desc): [C(70000), A(50000), B(40000)]
```

B22. Second-highest number in a list (stream).

```
Optional<Integer> second = List.of(5, 9, 3, 9, 7).stream()
    .distinct().sorted(Comparator.reverseOrder()).skip(1).findFirst();
// Output: Optional[7]
```

B23. Second-largest distinct element, single pass O(n).

```
static OptionalInt secondLargest(int[] a) {
    long first = Long.MIN_VALUE, second = Long.MIN_VALUE;
    for (int x : a) {
        if (x > first) { second = first; first = x; }
        else if (x < first && x > second) second = x;
    }
    return second == Long.MIN_VALUE ? OptionalInt.empty() : OptionalInt.of((int) second);
}
// secondLargest(new int[]{5, 9, 3, 9, 7}) -> OptionalInt[7]; {4, 4} or {} -> OptionalInt.empty
```

Using `long` sentinels means no real `int` value (including `Integer.MIN_VALUE`) can be mistaken for "not found".

B24. Flatten a list of lists with flatMap.

```
List<Integer> flat = List.of(List.of(1, 2), List.of(3), List.of(4, 5)).stream()
    .flatMap(List::stream).toList();
// Output: [1, 2, 3, 4, 5]
```

B25. Count elements by category (groupingBy + counting).

```
Map<String, Long> count = emps.stream()
    .collect(Collectors.groupingBy(Employee::dept, Collectors.counting()));
// Output: {HR=1, IT=2}
```

B26. Sort a map by values.

```
Map<String, Integer> scores = Map.of("a", 3, "b", 1, "c", 2);
Map<String, Integer> sorted = scores.entrySet().stream()
    .sorted(Map.Entry.comparingByValue()) // .comparingByValue(Comparator.reverseOrder()) for
desc
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue, (x, y) -> x,
LinkedHashMap::new));
// Output: {b=1, c=2, a=3}
```

B27. Filter a list based on multiple conditions.

```
Predicate<Employee> inIT = e -> e.dept().equals("IT");
Predicate<Employee> highPaid = e -> e.salary() > 60000;
List<String> r = emps.stream().filter(inIT.and(highPaid)).map(Employee::name).toList();
// can also chain .filter(a).filter(b), or use .or() / .negate()
// Output: [C]
```

B28. Partition into even/odd with partitioningBy.

```
Map<Boolean, List<Integer>> p = List.of(1, 2, 3, 4, 5).stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
// Output: {false=[1, 3, 5], true=[2, 4]}
```

B29. Handle duplicate keys in Collectors.toMap().

Without a merge function, a duplicate key throws `IllegalStateException`. The third argument resolves collisions.

```
Map<String, Double> totalByDept = emps.stream()
    .collect(Collectors.toMap(Employee::dept, Employee::salary, Double::sum));
// keep the first value: (a, b) -> a ; keep the last: (a, b) -> b
// Output: {HR=40000.0, IT=120000.0}
```

B30. Caching with ConcurrentHashMap.computeIfAbsent().

`computeIfAbsent` is atomic per key: the loader runs at most once for each missing key, even when many threads call it at once. Keep the loader short, and never update the same map recursively from inside it (that throws `IllegalStateException`).

```
private final Map<Long, User> cache = new ConcurrentHashMap<>();
User getUser(long id) {
    return cache.computeIfAbsent(id, k -> userRepository.findById(k)); // DB hit only on first call
}
```

B31. Custom collector to join strings in uppercase.

```
Collector<String, StringJoiner, String> upperJoin = Collector.of(
    () -> new StringJoiner(", "), // supplier
    (j, s) -> j.add(s.toUpperCase()), // accumulator
    StringJoiner::merge, // combiner (parallel)
    StringJoiner::toString); // finisher
String r = List.of("a", "b", "c").stream().collect(upperJoin);
// Output: A, B, C
```

B32. Stream a large file line by line efficiently.

`Files.lines` reads lazily through a buffered reader, so the whole file is never held in memory. Use try-with-resources so the file handle is closed.

```
try (Stream<String> lines = Files.lines(Path.of("app.log"))) {
    long errors = lines.filter(l -> l.contains("ERROR")).count();
    System.out.println(errors);
} catch (IOException e) { throw new UncheckedIOException(e); }
// Output: number of ERROR lines, e.g. 42
```

B33. Generic method using functional interfaces.

```
static <T, R> List<R> filterAndMap(List<T> list, Predicate<? super T> p, Function<? super T, ? extends R> f) {
    return list.stream().filter(p).<R>map(f).toList();
}
// filterAndMap(List.of("a", "bb", "ccc"), s -> s.length() > 1, String::length)
// Output: [2, 3]
```

B34. Stream pipeline: CSV → filter → map → collect to a record.

```
record Person(String name, int age) {}
List<String> csv = List.of("name,age", "Ravi,30", "Asha,17", "John,25");
List<Person> adults = csv.stream()
    .skip(1) // header
    .map(line -> line.split(","))
    .map(f -> new Person(f[0].trim(), Integer.parseInt(f[1].trim())))
    .filter(p -> p.age() >= 18)
    .toList();
// Output: [Person[name=Ravi, age=30], Person[name=John, age=25]]
```

B35. Longest string.

```
Optional<String> longest = List.of("java", "spring", "kafka").stream()
    .max(Comparator.comparingInt(String::length));
// Output: Optional[spring]
```

B36. Convert "this is test" into "THIS IS TEST" using Stream API.

```
String r = Arrays.stream("this is test".split(" "))
    .map(String::toUpperCase).collect(Collectors.joining(" "));
// Output: THIS IS TEST
```

B37. "Get the highest substring"

(a) Lexicographically largest substring. It is always a suffix: any substring is $\leq$ the suffix that starts at the same index, because a prefix is smaller than the longer string.

```
String s = "banana";
String max = IntStream.range(0, s.length()).mapToObj(s::substring)
    .max(Comparator.naturalOrder()).orElse("");
// Output: nana
```

(b) Longest substring without repeating characters. Sliding window, $O(n)$.

```
static String longestUnique(String s) {
    Map<Character, Integer> last = new HashMap<>();
    int start = 0, bestStart = 0, bestLen = 0;
    for (int i = 0; i < s.length(); i++) {
        Integer prev = last.put(s.charAt(i), i);
        if (prev != null && prev >= start) start = prev + 1;
        if (i - start + 1 > bestLen) { bestLen = i - start + 1; bestStart = start; }
    }
    return s.substring(bestStart, bestStart + bestLen);
}
// longestUnique("abcabcbb") -> Output: abc (length 3)
```

B38. First non-repeating character in a string.

```
Optional<Character> c = "swiss".chars().mapToObj(ch -> (char) ch)
    .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new,
Collectors.counting()))
    .entrySet().stream().filter(e -> e.getValue() == 1)
    .map(Map.Entry::getKey).findFirst();
// Output: Optional[w]
```

B39. Remove duplicate letters, smallest lexicographic result (LeetCode 316).

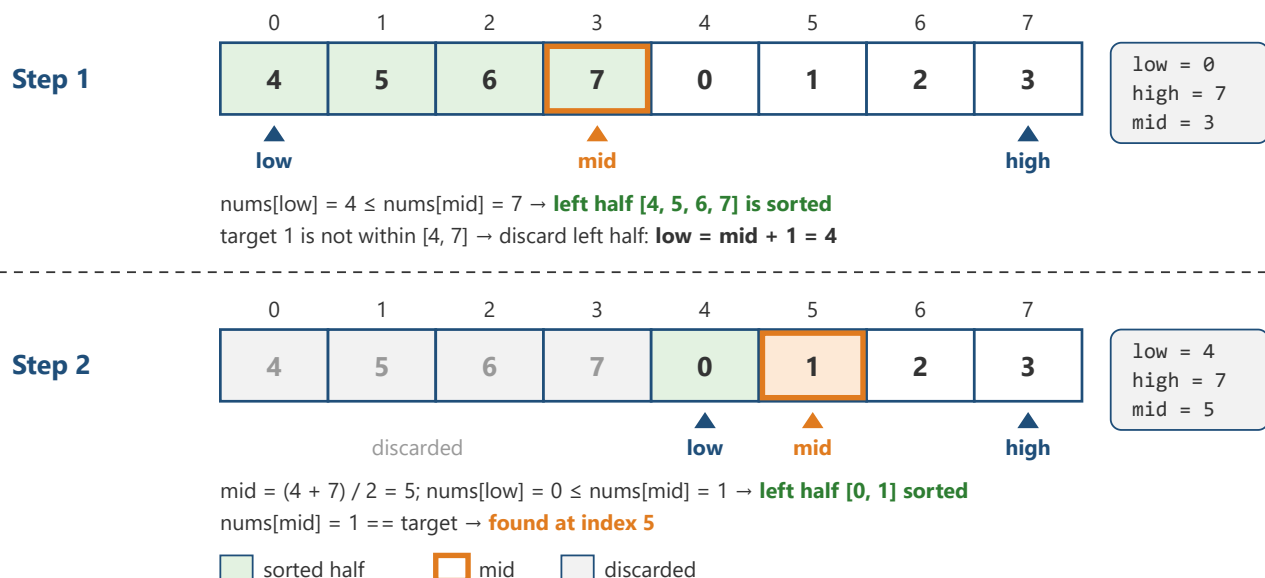
Greedy with a monotonic stack. Pop a larger character off the top if it appears again later. Skip characters already on the stack. $O(n)$.

```
static String removeDuplicateLetters(String s) {
    int[] last = new int[26];
    for (int i = 0; i < s.length(); i++) last[s.charAt(i) - 'a'] = i;
    boolean[] inStack = new boolean[26];
    StringBuilder st = new StringBuilder(); // used as a stack
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if (inStack[c - 'a']) continue;
        while (st.length() > 0 && st.charAt(st.length() - 1) > c
            && last[st.charAt(st.length() - 1) - 'a'] > i) {
            inStack[st.charAt(st.length() - 1) - 'a'] = false;
            st.deleteCharAt(st.length() - 1);
        }
        st.append(c);
        inStack[c - 'a'] = true;
    }
    return st.toString();
}
// "bcabc" -> abc ; "cbacdcbc" -> acdb
```

B40. Search in a rotated sorted array (4,5,6,7,0,1,2,3).

Search in Rotated Sorted Array (Binary Search)

nums = [4, 5, 6, 7, 0, 1, 2, 3], target = 1



$O(\log n)$: one half is always sorted; if target lies in it search there, else search the other half

Figure 9: Binary search in a rotated sorted array

Modified binary search, $O(\log n)$. At every step one half is sorted. If the target lies within that sorted half, search there; otherwise search the other half.

```
static int search(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[lo] <= a[mid]) { // left half sorted
            if (target >= a[lo] && target < a[mid]) hi = mid - 1; else lo = mid + 1;
        } else { // right half sorted
            if (target > a[mid] && target <= a[hi]) lo = mid + 1; else hi = mid - 1;
        }
    }
    return -1;
}
// search(new int[]{4,5,6,7,0,1,2,3}, 0) -> Output: 4
```

B41. Merge Sort, Quick Sort, Bubble Sort, Linear Search, Binary Search.

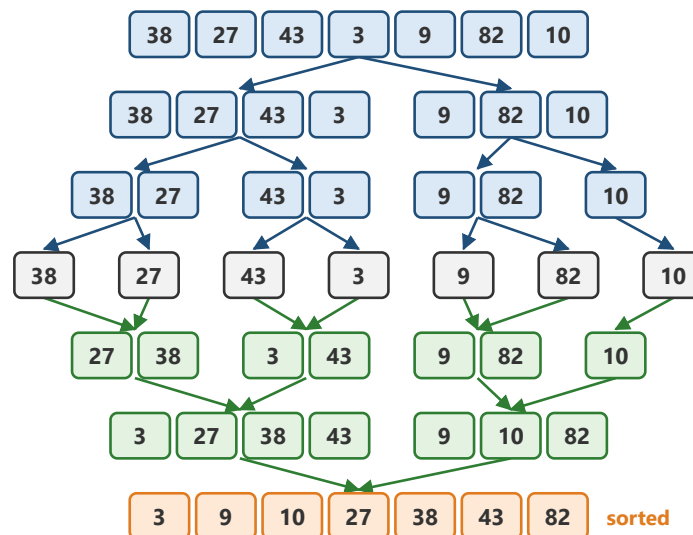
Merge Sort: Divide & Conquer on [38, 27, 43, 3, 9, 82, 10]

Divide

split in half until single elements

Conquer

merge two sorted halves in order



$\approx \log_2 n$ levels of splitting

$$T(n) = 2T(n/2) + O(n)$$

each merge level touches n items: $O(n)$

Algorithm	Time complexity	Stable?	Space / note
Bubble sort	$O(n^2)$ (best $O(n)$ if no swaps)	Yes	$O(1)$, in-place
Merge sort	$O(n \log n)$ in all cases	Yes	$O(n)$ extra space
Quick sort	avg $O(n \log n)$, worst $O(n^2)$	No	$O(\log n)$ stack, in-place
Linear search	$O(n)$	—	works on any array
Binary search	$O(\log n)$	—	sorted array only

Figure 10: Merge sort divide and conquer, with sorting and searching complexities

Algorithm	Idea	Best	Avg	Worst	Space	Stable?
Merge Sort	Divide in half, sort each half recursively, merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes

Algorithm	Idea	Best	Avg	Worst	Space	Stable?
Quick Sort	Pick a pivot, partition smaller/larger, recurse	$O(n \log n)$	$O(n \log n)$	$O(n^2)$ (bad pivot / sorted input)	$O(\log n)$ stack	No
Bubble Sort	Swap adjacent out-of-order pairs; the largest "bubbles" to the end	$O(n)$ (with swapped flag)	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Linear Search	Check each element in turn	$O(1)$	$O(n)$	$O(n)$	$O(1)$	n/a
Binary Search	On sorted data, halve the range by comparing with the middle	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$ iterative	n/a

Java uses dual-pivot QuickSort for primitive arrays and TimSort (a stable merge/insertion hybrid) for objects in `Arrays.sort` and `Collections.sort`.

```
static int binarySearch(int[] a, int key) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2; // avoids int overflow
        if (a[mid] == key) return mid;
        if (a[mid] < key) lo = mid + 1; else hi = mid - 1;
    }
    return -1;
}
// binarySearch(new int[]{1, 3, 5, 7, 9}, 7) -> Output: 3
```

B42. Count word frequency in a given string.

- Trim the string, make it lowercase, split on `\\s+`, then group with `groupingBy(identity, LinkedHashMap::new, counting())`. The `LinkedHashMap` keeps the words in the order they first appear.
- Time is $O(n)$.

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.*;

public class WordFrequency {
    public static void main(String[] args) {
        String s = "the cat and the hat";
        Map<String, Long> freq = Arrays.stream(s.trim().toLowerCase().split("\\s+"))
            .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new,
            Collectors.counting()));
        System.out.println(freq);
    }
}
// Output: {the=2, cat=1, and=1, hat=1}
```

B43. Merge two arrays without duplicates.

- For a sorted result, use `IntStream.concat(...).distinct().sorted().toArray()`.
- To keep the order elements first appear in, add them all to a `LinkedHashSet`.

```
import java.util.*;
import java.util.stream.*;

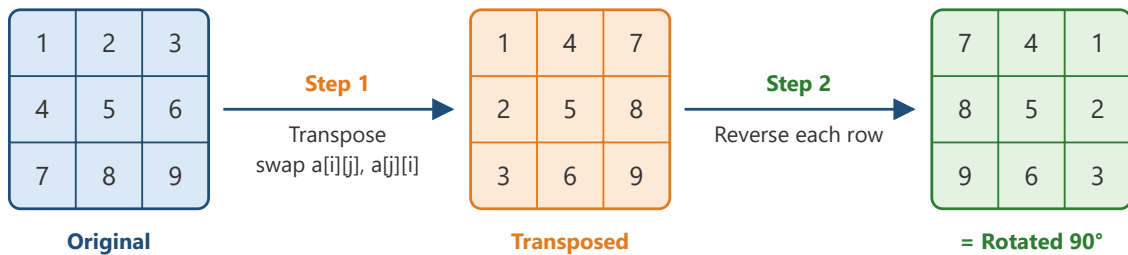
public class MergeArrays {
    public static void main(String[] args) {
        int[] a = {1, 2, 3, 4}, b = {3, 4, 5, 6};
        int[] merged = IntStream.concat(Arrays.stream(a),
Arrays.stream(b)).distinct().sorted().toArray();
        System.out.println(Arrays.toString(merged));

        Set<Integer> set = new LinkedHashSet<>();
        for (int x : a) set.add(x);
        for (int x : b) set.add(x);
        System.out.println(set);
    }
}
// Output:
// [1, 2, 3, 4, 5, 6]
// [1, 2, 3, 4, 5, 6]
```

B44. Rotate an $N \times N$ matrix by $K \times 90^\circ$ clockwise.

Rotate $N \times N$ matrix by $K \times 90^\circ$ clockwise

90° clockwise = transpose, then reverse each row (in place, $O(n^2)$)



Apply the 90° step k times ($k = K \bmod 4$)

1	2	3
4	5	6
7	8	9

K=0 (0°)

7	4	1
8	5	2
9	6	3

K=1 (90°)

9	8	7
6	5	4
3	2	1

K=2 (180°)

3	6	9
2	5	8
1	4	7

K=3 (270°)

$k = ((K \% 4) + 4) \% 4$ — works for any K , including negative

Figure 11: Rotate $N \times N$ matrix by $K \times 90^\circ$ clockwise

- First normalise K with $k = ((K \% 4) + 4) \% 4$. Four rotations bring the matrix back to where it started, and this formula also handles negative K (a counter-clockwise rotation).
- One 90° clockwise rotation, in place: **transpose**, then **reverse each row**. Repeat this k times.

- Each rotation is $O(n^2)$ time and $O(1)$ extra space. A 180° rotation can be done in one step by reversing the order of the rows and then reversing each row.

```
import java.util.Arrays;

public class RotateMatrix {
    static void rotate90(int[][] m) {
        int n = m.length;
        for (int i = 0; i < n; i++)
            for (int j = i + 1; j < n; j++) { int t = m[i][j]; m[i][j] = m[j][i]; m[j][i] = t; }
        for (int[] row : m)
            for (int l = 0, r = n - 1; l < r; l++, r--) { int t = row[l]; row[l] = row[r]; row[r] = t; }
    }

    static void rotate(int[][] m, int K) {
        int k = ((K % 4) + 4) % 4;
        for (int i = 0; i < k; i++) rotate90(m);
    }

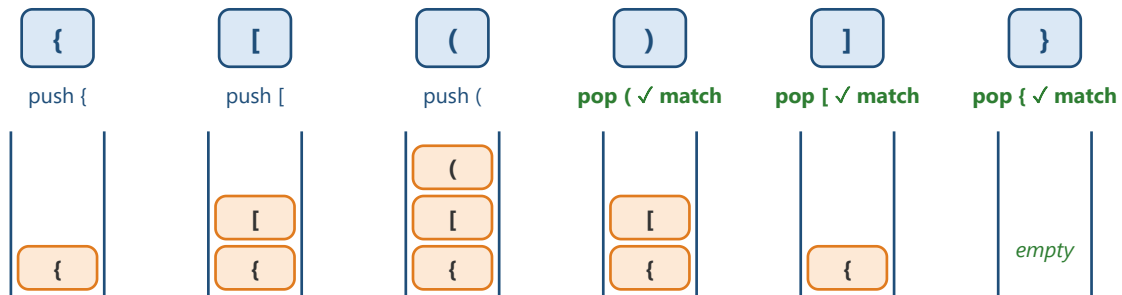
    public static void main(String[] args) {
        int[][] m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        rotate(m, 1);
        for (int[] row : m) System.out.println(Arrays.toString(row));
    }
}

// Output:
// [7, 4, 1]
// [8, 5, 2]
// [9, 6, 3]
```

B45. Check whether brackets (), {}, [] are balanced.

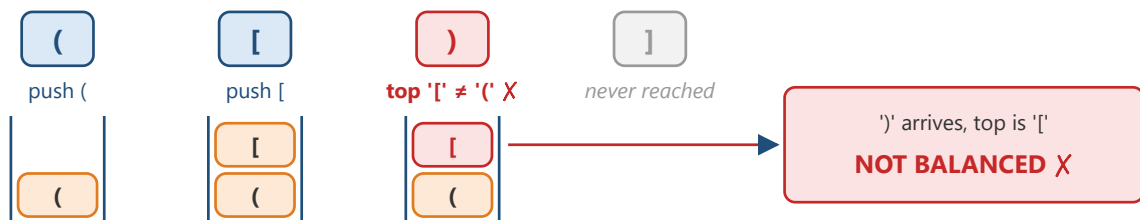
Balanced brackets with a stack

Example 1: "{ [()] }" — scan left to right; stack shown bottom → top



Every closer matched the top, stack empty at end → **BALANCED ✓**

Example 2: "([)]"



Last opened must close first → **LIFO** → **stack**. $O(n)$ time, $O(n)$ space

Figure 12: Balanced brackets with a stack

- Use a stack (`ArrayDeque`). Push every opening bracket. On a closing bracket, pop and check that it matches. If the stack is empty or the bracket does not match, return false.
- At the end the stack must be empty. That catches unclosed brackets like `"(("`.
- The stack is what makes brackets close in reverse order, so `"([)]"` fails.
- $O(n)$ time, $O(n)$ space.

```
import java.util.*;

public class BalancedBrackets {
    static boolean isBalanced(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            switch (c) {
                case '(', '{', '[' -> stack.push(c);
                case ')', '}', ']' -> {
                    if (stack.isEmpty()) return false;
                    char open = stack.pop();
                    if ((c == ')') && open != '(' || (c == '}') && open != '{') || (c == ']') && open != '[') return false;
                }
                default -> { }
            }
        }
        return stack.isEmpty();
    }
}
```

```
public static void main(String[] args) {  
    System.out.println(isBalanced("{[()]}" ));  
    System.out.println(isBalanced("([])" ));  
    System.out.println(isBalanced("(" ));  
}  
}  
// Output:  
// true  
// false  
// false
```

C. Spring Boot, Spring Framework, JPA/Hibernate & Testing

C1. What is @SpringBootApplication?

It is a convenience meta-annotation that combines three annotations:

- `@SpringBootApplication`, a specialised `@Configuration`, so the class can declare beans.
- `@EnableAutoConfiguration`, which turns on auto-configuration based on the classpath and properties.
- `@ComponentScan`, which scans the class's own package and its sub-packages.

Put it on the main class in the root package. You can exclude auto-configs with

`@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)`.

```
@SpringBootApplication
public class CityApp {
    public static void main(String[] args) { SpringApplication.run(CityApp.class, args); }
}
```

C2. What happens if @EnableAutoConfiguration (or equivalent) is not specified?

Without it, Spring Boot creates none of its auto-configured beans. You get no embedded Tomcat setup, no `DataSource`, no `ObjectMapper`, no `DispatcherServlet` and no JPA. The app still starts as a plain Spring context with only your explicit `@Configuration`, `@Bean` and scanned components. For a web app, `SpringApplication.run` then usually fails with errors like *"Unable to start ServletWebServerApplicationContext due to missing ServletWebServerFactory bean"*. You would have to configure everything by hand, as in classic Spring.

C3. @Configuration vs @Bean; why do we need configuration classes and bean annotations?

	@Configuration	@Bean
Target	Class	Method
Purpose	Marks a source of bean definitions	The method's return value becomes a bean
Proxying	CGLIB-proxied (<code>proxyBeanMethods=true</code>), so calling one <code>@Bean</code> method from another returns the same singleton	Inside <code>@Component</code> ("lite mode"), inter-method calls create new instances

Why we need them: `@Component` only works on classes you own. `@Bean` lets you register **third-party classes** such as `RestTemplate`, `ObjectMapper` or a Kafka `ProducerFactory`, or beans that need custom construction logic. It also keeps wiring explicit and in one place.

```
@Configuration
public class AppConfig {
    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder b) {
```

```
        return b.setConnectTimeout(Duration.ofSeconds(3)).build();
    }
}
```

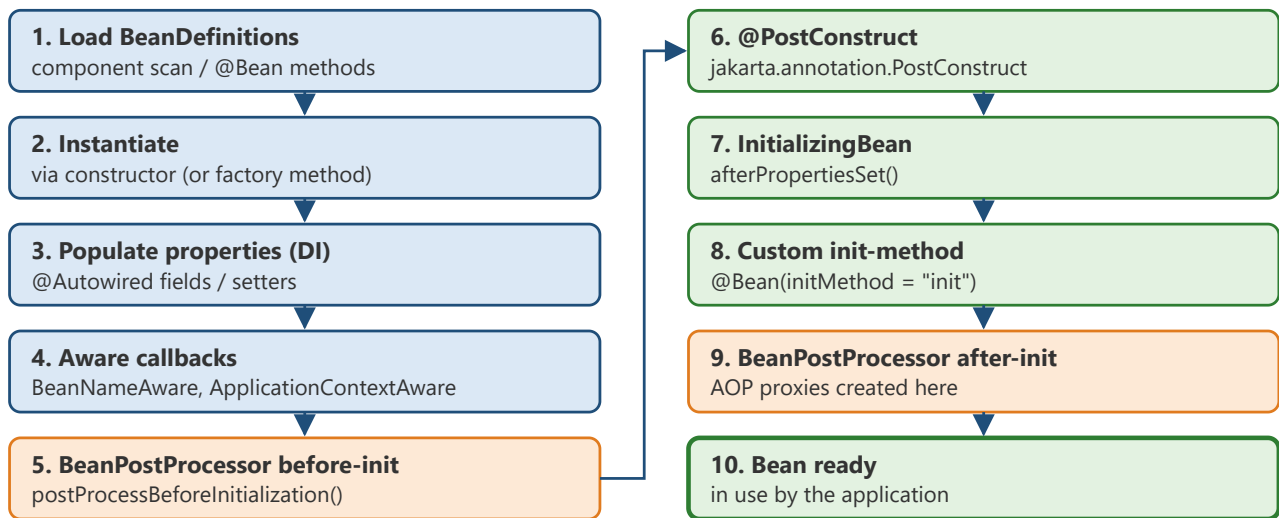
C4. What is a Bean? How is a Bean created internally?

A bean is an object whose creation, wiring and lifecycle the Spring IoC container (`ApplicationContext`) manages. Internally:

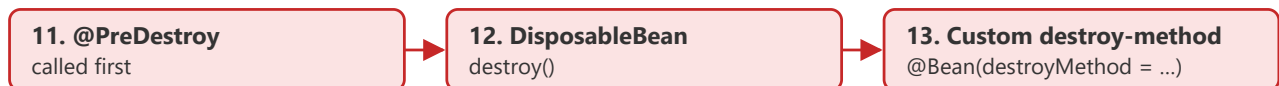
1. **Scan/parse.** Config classes and `@Component` classes are read into `BeanDefinition` s (class, scope, dependencies, init/destroy methods).
2. **BeanFactoryPostProcessor** s modify the definitions. For example, `PropertySourcesPlaceholderConfigurer` resolves `${...}` placeholders.
3. **Instantiate.** The constructor runs via reflection (constructor injection resolves its dependencies at this point).
4. **Populate.** Field and setter injection happens.
5. **Aware callbacks, then BeanPostProcessor before-init, then init, then after-init.** The after-init step is where AOP/transaction **proxies** are created.
6. The bean is stored in the singleton cache and handed out on request.

C5. Bean life cycle.

Spring Bean Lifecycle (Spring Boot 3 / Spring 6)



Shutdown (context.close() / JVM shutdown hook)



Note: prototype scope: container does not call destroy callbacks.

Init callbacks run in order 6 → 7 → 8 (same order for destroy: 11 → 12 → 13).

Figure 13: Spring Bean Lifecycle (Spring Boot 3 / Spring Framework 6)

Instantiate → inject dependencies → `BeanNameAware` / `BeanFactoryAware` / `ApplicationContextAware` → `BeanPostProcessor.postProcessBeforeInitialization` → `@PostConstruct` → `InitializingBean.afterPropertiesSet()` → custom `initMethod` → `postProcessAfterInitialization` (proxies created here) → **bean ready** → on context close: `@PreDestroy` → `DisposableBean.destroy()` → `destroyMethod`. Note: Spring does **not** call destroy callbacks for prototype beans.

```
@Component
public class SensorClient {
    @PostConstruct void init() { /* open connection */ }
    @PreDestroy void close() { /* release resources */ }
}
```

C6. Bean scopes; @Scope annotation.

Scope	Instances	Notes
singleton (default)	One per container	Must be stateless or thread-safe
prototype	New one on every <code>getBean</code> / injection	No destroy callback

Scope	Instances	Notes
request	One per HTTP request	Web-aware context only
session	One per HTTP session	Web only
application	One per <code>ServletContext</code>	Web only
websocket	One per WebSocket session	Web only

```
@Component
@Scope(value = WebApplicationContext.SCOPE_REQUEST, proxyMode = ScopedProxyMode.TARGET_CLASS)
public class RequestContext { }
```

Gotcha: if you inject a prototype or request bean into a singleton, it is injected only once. To get a fresh instance each time, use `proxyMode`, `ObjectProvider<T>` or `@Lookup`.

C7. @Lazy.

By default, singletons are created eagerly at startup, which makes misconfiguration fail fast. `@Lazy` delays creation until the bean is first requested. Uses:

- Heavy beans that are rarely needed.
- Breaking circular dependencies: `@Lazy` on the injection point injects a proxy.
- Faster startup. Globally: `spring.main.lazy-initialization=true`.

Trade-off: errors show up at runtime instead of at startup, and the first request is slower.

```
public ReportService(@Lazy PdfEngine engine) { this.engine = engine; }
```

C8. @Component; component scanning.

`@Component` marks a class as a Spring-managed bean. It has these stereotypes:

- `@Service` (business layer).
- `@Repository` (DAO layer; adds exception translation to `DataAccessException`).
- `@Controller` / `@RestController` (web layer).
- `@Configuration`.

Component scanning (`@ComponentScan`, included in `@SpringBootApplication`) searches the main class's package and its sub-packages for these annotations and registers them. Classes outside that tree need `@ComponentScan(basePackages = "com.trinity.common")` or `@Import`. You can also add `includeFilters` and `excludeFilters`.

C9. Dependency injection in Spring Boot; types of DI.

DI means the container supplies an object's dependencies, so the object does not create them itself (loose coupling, easier testing).

Type	How	Verdict
Constructor	Dependencies passed via constructor (<code>@Autowired</code> optional when there is a single constructor)	Preferred
Setter	<code>@Autowired</code> on a setter	Use for optional or changeable dependencies
Field	<code>@Autowired</code> on a field	Avoid: hides dependencies, cannot be <code>final</code> , harder to unit test

Constructor injection is preferred because:

- Fields can be `final` , so the object is immutable.
- Dependencies are mandatory and visible.
- You can unit test with `new` and no Spring.
- Circular dependencies fail at startup.

```
@Service
@RequiredArgsConstructor // Lombok generates the constructor
public class EmployeeService {
    private final EmployeeRepository repo;
}
```

C10. Multiple beans of the same type: how to specify which one?

If two beans match one injection point, Spring throws `NoUniqueBeanDefinitionException` . Options:

- **@Primary** : the default choice when there is no qualifier.
- **@Qualifier("beanName")** : an explicit choice that **overrides** **@Primary** . This is how you use the non-primary bean.
- **By name**: if the parameter or field name matches the bean name, Spring uses it as a fallback.
- **All of them**: inject `List<NotificationSender>` or `Map<String, NotificationSender>` (key = bean name) and choose at runtime (strategy pattern).
- **Three beans, want only one registered**: make the others conditional with `@Profile` , `@ConditionalOnProperty` , or by excluding them from the scan.

```
public interface NotificationSender { void send(String msg); }
@Component("sms") @Primary class SmsSender implements NotificationSender { public void send(String m) {} }
@Component("email") class EmailSender implements NotificationSender { public void send(String m) {} }
@Component("push") class PushSender implements NotificationSender { public void send(String m) {} }

@Service
class AlertService {
    private final NotificationSender defaultSender; // gets sms (@Primary)
    private final NotificationSender emailSender;
    AlertService(NotificationSender defaultSender,
                 @Qualifier("email") NotificationSender emailSender) {
        this.defaultSender = defaultSender; this.emailSender = emailSender;
    }
}
```

```
}
}
```

Two beans with the same name: Spring Boot disables overriding by default, so startup fails with `BeanDefinitionOverrideException`. To fix it, rename one with `@Bean("smsSenderV2")` or `@Component("smsV2")`. You can also set `spring.main.allow-bean-definition-overriding=true`, where the last definition wins, but that is not recommended because it hides bugs.

C11. What is JPA? (vs Hibernate vs Spring Data JPA)

	What it is
JPA (Jakarta Persistence)	A specification : annotations (<code>@Entity</code> , <code>@Id</code>), <code>EntityManager</code> and JPQL. It contains no implementation.
Hibernate	The most popular JPA implementation (ORM provider), plus its own extras (<code>Session</code> , Envers, caching).
Spring Data JPA	An abstraction on top of JPA . Repository interfaces (<code>JpaRepository</code>), derived queries (<code>findByName</code>), paging and auditing. It removes the boilerplate DAO code.

Request flow: your code calls Spring Data's `JpaRepository`, which uses JPA's `EntityManager`, which Hibernate implements, which uses JDBC to talk to the DB.

C12. @GeneratedValue (strategies).

Strategy	How	Notes
IDENTITY	DB auto-increment column (MySQL)	Simple, but disables JDBC batch inserts because the ID is known only after the insert
SEQUENCE	DB sequence (PostgreSQL, Oracle)	Best performance. <code>allocationSize</code> pre-allocates IDs, and batching works
TABLE	A separate table simulates a sequence	Portable but slow (row locks). Avoid
AUTO (default)	Provider picks one (Hibernate 6 usually picks SEQUENCE)	Not explicit
UUID (JPA 3.1)	Generates a UUID	Good for distributed systems

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "emp_seq")
@SequenceGenerator(name = "emp_seq", sequenceName = "emp_seq", allocationSize = 50)
private Long id;
```

C13. @RequestParam, @RequestBody, @PathVariable; handling request parameters.

Annotation	Source	Example URL / body	Typical use
@PathVariable	URI path segment	/employees/42	Identify a resource
@RequestParam	Query string or form field	/employees?dept=IT&page=0	Filtering, paging, optional values
@RequestBody	HTTP body (JSON via Jackson)	{"name": "Asha"}	Create or update payloads

```
@GetMapping("/employees/{id}")
public EmployeeDto get(@PathVariable Long id) { ... }

@GetMapping("/employees")
public List<EmployeeDto> search(@RequestParam(required = false) String dept,
                               @RequestParam(defaultValue = "0") int page,
                               @RequestParam Map<String, String> allParams) { ... }

@PostMapping("/employees")
public ResponseEntity<EmployeeDto> create(@Valid @RequestBody EmployeeRequest req) { ... }

@GetMapping("/employees/filter")
public List<EmployeeDto> filter(@ModelAttribute EmployeeFilter filter) { ... } // binds ?dept=&city=
to a POJO
```

Other details:

- @RequestParam is required=true by default, and a missing value returns 400. Use defaultValue or Optional<T> for optional values.
- @Valid plus Bean Validation (@NotBlank , @Email) throws MethodArgumentNotValidException , which returns 400.
- Also available: @RequestHeader and @CookieValue .

C14. What are REST controllers used for? (@RestController vs @Controller)

REST controllers expose HTTP endpoints that return **data** (JSON/XML) to clients such as an Angular/React UI, mobile apps or other microservices.

@Controller	@RestController
Return value is a view name (Thymeleaf/JSP)	Return value is serialised to the response body
Needs @ResponseBody per method to return data	= @Controller + @ResponseBody on every method
Server-side MVC pages	REST APIs

C15. Skeleton for getEmployeeById.

```
@Entity
public class Employee {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    private String name;
    // getters/setters
}
```

```

}

public interface EmployeeRepository extends JpaRepository<Employee, Long> { }

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String msg) { super(msg); }
}

@Service
@RequiredArgsConstructor
public class EmployeeService {
    private final EmployeeRepository repo;
    @Transactional(readOnly = true)
    public Employee getById(Long id) {
        return repo.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Employee " + id + " not found"));
    }
}

@RestController
@RequestMapping("/api/employees")
@RequiredArgsConstructor
public class EmployeeController {
    private final EmployeeService service;
    @GetMapping("/{id}")
    public ResponseEntity<Employee> getEmployeeById(@PathVariable Long id) {
        return ResponseEntity.ok(service.getById(id));
    }
}

```

A `@RestControllerAdvice` (see C20) maps `ResourceNotFoundException` to 404. An alternative without exceptions is `return repo.findById(id).map(ResponseEntity::ok).orElse(ResponseEntity.notFound().build());`. In real code, return a DTO rather than the entity.

C16. PUT vs PATCH.

	PUT	PATCH
Semantics	Replace the whole resource	Partial update (only the fields sent)
Missing fields	Set to null/default	Left unchanged
Idempotent	Yes	Not guaranteed (usually is if implemented as "set field")
Body	Full representation	Changed fields only, or JSON Patch / Merge Patch

PUT can also create a resource at a known URI. PATCH is used for things like `PATCH /devices/7 {"status":"OFFLINE"}`.

C17. HTTP 204.

204 No Content means the request succeeded and there is no response body. It is typical for `DELETE`, or for a `PUT / PATCH` that does not return the updated resource.

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    service.delete(id);
    return ResponseEntity.noContent().build();
}
```

Compare with 200 (success, with a body) and 201 Created (with a `Location` header).

C18. Spring Boot Actuator.

Actuator provides production-ready endpoints for **monitoring and managing** the app. Add `spring-boot-starter-actuator`. Key endpoints:

- `/actuator/health`: liveness and readiness probes for Kubernetes.
- `/info`, `/metrics` (Micrometer), `/prometheus`.
- `/env`, `/beans`, `/mappings`, `/loggers` (change log level at runtime), `/threaddump`, `/heapdump`.

Only `health` is exposed over HTTP by default. Expose more carefully and secure them with Spring Security.

```
management:
  endpoints.web.exposure.include: health,info,metrics,prometheus
  endpoint.health.show-details: when-authorized
```

You can add custom checks by implementing `HealthIndicator` (for example, a check for an external CCTV/VMS system).

C19. Front Controller design pattern (DispatcherServlet).

DispatcherServlet (Front Controller) Request Flow

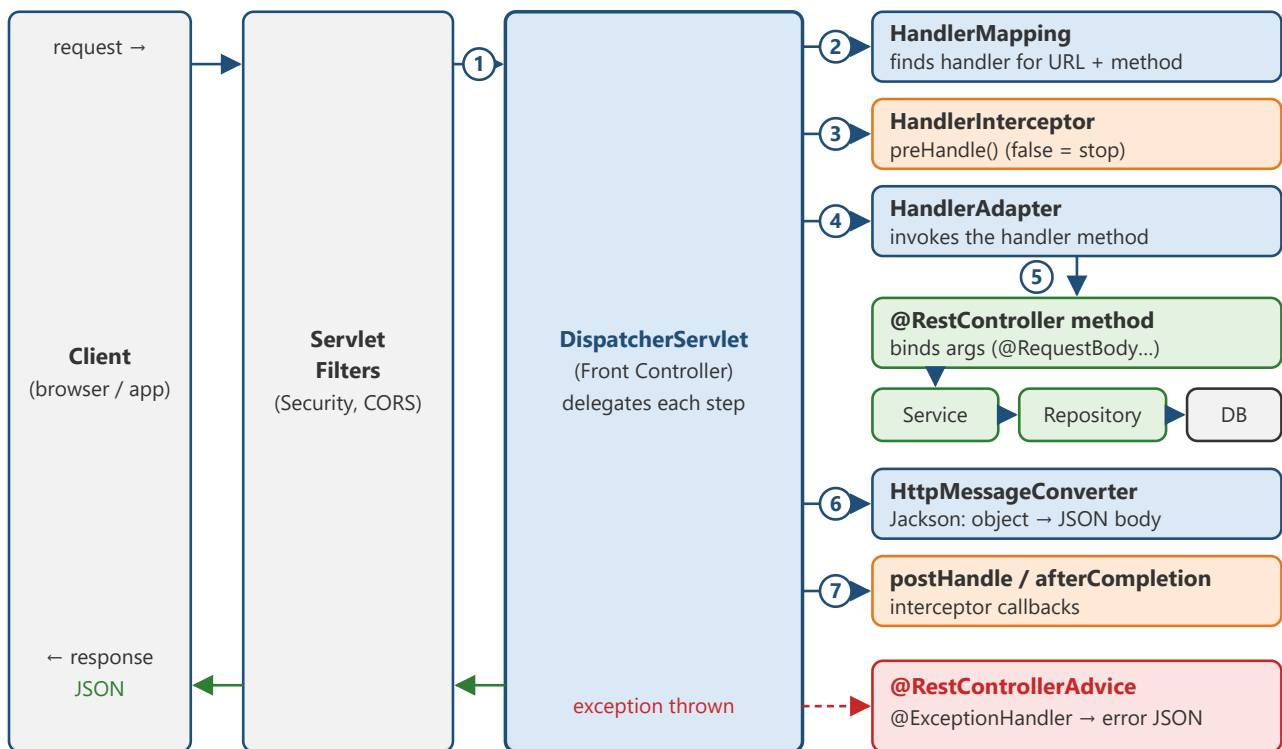


Figure 14: Spring MVC DispatcherServlet (Front Controller) request flow

A **single entry point** handles every request and delegates it to the right handler. Cross-cutting concerns (routing, security, i18n, error handling) therefore live in one place. In Spring MVC, that entry point is the **DispatcherServlet**:

1. A request arrives at `DispatcherServlet`.
2. `HandlerMapping` finds the controller method.
3. `HandlerAdapter` invokes it, using argument resolvers for `@PathVariable`, `@RequestBody` and so on.
4. The return value is written by an `HttpMessageConverter` (Jackson to JSON) or resolved by a `ViewResolver`.
5. Exceptions go to a `HandlerExceptionResolver` (`@ControllerAdvice`).
6. The response is sent.

Spring Boot auto-registers `DispatcherServlet` at `/`.

C20. Exception handling in Spring Boot.

- `@ExceptionHandler` works for one controller only.
- `@RestControllerAdvice` (= `@ControllerAdvice` + `@ResponseBody`) is **global**, so you get a consistent error format.
- Throw a **custom exception** from the service layer and map it to a status in the advice.

- Spring 6 adds `ProblemDetail` (RFC 7807/9457, `application/problem+json`). Enable it for framework exceptions with `spring.mvc.problemdetails.enabled=true`.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ProblemDetail handleNotFound(ResourceNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Resource Not Found");
        return pd;
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail handleValidation(MethodArgumentNotValidException ex) {
        ProblemDetail pd = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
        pd.setProperty("errors", ex.getBindingResult().getFieldErrors().stream()
            .collect(Collectors.toMap(FieldError::getField,
                fe -> String.valueOf(fe.getDefaultMessage()), (a, b) -> a)));
        return pd;
    }

    @ExceptionHandler(Exception.class)
    public ProblemDetail handleGeneric(Exception ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR, "Unexpected error");
    }
}
```

Alternatives are `@ResponseStatus` on the exception class, or `throw new ResponseStatusException(HttpStatus.NOT_FOUND)`.

C21. @ConfigurationProperties; creating custom configuration properties.

`@ConfigurationProperties` binds a group of properties under a prefix to a **type-safe** object. Compared with many `@Value` annotations, it supports relaxed binding, nested objects and lists, and validation.

```
smartcity:
  vms:
    base-url: https://vms.local/api
    timeout: 5s
    cameras: [CAM-01, CAM-02]
```

```
@Validated
@ConfigurationProperties(prefix = "smartcity.vms")
public record VmsProperties(@NotBlank String baseUrl,
    @DefaultValue("3s") Duration timeout,
    List<String> cameras) { }
```

```
@SpringBootApplication
@ConfigurationPropertiesScan // or @EnableConfigurationProperties(VmsProperties.class) on a
@Configuration
public class CityApp { ... }
```

```
@Service
public class VmsClient {
    private final VmsProperties props;
```

```
public VmsClient(VmsProperties props) { this.props = props; }
}
```

Records give immutable constructor binding. Add `spring-boot-configuration-processor` to get IDE auto-completion through metadata.

C22. Configuring different environments.

- **Profile files:** `application.yml` holds common settings. `application-dev.yml`, `application-qa.yml` and `application-prod.yml` override them.
- **Activate a profile** with `spring.profiles.active=prod`, the `SPRING_PROFILES_ACTIVE=prod` env var, `--spring.profiles.active=prod`, or `-Dspring.profiles.active=prod`.
- **Profile-specific beans:** `@Profile("dev")` or `@Profile("!prod")` on beans or config classes.
- **Multi-document YAML:** use `---` with `spring.config.activate.on-profile: prod`.
- **Secrets and environment values:** environment variables (relaxed binding: `SPRING_DATASOURCE_URL`), Kubernetes ConfigMaps/Secrets, Vault.
- **Microservices:** centralise config with **Spring Cloud Config Server** (Git-backed) via `spring.config.import=configserver:`, and refresh with `@RefreshScope`.
- **Precedence** (high to low): command-line args, env vars, profile-specific files, `application.yml`.

C23. SessionFactory: what it is and how to configure it.

`SessionFactory` is Hibernate's **heavyweight, thread-safe, one-per-database** factory that creates `Session` objects. A `Session` is lightweight and not thread-safe, and you use one per unit of work. The factory holds the mapping metadata, the connection provider and the second-level cache. The JPA equivalent is `EntityManagerFactory` → `EntityManager`.

In plain Hibernate you build it with `new`

`Configuration().configure("hibernate.cfg.xml").buildSessionFactory()`. **In Spring Boot you do not configure it.** Boot auto-configures an `EntityManagerFactory` (Hibernate's `SessionFactoryImpl` implements it) from `spring.datasource.*` and `spring.jpa.*`. If you need native Hibernate APIs, unwrap it:

```
@Bean
public SessionFactory sessionFactory(EntityManagerFactory emf) {
    return emf.unwrap(SessionFactory.class);
}
// or per-operation: Session session = entityManager.unwrap(Session.class);
```

C24. Configuring multiple data sources.

Steps:

1. Define a separate property prefix for each data source.
2. Create a `DataSource`, an `EntityManagerFactory` and a `TransactionManager` per DB. Mark one set `@Primary`.
3. Point each `@EnableJpaRepositories` at its own repository package and entity package.

```

@Configuration
@EnableJpaRepositories(basePackages = "com.trinity.citizen.repo",
    entityManagerFactoryRef = "citizenEmf", transactionManagerRef = "citizenTx")
public class CitizenDbConfig {

    @Bean @Primary
    @ConfigurationProperties("app.datasource.citizen")
    public DataSourceProperties citizenDsProps() { return new DataSourceProperties(); }

    @Bean @Primary
    public DataSource citizenDataSource() {
        return citizenDsProps().initializeDataSourceBuilder().build();
    }

    @Bean @Primary
    public LocalContainerEntityManagerFactoryBean citizenEmf(EntityManagerFactoryBuilder builder) {
        return builder.dataSource(citizenDataSource())
            .packages("com.trinity.citizen.entity")
            .persistenceUnit("citizen").build();
    }

    @Bean @Primary
    public PlatformTransactionManager citizenTx(@Qualifier("citizenEmf") EntityManagerFactory emf) {
        return new JpaTransactionManager(emf);
    }
}
// A second class, e.g. IotDbConfig, repeats this without @Primary, using "app.datasource.iot" and
// other packages.

```

For the non-primary DB, use `@Transactional("iotTx")`. Use `AbstractRoutingDataSource` for read/write splitting or multi-tenancy.

C25. Lazy vs eager loading in Hibernate.

Relationship	Default fetch
@ManyToOne , @OneToOne	EAGER
@OneToMany , @ManyToMany	LAZY

- **Lazy** loads a proxy and runs the query only when the association is accessed. **Eager** loads it immediately. Best practice: make everything **LAZY** (`@ManyToOne(fetch = FetchType.LAZY)`) and fetch what each use case needs.
- **LazyInitializationException** : you accessed a lazy association after the session or transaction closed, for example in the controller or during JSON serialisation. Fix it by fetching inside a transaction, using a fetch join or entity graph, or mapping to a DTO. Do not rely on Open-Session-In-View; Boot enables it by default, so set `spring.jpa.open-in-view=false`.
- **N+1 problem**: 1 query loads N parents, then N more queries load each child collection.

Fixes:

```

@Query("select d from Department d join fetch d.employees where d.id = :id")
Optional<Department> findWithEmployees(@Param("id") Long id);

```

```
@EntityGraph(attributePaths = "employees")
List<Department> findAll();
```

Other options are `@BatchSize(size = 50)` or `hibernate.default_batch_fetch_size`, and DTO projections.

C26. One-to-Many relationship across multiple tables.

For a bidirectional mapping, the **child (@ManyToOne) owns the foreign key**, and the parent uses `mappedBy`.

```
@Entity
public class Department {
    @Id @GeneratedValue private Long id;
    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Employee> employees = new ArrayList<>();

    public void addEmployee(Employee e) { employees.add(e); e.setDepartment(this); } // keep both
    sides in sync
    public void removeEmployee(Employee e) { employees.remove(e); e.setDepartment(null); }
}

@Entity
public class Employee {
    @Id @GeneratedValue private Long id;
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id") // FK column in employee table
    private Department department;
}
```

- `cascade = ALL` : saving or deleting the department cascades to its employees.
- `orphanRemoval = true` : removing an employee from the list deletes its row.
- **Join table option** (unidirectional, no FK in the child table): `@OneToMany @JoinTable(name="dept_emp", joinColumns=@JoinColumn(name="dept_id"), inverseJoinColumns=@JoinColumn(name="emp_id"))`. A unidirectional `@OneToMany` without `@JoinColumn` creates a join table by default.
- **Chains across tables** (City → Zone → Device) are just repeated one-to-many mappings. Fetch the levels you need with join fetch or an entity graph.
- Avoid `toString` / `equals` that touch both sides, because that causes infinite recursion. In JSON, use DTOs or `@JsonManagedReference`.

C27. Implementing audit logs.

1. **Spring Data JPA auditing** records *who* created or changed a row and *when*:

```
@Configuration
@EnableJpaAuditing(auditorAwareRef = "auditorAware")
public class AuditConfig {
    @Bean
    public AuditorAware<String> auditorAware() {
        return () -> Optional.ofNullable(SecurityContextHolder.getContext().getAuthentication())
            .map(Authentication::getName);
    }
}

@MappedSuperclass
```

```
@EntityListeners(AuditingEntityListener.class)
public abstract class Auditable {
    @CreatedBy      private String createdBy;
    @CreatedDate    private Instant createdAt;
    @LastModifiedBy private String updatedBy;
    @LastModifiedDate private Instant updatedAt;
}
```

2. Hibernate Envers keeps the *full history of changes*. Add `@Audited` on the entity, and Envers writes every revision to `<table>_AUD` plus `REVINFO`. Query old versions with `AuditReader`.

3. AOP-based audit table records business actions (who did what, on which resource, from which IP). Use a custom `@AuditLog(action="DELETE_DEVICE")` annotation with an `@Aspect @Around` advice that saves an `audit_log` row. Save it asynchronously, or publish to Kafka, so the main flow is not slowed down.

Other options are DB triggers and CDC (Debezium). Audit tables should be append-only.

C28. URL shortener in Java/Spring Boot.

Design:

- `POST /api/shorten {longUrl}` returns `{shortUrl}`.
- `GET /{code}` looks up the original and **redirects**:
 - **301** (permanent) lets browsers cache it, so there is less load but no analytics.
 - **302/307** makes every hit go through the service, so you can track clicks.

Code generation:

- **Base62 of an auto-increment or sequence ID** (`[0-9a-zA-Z]`). Seven characters give about 3.5 trillion codes. It is unique by construction, so there are no collisions.
- Distributed: use a Snowflake ID or pre-allocated ID ranges per instance.
- Alternative: a hash (MD5/SHA-256 of the URL, first 7 characters). Collisions are possible, so check and retry with a salt.

Duplicates and conflicts:

- Put a `UNIQUE` constraint on `short_code`. On `DataIntegrityViolationException`, regenerate.
- To return the same code for the same long URL, add a unique index on a `long_url_hash` and look it up first.
- For custom aliases, check availability and return **409 Conflict** if it is taken.

Storage:

- Table `url_mapping(id, short_code UNIQUE, long_url, created_at, expires_at, click_count)` in an RDBMS or a key-value/NoSQL store.
- Hot lookups go through a **Redis cache-aside** (`code → longUrl`, with a TTL).
- Record analytics asynchronously (Kafka).
- Add rate limiting, and validate URLs to block malicious links.

```
private static final String BASE62 = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
static String encode(long id) {
    StringBuilder sb = new StringBuilder();
    do { sb.append(BASE62.charAt((int) (id % 62))); id /= 62; } while (id > 0);
```

```

    return sb.reverse().toString();
}

@GetMapping("/{code}")
public ResponseEntity<Void> redirect(@PathVariable String code) {
    String longUrl = service.resolve(code); // Redis -> DB, throws NotFound
    return ResponseEntity.status(HttpStatus.FOUND).location(URI.create(longUrl)).build();
}

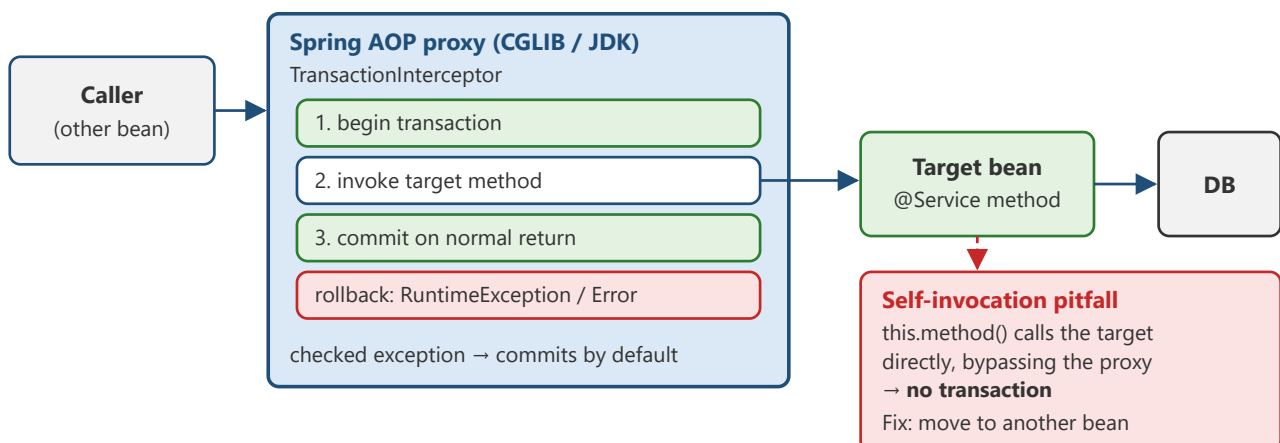
```

C29. What does LOC mean?

LOC = Lines of Code. It is a size metric for a codebase or a change, sometimes used for effort or productivity estimates and code-quality ratios (defects per KLOC). Variants are SLOC (source lines, excluding blanks and comments) and KLOC (thousands). It is a weak productivity measure, because fewer, cleaner lines are often better. In a **banking/trade-finance** context, LOC can also mean **Letter of Credit**: a bank's guarantee of payment to a seller on behalf of a buyer.

C30. @Transactional and why transaction management is used.

@Transactional: How the Spring AOP Proxy Works



Propagation: @Transactional(propagation = ...)

Propagation	Behaviour
REQUIRED (default)	Join the existing transaction, or create a new one
REQUIRES_NEW	Suspend the outer transaction, always start a new one
NESTED	Savepoint inside the existing transaction; else like REQUIRED
SUPPORTS	Join if a transaction exists, else run non-transactionally
MANDATORY	Must run inside an existing transaction, else throw exception
NOT_SUPPORTED	Suspend any existing transaction, run non-transactionally
NEVER	Throw an exception if a transaction exists

Figure 15: @Transactional proxy mechanism and propagation types

Transactions guarantee **ACID** for a unit of work. Either all DB operations commit or all roll back, for example "debit + credit" or "save incident + save assignment". `@Transactional` is declarative. Spring wraps the bean in

an **AOP proxy** that begins, commits or rolls back through the `PlatformTransactionManager`.

Key attributes:

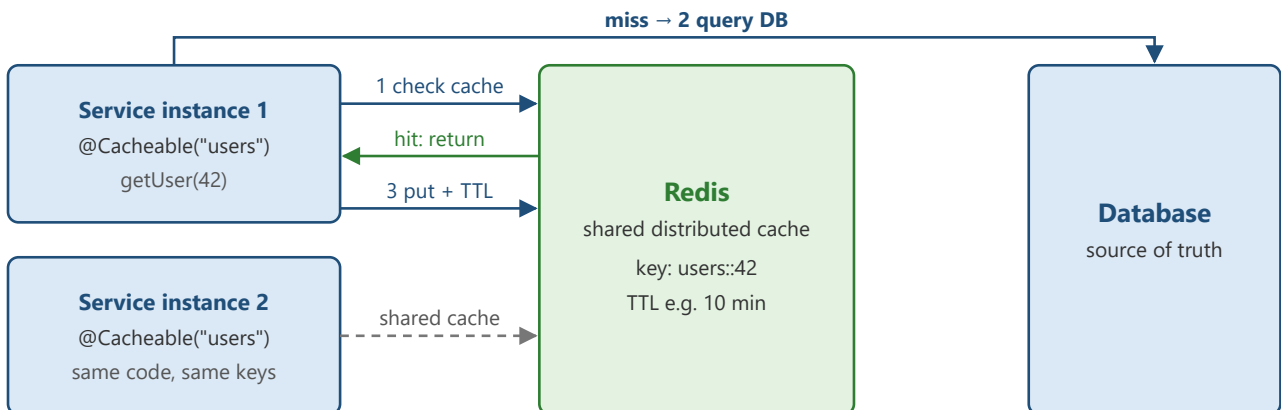
Attribute	Key points
<code>propagation</code>	<code>REQUIRED</code> (default: join the existing transaction or create one), <code>REQUIRES_NEW</code> (suspend the outer one and start a new one, e.g. for an audit log), <code>NESTED</code> (savepoint), <code>SUPPORTS</code> , <code>MANDATORY</code> , <code>NEVER</code> , <code>NOT_SUPPORTED</code>
<code>rollbackFor</code>	By default, rollback happens only on unchecked exceptions (<code>RuntimeException</code> , <code>Error</code>). Checked exceptions commit unless you set <code>rollbackFor = Exception.class</code>
<code>readOnly = true</code>	Flush and dirty-checking optimisations for queries
<code>isolation</code> , <code>timeout</code>	Isolation level and timeout

Pitfalls:

- **Self-invocation.** Calling a `@Transactional` method from another method of the *same class* bypasses the proxy, so no transaction is applied. Fix it by moving the method to another bean or injecting `self/ TransactionTemplate`.
- `@Transactional` has no effect on **private** methods; it applies only to proxied, public methods.
- Catching the exception inside the method prevents rollback.

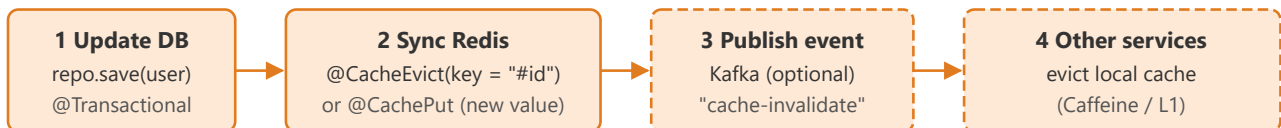
C31. @Cacheable, caching for DB performance, and cache consistency across instances.

Cache-aside with @Cacheable across multiple instances



Read path: 1 check cache → hit: return; miss → 2 query DB → 3 put in cache with TTL, then return

Write path



Prefer @CacheEvict over @CachePut on writes: avoids racing a stale value into the cache.
Evict after the DB commit; TTL bounds staleness if an eviction is ever lost.

Figure 16: Cache-aside with @Cacheable across multiple service instances

Enable caching with @EnableCaching. With @Cacheable, the first call runs the method and stores the result. Later calls with the same key return the cached value without hitting the DB, which reduces latency and DB load for read-heavy data such as master data or device configs.

```

@Cacheable(value = "employees", key = "#id", unless = "#result == null")
public EmployeeDto getById(Long id) { ... }

@CachePut(value = "employees", key = "#result.id")
public EmployeeDto update(EmployeeDto dto) { ... }

@CacheEvict(value = "employees", key = "#id")
public void delete(Long id) { ... }
  
```

Preventing DB problems:

Problem	Fix
Stale or unbounded memory	TTL plus size-based eviction (Caffeine <code>maximumSize</code> , Redis <code>time-to-live</code>)
Cache stampede (a hot key expires and many requests hit the DB)	@Cacheable(sync = true), a lock or single-flight, jittered TTLs, refresh-ahead
Cache penetration (repeated lookups for non-existent keys)	Cache null or empty results with a short TTL, a Bloom filter, input validation

Problem	Fix
Avalanche (many keys expire together)	Randomised TTLs

Consistency across instances and microservices:

- A local in-process cache (Caffeine) on each pod drifts, so use a **distributed cache (Redis)** shared by all instances.
- Use the **cache-aside** pattern: read from cache, fall back to the DB and populate the cache. On write, **update the DB, then evict** (`@CacheEvict`) instead of updating the cache.
- When another service owns the data, publish a **domain event (Kafka)** or Redis pub/sub on change, and consumers evict their entries.
- Short TTLs are the safety net. For a two-level cache (local + Redis), broadcast invalidations.

C32. JUnit annotations.

Annotation	Purpose
<code>@Test</code>	Marks a test method
<code>@BeforeEach</code> / <code>@AfterEach</code>	Run before or after each test (setup and cleanup)
<code>@BeforeAll</code> / <code>@AfterAll</code>	Run once per class (must be <code>static</code> unless <code>@TestInstance(PER_CLASS)</code>)
<code>@DisplayName</code>	Readable test name
<code>@Disabled</code>	Skip a test
<code>@ParameterizedTest</code> + <code>@ValueSource</code> / <code>@CsvSource</code> / <code>@MethodSource</code>	Run one test with many inputs
<code>@Nested</code>	Group related tests in an inner class
<code>@ExtendWith</code>	Register extensions (<code>MockitoExtension</code> , <code>SpringExtension</code>)
<code>@Tag</code> , <code>@Timeout</code> , <code>@RepeatedTest</code>	Filtering, time limits, repetition
Mockito <code>@Mock</code>	Create a mock
Mockito <code>@InjectMocks</code>	Create the class under test and inject the mocks into it
<code>@Spy</code> , <code>@Captor</code>	Partial mock, argument captor

```
@ParameterizedTest
@CsvSource({"2,3,5", "10,-1,9"})
void adds(int a, int b, int expected) { assertEquals(expected, a + b); }
```

C33. Writing test cases for Spring Boot applications.

Level	Tool	Loads	Use for
Unit	JUnit 5 + Mockito (@ExtendWith(MockitoExtension.class))	No Spring	Service logic, fast
Web slice	@WebMvcTest(EmployeeController.class) + MockMvc	MVC layer only	Controllers, validation, JSON, advice
JPA slice	@DataJpaTest	JPA + embedded/Testcontainers DB, rollback per test	Repositories, custom queries
Integration	@SpringBootTest (+ webEnvironment = RANDOM_PORT)	Full context	End-to-end flows
Real infra	Testcontainers (@Testcontainers , @Container , @ServiceConnection)	Real Postgres/Kafka/Redis in Docker	Prod-like integration tests

Mock Spring beans with `@MockBean` up to Boot 3.3. It is deprecated from **Boot 3.4**; use `@MockitoBean` instead.

```
@ExtendWith(MockitoExtension.class)
class EmployeeServiceTest {
    @Mock EmployeeRepository repo;
    @InjectMocks EmployeeService service;

    @Test
    void returnsEmployee() {
        Employee e = new Employee(); e.setName("Asha");
        when(repo.findById(1L)).thenReturn(Optional.of(e));
        assertEquals("Asha", service.getById(1L).getName());
        verify(repo).findById(1L);
    }
}
```

Follow Arrange-Act-Assert (Given-When-Then), keep one behaviour per test, and use JaCoCo for coverage.

C34. Testing REST APIs.

MockMvc (no real server; usually with `@WebMvcTest`):

```
@WebMvcTest(EmployeeController.class)
class EmployeeControllerTest {
    @Autowired MockMvc mockMvc;
    @MockitoBean EmployeeService service; // @MockBean on Boot < 3.4

    @Test
    void getById_returns200() throws Exception {
        Employee e = new Employee(); e.setId(1L); e.setName("Asha");
        when(service.getById(1L)).thenReturn(e);

        mockMvc.perform(get("/api/employees/1").accept(MediaType.APPLICATION_JSON))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("Asha"));
    }
}
```

```

    }

    @Test
    void create_invalidBody_returns400() throws Exception {
        mockMvc.perform(post("/api/employees").contentType(MediaType.APPLICATION_JSON).content("{}"))
            .andExpect(status().isBadRequest());
    }
}

```

Other tools:

- `TestRestTemplate` / `WebTestClient` with `@SpringBootTest(webEnvironment = RANDOM_PORT)` for real HTTP calls.
- **RestAssured** for fluent BDD-style API tests:
`given().when().get("/api/employees/1").then().statusCode(200) .`
- **Postman/Newman** for manual and collection-based testing in CI.
- WireMock to stub downstream services.

C35. Handling exceptions in test cases.

```

@Test
void getById_notFound_throws() {
    when(repo.findById(99L)).thenReturn(Optional.empty());

    ResourceNotFoundException ex = assertThrows(ResourceNotFoundException.class,
        () -> service.getById(99L));
    assertEquals("Employee 99 not found", ex.getMessage());
}

@Test
void validInput_doesNotThrow() {
    when(repo.findById(1L)).thenReturn(Optional.of(new Employee()));
    assertDoesNotThrow(() -> service.getById(1L));
}

// Mocking a dependency to throw
when(repo.save(any())).thenThrow(new DataIntegrityViolationException("dup"));

// Controller layer: exception mapped to HTTP status by @RestControllerAdvice
@Test
void getById_notFound_returns404() throws Exception {
    when(service.getById(99L)).thenThrow(new ResourceNotFoundException("Employee 99 not found"));
    mockMvc.perform(get("/api/employees/99"))
        .andExpect(status().isNotFound())
        .andExpect(jsonPath("$.detail").value("Employee 99 not found"));
}

```

Notes:

- Prefer `assertThrows` over `@Test(expected=...)` (JUnit 4), because it lets you assert on the message.
- With AssertJ: `assertThatThrownBy(() -> ...).isInstanceOf(X.class).hasMessageContaining("99")`.

D. Security, Microservices & System Design

D1. Authentication vs authorization in Java/Spring Boot.

- **Authentication** checks *who you are* (username/password, JWT, OAuth2 login). If it fails you get **401 Unauthorized**.
- **Authorization** checks *what you are allowed to do* (roles/authorities such as `ROLE_ADMIN` or `incident:write`). If it fails you get **403 Forbidden**.
- Authentication always comes first. In Spring Security, the `AuthenticationManager` authenticates the user and puts an `Authentication` object in the `SecurityContextHolder`. Authorization (`authorizeHttpRequests`, `@PreAuthorize`) then reads that object's `getAuthorities()`.

D2. How do you implement authentication & authorization in your project?

Spring Security 6: JWT authentication flow (Spring Boot 3)

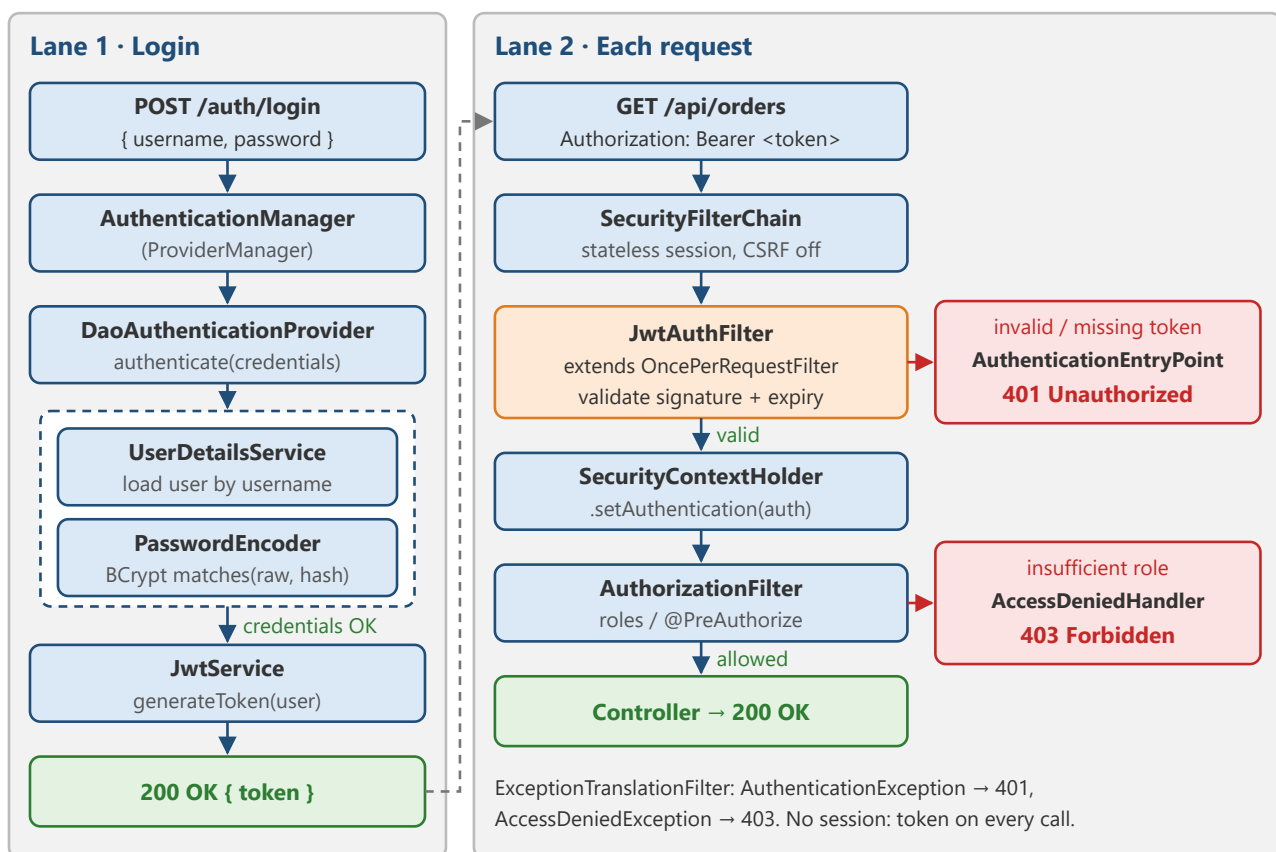


Figure 17: Spring Security 6 JWT flow: login lane and per-request lane

Classes involved: `SecurityFilterChain` (the configuration), `AuthenticationManager` → `DaoAuthenticationProvider` → `UserDetailsService` (loads the user from the DB) and `PasswordEncoder` (BCrypt), a `JwtAuthFilter` extends `OncePerRequestFilter`, and `SecurityContextHolder` (holds the authenticated user for the current request).

Login: POST /auth/login -> AuthenticationManager.authenticate(UsernamePasswordAuthenticationToken)
-> UserDetailsServiceImpl.loadUserByUsername -> PasswordEncoder.matches -> issue JWT (access + refresh)
Request: Bearer JWT -> JwtAuthFilter (validate signature/expiry) -> load authorities
-> SecurityContextHolder.setAuthentication -> authorizeHttpRequests / @PreAuthorize -> controller

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {
    private final JwtAuthFilter jwtAuthFilter;

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable()) // stateless REST API
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**", "/actuator/health").permitAll()
                .requestMatchers(HttpMethod.DELETE, "/api/incidents/**").hasRole("ADMIN")
                .requestMatchers("/api/incidents/**").hasAnyRole("OPERATOR", "ADMIN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }

    @Bean PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

    @Bean
    AuthenticationManager authenticationManager(AuthenticationConfiguration cfg) throws Exception {
        return cfg.getAuthenticationManager();
    }
}
```

```
@Component
@RequiredArgsConstructor
public class JwtAuthFilter extends OncePerRequestFilter {
    private final JwtService jwtService;
    private final UserDetailsServiceImpl userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String header = req.getHeader(HttpHeaders.AUTHORIZATION);
        if (header != null && header.startsWith("Bearer ")
            && SecurityContextHolder.getContext().getAuthentication() == null) {
            String token = header.substring(7);
            String username = jwtService.extractUsername(token);
            UserDetails user = userDetailsService.loadUserByUsername(username);
            if (jwtService.isValid(token, user)) {
                var auth = new UsernamePasswordAuthenticationToken(user, null, user.getAuthorities());
                auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(req));
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);
    }
}
```

```
}
}
```

Authorization in web apps generally:

- **Roles vs authorities.** A role is a coarse authority with the `ROLE_` prefix (`hasRole("ADMIN")`). An authority is a fine-grained permission (`hasAuthority("incident:write")`).
- **URL-based.** Set in `authorizeHttpRequests` inside the filter chain, as shown above.
- **Method-based.** `@PreAuthorize` / `@PostAuthorize` on service methods, enabled with `@EnableMethodSecurity`.
- **Domain/data-level.** For example, "an operator can only see incidents in their own zone". Implement it with SpEL that calls a bean, or with query filters.

D3. Handling exceptions in authentication and authorization.

- **401:** implement `AuthenticationEntryPoint`. It runs when an unauthenticated user hits a protected resource (missing, invalid or expired token).
- **403:** implement `AccessDeniedHandler`. It runs when the user is authenticated but lacks the required role.
- **Why `@ControllerAdvice` doesn't catch these:** security filters run *before* the `DispatcherServlet`, so exceptions thrown there never reach the controller layer. `ExceptionHandler` turns them into calls to the entry point and the handler. One exception: method-security `AccessDeniedException` thrown from a `@PreAuthorize` *inside* the controller or service *can* reach `@ControllerAdvice`. For JWT parsing errors in your own filter, catch them and write the response yourself, or delegate to a `HandlerExceptionResolver`.

```
http.exceptionHandling(ex -> ex
    .authenticationEntryPoint((req, res, e) -> {
        res.setStatus(HttpStatus.SC_UNAUTHORIZED);
        res.setContentType("application/json");
        res.getWriter().write("{\"error\":\"Unauthorized\",\"message\":\"\" + e.getMessage() + \"\"}");
    })
    .accessDeniedHandler((req, res, e) -> {
        res.setStatus(HttpStatus.SC_FORBIDDEN);
        res.setContentType("application/json");
        res.getWriter().write("{\"error\":\"Forbidden\"}");
    })
);
```

D4. Pre-authorization (@PreAuthorize).

- Turn it on with `@EnableMethodSecurity` (Spring Security 6; it replaces `@EnableGlobalMethodSecurity`). `prePostEnabled` is true by default.
- `@PreAuthorize` evaluates a SpEL expression **before** the method runs. If the result is false, it throws `AccessDeniedException` (403). `@PostAuthorize` checks the `returnObject` after the method runs.

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteIncident(Long id) { ... }

@PreAuthorize("hasAnyAuthority('incident:write','ROLE_SUPERVISOR')")
public Incident update(Long id, IncidentDto dto) { ... }
```

```

@PreAuthorize("#userId == authentication.principal.id or hasRole('ADMIN')")
public UserProfile getProfile(Long userId) { ... }

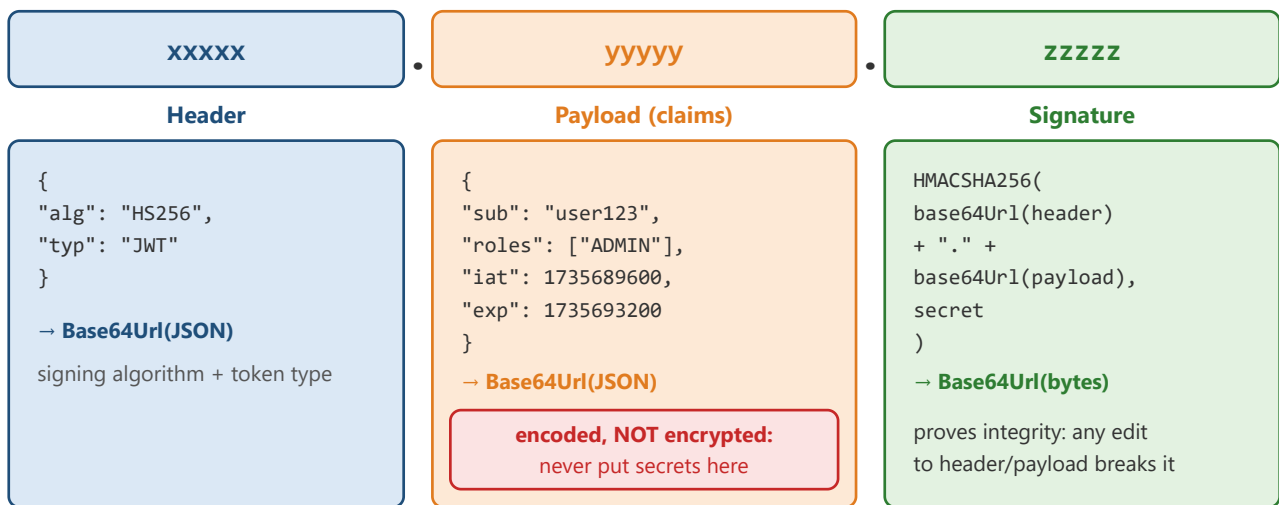
@PreAuthorize("@zoneSecurity.canAccess(authentication, #zoneId)") // custom bean check
public List<Sensor> sensorsInZone(String zoneId) { ... }

@PostAuthorize("returnObject.assignedTo == authentication.name")
public Incident getIncident(Long id) { ... }

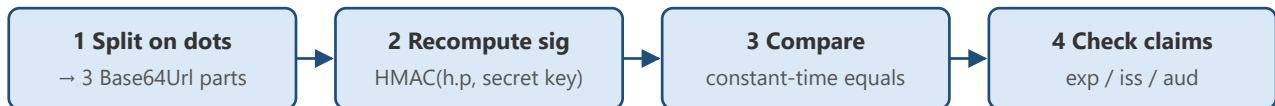
```

D5. How does JWT work?

JWT structure (signed JWS, HS256)



Verification on every request



Any step fails → reject with 401. Pin the expected alg; never accept "alg": "none".
 HS256: shared secret (≥ 256-bit). RS256 / ES256: private key signs, public key verifies.

Figure 18: JWT structure: header, payload, signature and verification steps

- **Structure:** `header.payload.signature`, each part Base64URL-encoded.
 - **Header** holds the algorithm, e.g. `{ "alg": "HS256", "typ": "JWT" }`.
 - **Payload** holds claims: `sub`, `roles`, `iat`, `exp`, `iss`.
 - **Signature** is `HMACSHA256(header + "." + payload, secret)`, or RS256 with a private key (services verify with the public key).
- **Signed, not encrypted.** Anyone can read the payload, so never put secrets in it. The signature only proves the token was not tampered with.
- **Stateless.** The server keeps no session. Every request carries `Authorization: Bearer <token>`, and each service checks the signature and `exp` locally. This scales well across microservices.
- **Expiry and refresh.** Keep the access token short-lived (5–15 min). Use a longer-lived refresh token, stored server-side or in Redis so it can be revoked, to get a new access token from `/auth/refresh`. Rotate refresh

tokens on each use.

- **Revocation** is hard because tokens are stateless. Options: short TTL, a Redis blacklist of `jti` values, or a token version stored on the user.

D6. Explain the authentication layer in your project (template; adapt to your project).

"Our Smart City platform has an **auth-service** that owns users, roles (ADMIN, SUPERVISOR, OPERATOR, CITIZEN) and permissions in PostgreSQL. On login, the `AuthenticationManager` validates credentials through our `UserDetailsService` and a `BCryptPasswordEncoder`, then issues an RS256-signed access token (15 min) and a refresh token (7 days, stored in Redis). The **API Gateway** (Spring Cloud Gateway) validates the JWT for every request: it checks the signature with the public key and checks expiry, then forwards the user ID and roles as headers. Downstream services such as incident, traffic and notification also run as resource servers with `oauth2ResourceServer().jwt()`, so they apply `@PreAuthorize` rules like operator-zone checks. Service-to-service calls use client-credentials tokens. A custom `AuthenticationEntryPoint` and `AccessDeniedHandler` return consistent JSON 401/403 responses. Failed logins are rate-limited and audited."

```
Angular --login--> Gateway --> auth-service --(JWT + refresh)--> Angular (stores in memory)
Angular --Bearer--> Gateway [validate JWT] --> incident-service [@PreAuthorize] --> DB
```

D7. Security layers for REST APIs; best practices; securing internal APIs.

- **Transport:** HTTPS/TLS everywhere. Use **mTLS** between internal services, or a service mesh such as Istio.
- **Edge:** the API Gateway is the single entry point. It handles JWT validation, **rate limiting** (Redis `RequestRateLimiter`), IP allow-lists and a WAF.
- **Identity:** OAuth2/OIDC (Keycloak). Users use the auth-code + PKCE flow. Service-to-service calls use the **client credentials** flow. **API keys** go to third-party or IoT device clients, scoped and rotated.
- **Application:**
 - Restrictive **CORS** (`allowedOrigins` set to your UI domain only, never `*` together with credentials).
 - **Input validation** with `@Valid` and Bean Validation.
 - Parameterized queries via JPA to prevent SQL injection.
 - Output encoding.
 - CSRF protection is only needed for cookie-based sessions.
- **Data:** never log tokens, passwords or PII (mask them). Encrypt sensitive columns. Return DTOs, not entities.
- **Secrets:** Vault, AWS Secrets Manager, or Kubernetes secrets with an encrypted Config Server. Never put secrets in Git.
- **Ops:** follow the OWASP Top 10, scan dependencies (OWASP Dependency-Check, Snyk), add security headers, expose only `health` and `info` actuator endpoints, and keep audit logs.
- **Internal APIs:** don't expose them publicly. Put them in a private network or VPC, reachable only through the gateway, and still verify tokens (zero trust).

D8. What are microservices; key things to take care of.

Microservices split an application into small, independently deployable services. Each one owns a single business capability and its own data, and they communicate over APIs or events.

Key things to take care of:

- **Boundaries:** use domain-driven design (DDD), keep each service cohesive, and give each one its own database.
- **Communication:** choose sync or async deliberately, version your APIs, and design for idempotency.
- **Resilience:** timeouts, retries, circuit breakers, bulkheads and fallbacks.
- **Data consistency:** use the Saga or Outbox pattern instead of distributed transactions.
- **Observability:** centralized logs (ELK), metrics (Prometheus/Grafana), tracing with correlation IDs.
- **Security:** gateway authentication, propagated JWTs, service-to-service authentication.
- **Config & discovery:** Config Server, Eureka or Kubernetes DNS.
- **DevOps:** a CI/CD pipeline per service, Docker/Kubernetes, health checks, contract tests.

D9. Explain the microservices architecture used in your project (template; adapt to your project).

Smart City Microservices — Reference Architecture

Template — adapt to your project

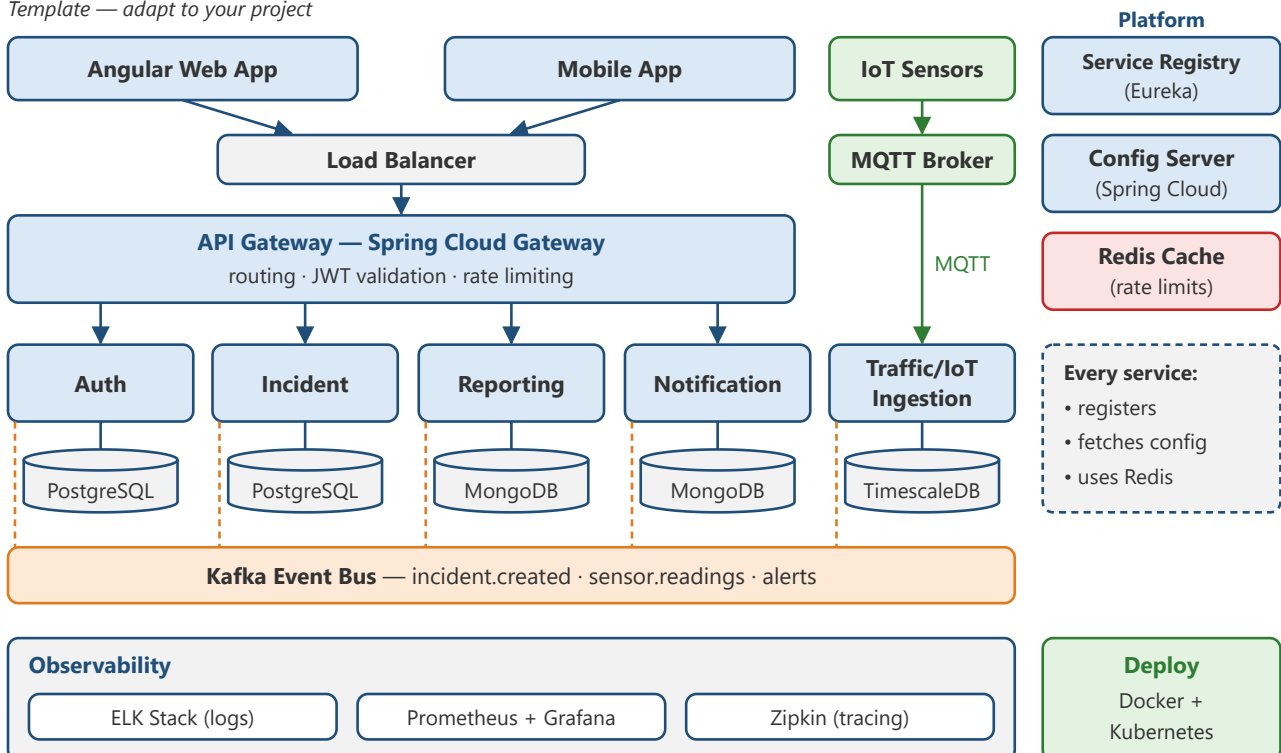
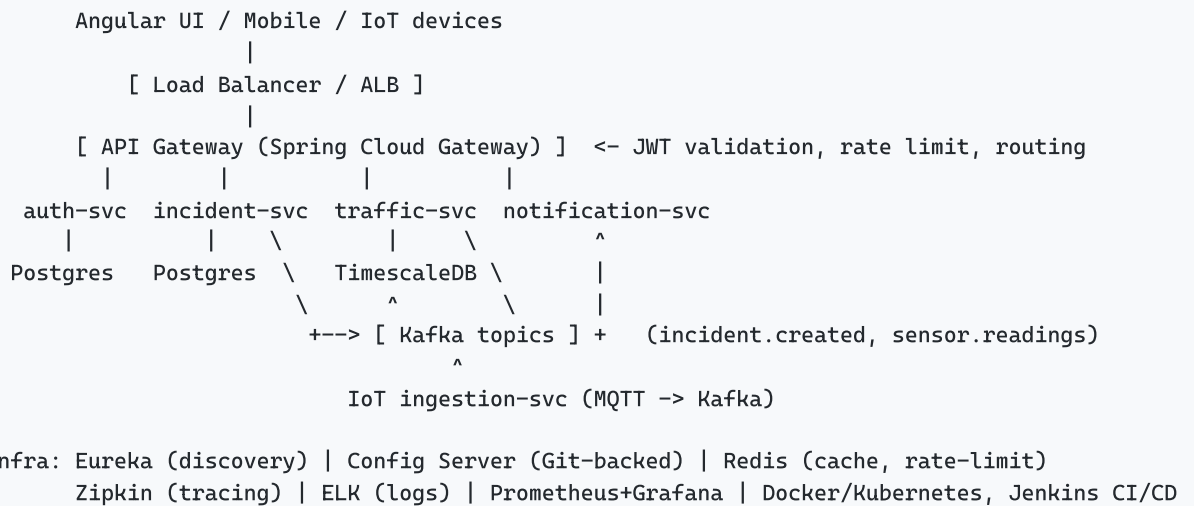


Figure 19: Smart City Microservices Reference Architecture



"Clients go through the gateway, which validates the JWT and routes to services registered in Eureka. Services pull their configuration from the Config Server. Synchronous calls use Feign with Resilience4j. Events such as `incident.created` go through Kafka, so the notification service sends SMS, email or push messages asynchronously. Redis caches master data such as zones and sensor metadata."

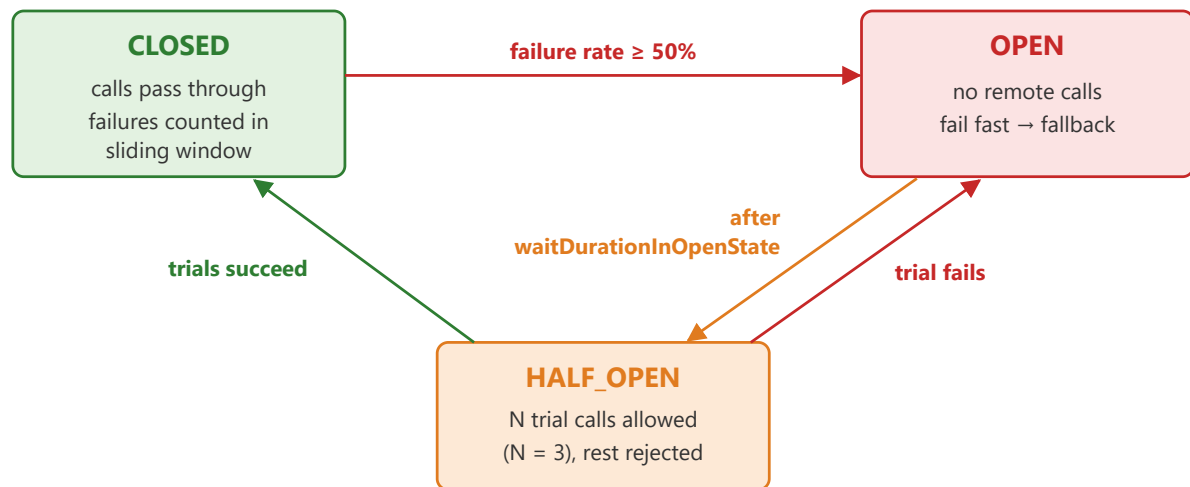
D10. How do microservices communicate?

- **Synchronous:** REST over HTTP using **OpenFeign** (declarative), **WebClient** (non-blocking) or `RestClient`, or **gRPC** (binary protobuf, fast, good for internal calls). Use it when the caller needs an immediate answer. The downside is tight runtime coupling.
- **Asynchronous:** **Kafka** (high-throughput event streams, replay) or **RabbitMQ** (task queues, routing). Use it for events and for decoupling services and absorbing load spikes.
- **Example answer:** "We use Feign plus Spring Cloud LoadBalancer for queries such as incident-svc fetching user details, and Kafka for events such as sensor readings and incident notifications."

```
@FeignClient(name = "user-service")
public interface UserClient {
    @GetMapping("/api/users/{id}")
    UserDto getUser(@PathVariable Long id);
}
```

D11. Circuit breaker.

Circuit Breaker — State Machine (Resilience4j)



Resilience4j config (application.yml)

```
slidingWindowSize: 10          waitDurationInOpenState: 10s
failureRateThreshold: 50      permittedNumberOfCallsInHalfOpenState: 3
```

Figure 20: Circuit Breaker State Machine

A circuit breaker stops calling a failing service so that failures don't cascade, and returns a fallback instead.

It has three states:

- **CLOSED**: calls go through normally. The breaker tracks the failure rate.
- **OPEN**: the failure rate went over the threshold. Calls fail fast to the fallback for the wait duration.
- **HALF_OPEN**: after the wait, a few trial calls are allowed. If they succeed the breaker goes back to **CLOSED**; if they fail it returns to **OPEN**.

```
CLOSED --(failure rate > 50%)--> OPEN --(wait 10s)--> HALF_OPEN --(trial ok)--> CLOSED
                        ^------(trial fails)-----|
```

```
@CircuitBreaker(name = "userService", fallbackMethod = "userFallback")
public UserDto getUser(Long id) { return userClient.getUser(id); }

private UserDto userFallback(Long id, Throwable ex) {
    return new UserDto(id, "Unknown", "N/A"); // or a cached value
}
```

```
resilience4j.circuitbreaker.instances.userService:
  sliding-window-size: 10
  failure-rate-threshold: 50
```

```
wait-duration-in-open-state: 10s
permitted-number-of-calls-in-half-open-state: 3
```

D12. Retry mechanism.

- Retry is for **transient** failures such as network blips and 503s. Don't retry 4xx errors or business errors.
- Use **exponential backoff with jitter** so retries don't create a thundering herd.
- Only retry **idempotent** operations, or make writes idempotent with an idempotency key.
- Put Retry *inside* the CircuitBreaker so that retries count toward the failure rate.

```
@Retry(name = "trafficApi", fallbackMethod = "fallback")
@CircuitBreaker(name = "trafficApi", fallbackMethod = "fallback")
public SignalStatus fetchStatus(String junctionId) { return client.status(junctionId); }
```

```
resilience4j.retry.instances.trafficApi:
  max-attempts: 3
  wait-duration: 500ms
  enable-exponential-backoff: true
  exponential-backoff-multiplier: 2
  retry-exceptions: [java.io.IOException, java.util.concurrent.TimeoutException]
```

- **Spring Retry** is an alternative: `@EnableRetry`, then `@Retryable(retryFor = IOException.class, maxAttempts = 3, backoff = @Backoff(delay = 500, multiplier = 2))` with a `@Recover` method.
- **Kafka**: use `@RetryableTopic(attempts = "4", backoff = @Backoff(delay = 1000, multiplier = 2))`. Failed messages go to retry topics and finally to a **DLT** (`@DltHandler`) for alerting or manual replay. Alternatively use `DefaultErrorHandler` with `DeadLetterPublishingRecoverer`.

D13. If Microservice B is down, what should A do?

1. **Timeouts**: never wait forever. Set connect and read timeouts, and use a Resilience4j `@TimeLimiter`.
2. **Retry** a few times with backoff, but only for transient errors.
3. **Circuit breaker + fallback**: fail fast and return a default, a **cached** last-known response (Redis), or a partial response.
4. **Go async**: if B's work can wait, publish to Kafka or a queue. B consumes the message when it recovers. The user gets "202 Accepted, processing".
5. **Graceful degradation**: for example, the incident dashboard still loads but shows "weather data unavailable".
6. **Bulkhead**: isolate the thread pools for calls to B so A's other features keep working.
7. **Alerting/monitoring**: health checks, Prometheus alerts on circuit state, Kubernetes restarts the pods, and multiple B replicas behind a load balancer.

D14. API Gateway (Spring Cloud Gateway).

API Gateway — Single Entry Point (Spring Cloud Gateway)

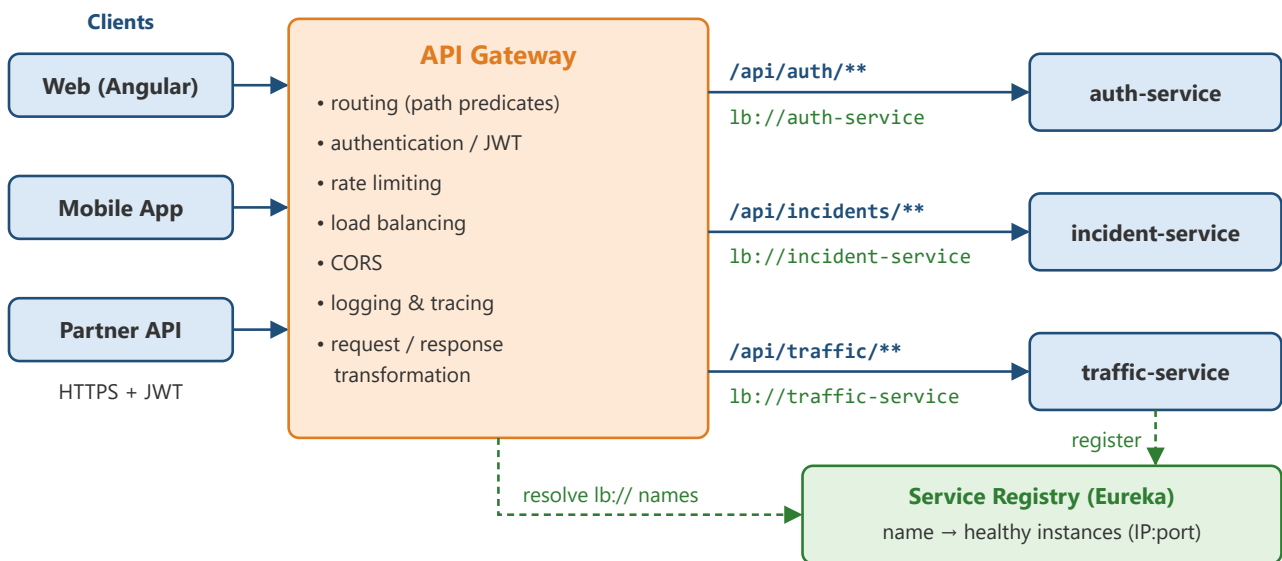


Figure 21: API Gateway Routing with Eureka

- It is the single entry point for all clients.
- Responsibilities:
 - **routing** by path or host
 - **authentication** (JWT validation)
 - **rate limiting** (Redis)
 - **load balancing** (`lb://service-name` via discovery)
 - CORS
 - request/response header rewriting
 - circuit breaker and fallback at the edge
 - logging and correlation ID injection
 - SSL termination
 - API aggregation and versioning
- Spring Cloud Gateway is reactive (Netty/WebFlux), built on **routes, predicates and filters**.

```
spring.cloud.gateway.routes:  
- id: incident-service  
  uri: lb://INCIDENT-SERVICE  
  predicates: [ Path=/api/incidents/** ]  
  filters:  
    - name: RequestRateLimiter  
      args: { redis-rate-limiter.replenishRate: 50, redis-rate-limiter.burstCapacity: 100 }  
    - name: CircuitBreaker  
      args: { name: incidentCB, fallbackUri: forward:/fallback/incidents }
```

D15. Load Balancer; Application Load Balancer.

- A load balancer distributes traffic across instances to give scalability and high availability. It health-checks instances and removes unhealthy ones.
- **L4 (transport, TCP/UDP)**: routes by IP and port. It is fast but cannot see content. AWS NLB is an example.
- **L7 (application, HTTP)**: routes by path, host, headers or cookies. It can do SSL termination and sticky sessions. AWS **ALB** is an example: `/api/traffic/*` goes to the traffic target group, and `admin.city.gov` goes to the admin UI.
- **Server-side vs client-side load balancing**: an ALB or Nginx sits in front of the system (server-side). **Spring Cloud LoadBalancer** is client-side: the caller gets the instance list from Eureka and picks one itself (round-robin by default). Feign and `@LoadBalanced` WebClient/RestTemplate use it automatically.

```
@Bean @LoadBalanced
WebClient.Builder webClientBuilder() { return WebClient.builder(); }
// webClientBuilder.build().get().uri("http://traffic-service/api/signals")...
```

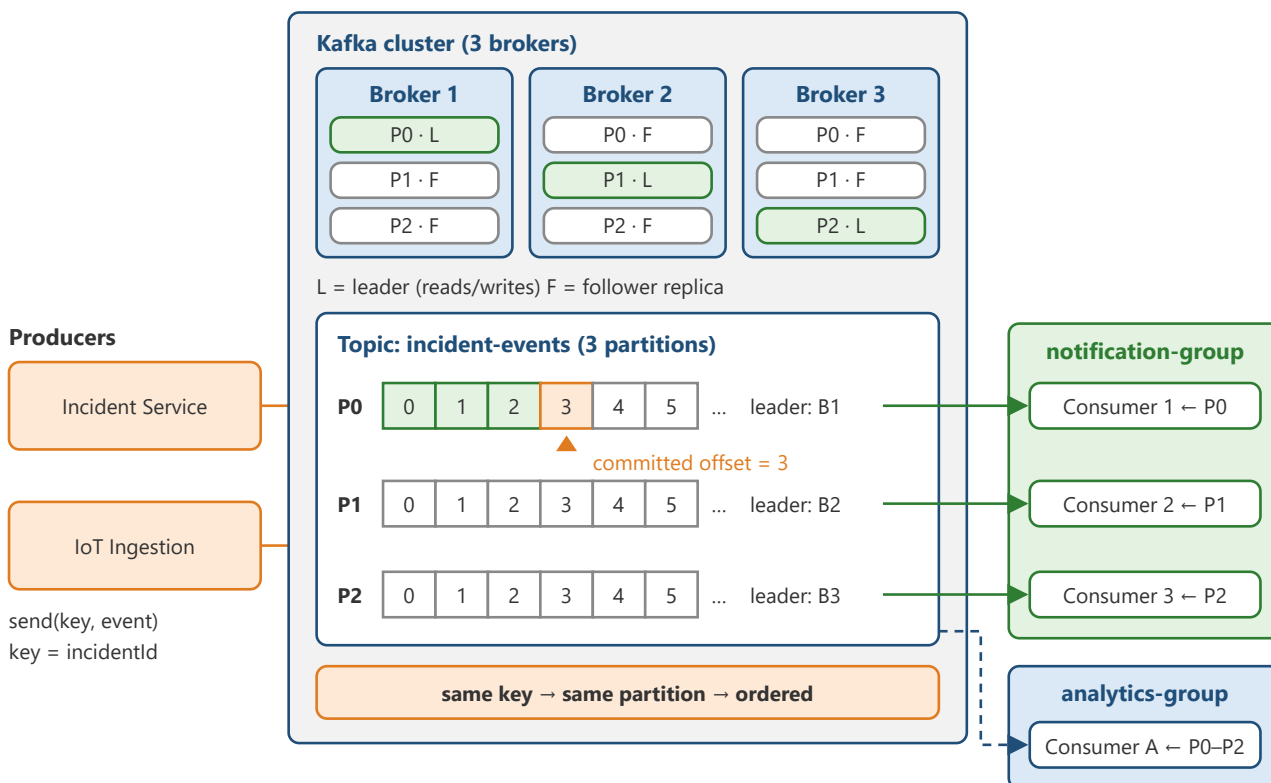
D16. Distributed caching and distributed tracing.

- **Distributed caching** means a cache shared by all instances. **Redis** is the usual choice; **Hazelcast** is an embedded or clustered alternative. Each instance's local cache would go stale independently, so a shared cache keeps them consistent.
 - Use it for master data, sessions, refresh tokens and rate limits.
 - Spring setup: `@EnableCaching` plus `@Cacheable("zones")`, `@CachePut`, `@CacheEvict`.
 - Always set a TTL. Handle cache invalidation and stampede (locking, or jittered TTLs).
- **Distributed tracing** follows one request across services.
 - A **traceld** is shared by the whole request. A **spanId** identifies one operation or hop.
 - In Spring Boot 3, use **Micrometer Tracing** (Brave or OpenTelemetry bridge) and export to **Zipkin** or **Jaeger**.
 - The IDs propagate through HTTP headers (W3C `traceparent`) and Kafka headers, and they appear in logs through MDC, so you can grep ELK by `traceld`.

```
management.tracing.sampling.probability: 1.0           # lower in prod, e.g. 0.1
management.zipkin.tracing.endpoint: http://zipkin:9411/api/v2/spans
logging.pattern.level: "%5p [%${spring.application.name},%X{traceId},%X{spanId}]"
```

D17. What is Kafka and its working flow.

Kafka working flow — topic "incident-events", replication factor 3



Each group keeps its own committed offsets: analytics-group reads the same events independently.

Figure 22: Kafka working flow: producers, brokers, partitions, replicas and consumer groups

Kafka is a distributed, durable, high-throughput **event streaming platform** built on an append-only log.

```

Producer --(key)--> Topic "sensor.readings" [P0][P1][P2] (on Brokers, replicated)
                    |   |   |
Consumer Group "analytics": C1<-P0,P1  C2<-P2
Consumer Group "alerts":    C3<-P0,P1,P2 (each group gets all messages)
  
```

- **Producer** sends records to a topic. The **key** decides the partition ($\text{hash}(\text{key}) \% \text{partitions}$), so records with the same key, such as the same sensor ID, land in the same partition and keep their order.
- **Topic** is split into **partitions**, which are the unit of parallelism. Ordering is guaranteed **only within a partition**.
- **Broker** is a Kafka server. A cluster holds the partitions. Each partition has a **leader** plus followers (**replication factor** of 3 is typical). `acks=all` together with `min.insync.replicas=2` gives durability.
- **Consumer group**: each partition is consumed by exactly one consumer in a group. Adding consumers (up to the partition count) scales consumption. Different groups each read every message independently.
- **Offsets** are a consumer's position in a partition, committed to `__consumer_offsets`. Messages are retained by time or size, not deleted on read, so they can be **replayed**.
- **Reliability**: idempotent producers and transactions give exactly-once semantics. Manual offset commits combined with idempotent consumers give safe processing.

D18. Explain one microservice you worked on (template: Incident Management Service; adapt to your project).

Responsibility: create, assign, track and close city incidents (accidents, waterlogging, sensor alarms) raised by operators, citizens or IoT alerts. **APIs:** `POST /api/incidents`, `GET /api/incidents?zone=&status=&page=`, `PUT /api/incidents/{id}/assign`, `PATCH /api/incidents/{id}/status`, `GET /api/incidents/{id}/timeline`. **Stack:** Spring Boot 3, Spring Data JPA, PostgreSQL (with PostGIS for location queries), Redis cache for zones and SLA configuration, Flyway migrations. **Events:** it consumes `sensor.alerts` and auto-creates incidents when a threshold is breached. It publishes `incident.created`, `incident.assigned` and `incident.closed`, which the notification-svc (SMS/email/push) and the dashboard (WebSocket) consume. **Security:** JWT, with `@PreAuthorize` checks so operators only see their own zone. **Challenges and fixes:**

- Duplicate alerts from flapping sensors: fixed with idempotency keys plus a Redis de-dup window.
- Slow list API: fixed with pagination, composite indexes on (zone, status, created_at), and DTO projections to avoid N+1 queries.
- Lost events when the DB commit succeeded but the Kafka publish failed: fixed with the **Outbox pattern**.
- SLA escalation: a scheduled job runs with ShedLock so only one instance executes it.

D19. When integrating 2–3 microservices, do they connect directly or through shared services?

Both. Services call each other **directly** over the network (Feign/WebClient, using a service name), but they rely on **shared infrastructure** to do it:

- **Service registry (Eureka/Kubernetes DNS):** finds instances by name. No hard-coded IPs.
- **Config Server:** central configuration (URLs, timeouts, feature flags).
- **API Gateway:** handles external client traffic. Internal calls usually skip it.
- **Message broker (Kafka):** for async and decoupled integration, so services don't need to know about each other at all.
- **Rules:** services must **never share a database**. Share contracts (OpenAPI, event schemas), not code-level coupling.

D20. Complete API request flow through microservices.

End-to-end API request — POST /api/incidents

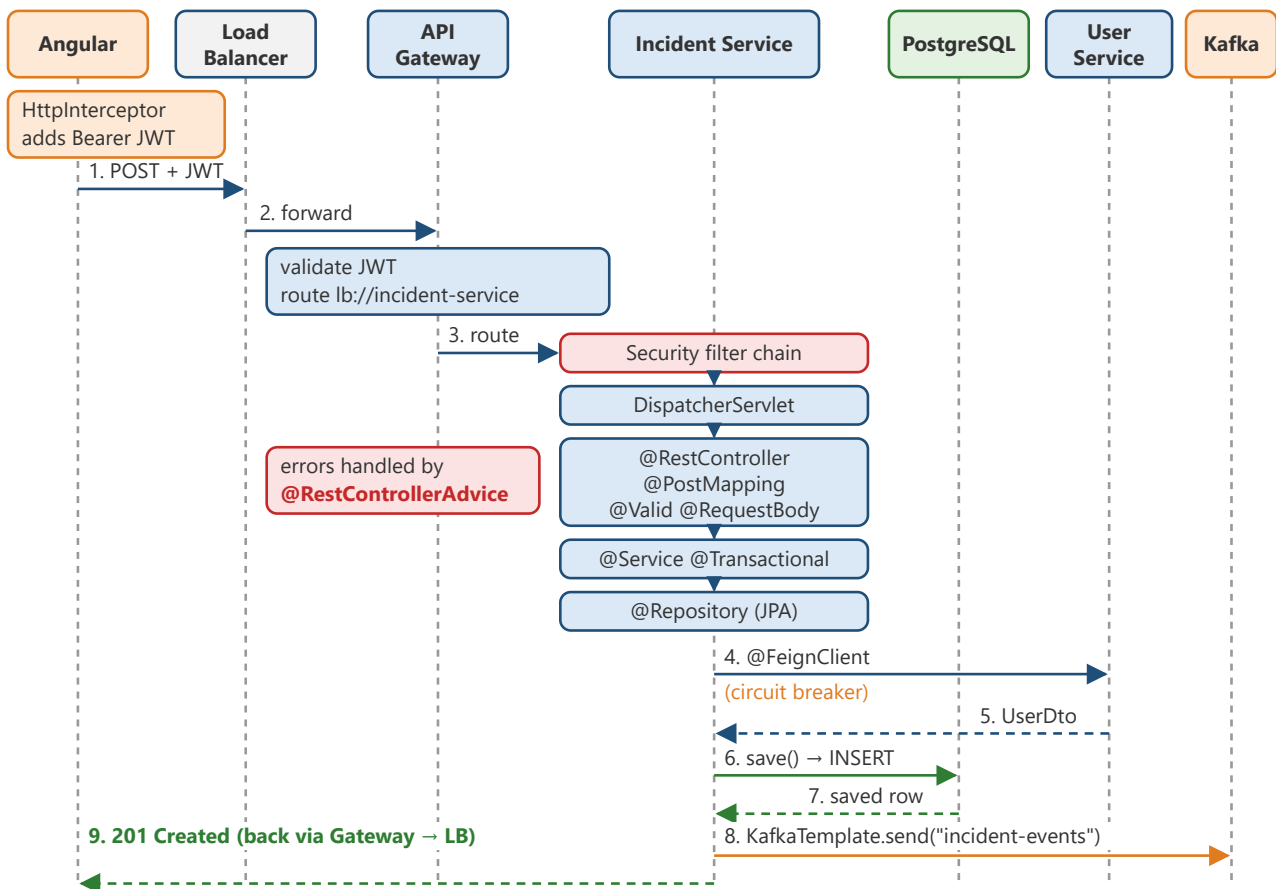


Figure 23: End-to-end API request sequence: Angular to Spring Boot incident service, PostgreSQL, User Service and Kafka

Angular -> HTTPS GET /api/incidents/42 (Authorization: Bearer <JWT>)
 -> ALB -> API Gateway: GlobalFilter adds X-Correlation-Id, validates JWT, rate limit, route lb://INCIDENT-SERVICE
 -> Spring Cloud LoadBalancer picks instance from Eureka
 -> incident-service: Servlet filters -> SecurityFilterChain (JwtAuthFilter/resource server -> SecurityContextHolder)
 -> DispatcherServlet -> @RestController @RequestMapping("/api/incidents") -> @GetMapping("/{id}") (@PathVariable, @Valid)
 -> @Service (@PreAuthorize, @Transactional(readOnly = true)) -> may call user-service via @FeignClient (+ @CircuitBreaker)
 -> @Repository (JpaRepository) -> Hibernate -> HikariCP -> PostgreSQL
 -> map Entity -> DTO -> (for writes) KafkaTemplate.send("incident.updated", event) after commit
 -> ResponseEntity<IncidentDto> -> Jackson JSON -> back through gateway -> client
 Errors anywhere in the MVC layer -> @RestControllerAdvice + @ExceptionHandler -> standard error JSON
 Security errors in filters -> AuthenticationEntryPoint (401) / AccessDeniedHandler (403)

```

@RestController
@RequestMapping("/api/incidents")
@RequiredArgsConstructor
public class IncidentController {
    private final IncidentService service;

    @GetMapping("/{id}")
    
```

```

    public ResponseEntity<IncidentDto> get(@PathVariable Long id) {
        return ResponseEntity.ok(service.findById(id));
    }
}

```

D21. API works internally but not for the client: primary investigation.

1. **Reproduce and get the facts.** Get the exact URL, the status code (none, 401, 403, 404, 502, CORS error?) and a timestamp or correlation ID.
2. **Network:** check DNS resolution, firewall and security groups, VPN/whitelisting, proxies, and whether the port is actually exposed externally.
3. **Gateway/LB:** check whether a route is defined for that path, path rewrites and prefix stripping, and ALB target health.
4. **CORS:** a browser-only failure with a preflight `OPTIONS` failing usually means the client origin is not in `allowedOrigins`.
5. **Auth:** check whether the client's token is valid, unexpired, issued by the right issuer/audience, and carries the right roles. 401 means authentication failed; 403 means it lacks the role.
6. **SSL/TLS:** look for expired or self-signed certificates, hostname mismatches, or TLS version mismatches.
7. **Contract:** check API version, request payload/headers, `Content-Type`, and environment-specific base URLs or config (profile differences).
8. **Logs:** trace the correlation ID through gateway and service logs and Zipkin to see where the request stops.

D22. API response is slow: primary investigation.

- **Measure first, don't guess.** Use APM or **tracing (Zipkin)** to see which span is slow: gateway, service code, DB, or a downstream call. Check whether it is slow always or only under load (p95/p99 in Grafana).
- **DB (most common cause):**
 - check query time in logs (`spring.jpa.show-sql` or the Hibernate statistics)
 - **N+1** queries (fix with `JOIN FETCH` or `@EntityGraph`)
 - missing indexes
 - fetching too much data (use pagination and projections)
- **External calls:** slow downstream services or missing timeouts. Fix with parallel calls (`CompletableFuture` / `WebClient`) or caching.
- **Resources:**
 - Thread pool exhaustion (Tomcat threads) or **HikariCP connection pool** exhaustion: look for "connection is not available" errors.
 - CPU, memory, **GC pauses** (GC logs, heap dump).
 - Thread dumps for locks or blocking.
- **Payload:** large JSON responses. Fix with compression, pagination, or returning fewer fields.
- **Fixes:** caching (Redis), indexes, async processing, scaling out.

D23. Delay caused by a stored procedure / SQL query: identify and fix.

Identify:

- the DB **slow query log** (MySQL), `pg_stat_statements` (Postgres), or AWR/SQL Profiler (Oracle/SQL Server)
- APM traces, and Hibernate SQL logs with timings

Analyze: run **EXPLAIN / EXPLAIN ANALYZE** (the execution plan) and look for:

- full table scans
- wrong join order
- high row estimates versus actual rows
- sorts or temp tables spilling to disk
- functions on indexed columns (`WHERE DATE(created_at) = ...` kills the index)

Fix:

- **Indexes:** composite indexes in the right column order, covering indexes.
- **Queries:** avoid `SELECT *`. Rewrite correlated subqueries as joins or `EXISTS`. Filter early.
- **Paging:** use **pagination** (keyset paging for big tables).
- **Statistics:** update stats or `ANALYZE`, rebuild fragmented indexes.
- **Stored procedure internals:** remove row-by-row cursors and use set-based logic. Watch for parameter sniffing.
- **Writes:** batch them (`hibernate.jdbc.batch_size`).
- **Reads:** cache frequently read results. Use materialized views or summary tables for reports. Archive or partition old data (for example, sensor readings by month).
- **Verify:** re-run the execution plan and compare timings before and after.

D24. Design patterns used in microservices, and in code.

Microservice / architecture patterns:

Pattern	One-liner (Smart City example)
API Gateway	Single entry point for auth, routing and rate limiting (Spring Cloud Gateway)
Service Registry/Discovery	Services register and find each other by name (Eureka)
Circuit Breaker	Fail fast plus fallback when a dependency is down (Resilience4j)
Database per Service	Each service owns its own schema; no shared DB
Saga	Distributed transaction as local transactions plus compensations (choreography via Kafka or an orchestrator)
CQRS	Separate write model from read model (e.g. dashboard read store)
Event Sourcing	Store state changes as events (incident timeline) and rebuild state by replaying them
Outbox	Write the event to an outbox table in the same DB transaction; a relay/Debezium publishes it to Kafka

Pattern	One-liner (Smart City example)
Strangler Fig	Gradually route features from the legacy monolith to new services
Sidecar	Helper container next to the service (Envoy/Istio proxy, log shipper)
Externalized Config	Config Server / ConfigMaps

Code-level patterns:

- **Singleton:** Spring beans are singleton-scoped by default.
- **Factory:** `NotificationSenderFactory` returns an SMS, Email or Push sender. Spring's `BeanFactory` is another example.
- **Builder:** Lombok `@Builder` for DTOs and events. `WebClient.builder()` is another example.
- **Strategy:** a `NotificationChannel` interface with multiple implementations, injected as a `Map<String, NotificationChannel>` and picked at runtime.
- **Observer:** Spring `ApplicationEventPublisher` with `@EventListener`; Kafka pub/sub follows the same idea.
- **Others worth naming:**
 - Template Method (`JdbcTemplate` , `KafkaTemplate`)
 - Proxy (Spring AOP, `@Transactional`)
 - Adapter (wrappers for third-party IoT vendor APIs)

```
public interface NotificationChannel { void send(Alert alert); }

@Component("SMS")    class SmsChannel    implements NotificationChannel { public void send(Alert a) { /*
... */ } }
@Component("EMAIL") class EmailChannel implements NotificationChannel { public void send(Alert a) { /*
... */ } }

@Service
@RequiredArgsConstructor
class NotificationService {
    private final Map<String, NotificationChannel> channels; // Strategy selected at runtime
    void notify(Alert alert) { channels.get(alert.channel()).send(alert); }
}
```

D25. Cloud experience: which cloud services and tools have you used?

Template answer. Keep only what you actually used, and be ready to go deeper on each one.

- "In my [Smart City] projects at Trinity Mobility, our Spring Boot microservices ran on [AWS]. We deployed containers on [EKS/ECS] (some services on [EC2]). Data was in [RDS PostgreSQL/MySQL], files and media in [S3], caching in [ElastiCache Redis], and events in [MSK Kafka]."
- "Traffic came in through an [ALB]. We used [CloudWatch] for logs, metrics and alarms, [IAM] roles for access, and [Secrets Manager] for DB credentials."
- "For tooling, we used Docker images, Kubernetes manifests/Helm, CI/CD on [Jenkins/GitHub Actions], and [Terraform] for infrastructure as code. My own part was [writing Dockerfiles and pipelines / debugging pods / setting up alarms]."

Need	AWS	Azure	GCP
VMs	EC2	Virtual Machines	Compute Engine
Kubernetes	EKS (or ECS)	AKS	GKE
Relational DB	RDS	Azure SQL / Database for PostgreSQL	Cloud SQL
Object storage	S3	Blob Storage	Cloud Storage
Cache	ElastiCache	Azure Cache for Redis	Memorystore
Kafka / messaging	MSK	Event Hubs (Kafka API)	Pub/Sub
Load balancer	ALB	Application Gateway	Cloud Load Balancing
Monitoring	CloudWatch	Azure Monitor	Cloud Monitoring/Logging
Identity / secrets	IAM, Secrets Manager	Entra ID, Key Vault	IAM, Secret Manager

D26. Stateless vs stateful services and authentication.

The source list had a garbled line here; this is the most likely question.

- **Stateless:** the server keeps no session. Each request carries everything needed to authenticate it, usually a JWT in the `Authorization` header. Any instance can serve any request, so the service scales out easily behind a load balancer.
- **Stateful:** the server keeps the session (`HttpSession` , `JSESSIONID` cookie). Then you need either sticky sessions or a shared session store such as **Spring Session + Redis**, so that the session survives when an instance restarts or a request lands on another instance.

Aspect	Stateless (JWT)	Stateful (session)
Where state lives	In the token/client	On the server / session store
Horizontal scaling	Easy, any instance	Needs sticky sessions or shared store
Logout / revoke	Hard: use short expiry, refresh tokens, a denylist	Easy: invalidate the session
Payload per request	Larger (token)	Small cookie
Typical fit	REST APIs, microservices, mobile/SPA (Angular)	Server-rendered web apps, admin portals

- **When each fits:** use stateless JWT (e.g. Spring Security OAuth2 resource server) for microservices behind an API gateway. Use stateful sessions when you need instant revocation or are building a classic MVC app. In that case, store the sessions in Redis.

E. SQL, Angular/JavaScript, Project & HR

Sample schema used below employee(id, name, salary, dept_id) · department(id, name) · users(id, first_name, last_name) · transactions(id, sender_id, receiver_id, amount)

E1. Find the second-highest salary (3 ways) + employees who earn it

```
-- 1) Subquery: the highest salary below the overall MAX (works on every DB)
SELECT MAX(salary) AS second_highest
FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);

-- 2) DISTINCT + LIMIT/OFFSET (MySQL / PostgreSQL)
SELECT DISTINCT salary FROM employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
-- Oracle 12c+ / SQL Server 2012+
SELECT DISTINCT salary FROM employee
ORDER BY salary DESC
OFFSET 1 ROWS FETCH NEXT 1 ROWS ONLY;

-- 3) DENSE_RANK: handles ties and generalises to the Nth highest
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employee
) t WHERE rnk = 2
GROUP BY salary; -- or use DISTINCT

-- Name + salary of every employee on the second-highest salary
SELECT name, salary FROM (
    SELECT name, salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employee
) t
WHERE rnk = 2;
```

Use `DISTINCT` (or `DENSE_RANK`) so that duplicate top salaries don't push the real second-highest value out. Way 1 returns NULL when no second value exists. For the Nth highest, change `2` to `N`, or use `OFFSET N-1`.

E2. Highest-paid employee(s) in each department (ties included)

```
-- Window function (MySQL 8+, PostgreSQL, Oracle, SQL Server)
SELECT name, salary, dept_id FROM (
    SELECT e.*, RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM employee e
) t
WHERE rnk = 1;

-- Without window functions (correlated / tuple subquery)
SELECT e.name, e.salary, d.name AS dept
FROM employee e JOIN department d ON d.id = e.dept_id
WHERE (e.dept_id, e.salary) IN (
```

```
SELECT dept_id, MAX(salary) FROM employee GROUP BY dept_id
);
```

`RANK / DENSE_RANK` keep ties. `ROW_NUMBER` returns exactly one row per department and drops the ties. SQL Server has no tuple `IN`, so use `WHERE e.salary = (SELECT MAX(salary) FROM employee x WHERE x.dept_id = e.dept_id)`.

E3. Name and salary of employees in each department

```
SELECT d.name AS department, e.name AS employee, e.salary
FROM employee e
JOIN department d ON d.id = e.dept_id
ORDER BY d.name, e.salary DESC;
```

Use `LEFT JOIN department ...` starting from `department` if departments with no employees should still appear (`FROM department d LEFT JOIN employee e ON ...`).

E4. Window functions

Window Functions: ROW_NUMBER vs RANK vs DENSE_RANK

`fn() OVER (ORDER BY salary DESC)`

name	dept	salary	ROW_NUMBER	RANK	DENSE_RANK
A	IT	9000	1	1	1
B	IT	8000	2	2	2
C	HR	8000	3	2	2
D	HR	7000	4	4	3

tie

ROW_NUMBER()

always unique: 1, 2, 3, 4
ties ordered arbitrarily

RANK()

ties share a rank,
next rank skips: 1, 2, 2, 4

DENSE_RANK()

ties share a rank,
no gaps: 1, 2, 2, 3

PARTITION BY dept: ranking restarts in each partition

`RANK() OVER (PARTITION BY dept ORDER BY salary DESC)`

partition: dept = 'IT'

name	salary	rank
A	9000	1
B	8000	2

partition: dept = 'HR'

name	salary	rank
C	8000	1
D	7000	2

C restarts at rank 1 in HR; without `PARTITION BY` the whole result set is one window.

Figure 24: SQL window ranking functions: `ROW_NUMBER`, `RANK`, `DENSE_RANK`

A window function calculates a value across a set of related rows (the "window"). Unlike `GROUP BY`, it does **not** collapse those rows. Syntax: `fn() OVER (PARTITION BY ... ORDER BY ... [ROWS ...])`.

```

SELECT name, dept_id, salary,
       ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS drnk,
       LAG(salary) OVER (PARTITION BY dept_id ORDER BY salary DESC) AS prev_salary,
       LEAD(salary) OVER (PARTITION BY dept_id ORDER BY salary DESC) AS next_salary,
       SUM(salary) OVER (PARTITION BY dept_id) AS dept_total,
       SUM(salary) OVER (PARTITION BY dept_id ORDER BY salary DESC) AS running_total
FROM employee;

```

salary	ROW_NUMBER	RANK	DENSE_RANK
900	1	1	1
800	2	2	2
800	3	2	2
700	4	4 (gap)	3 (no gap)

- **ROW_NUMBER**: a unique sequence. Typical uses are de-duplication and top-N per group.
- **RANK**: tied rows get the same rank, and the next rank skips numbers.
- **DENSE_RANK**: tied rows get the same rank, with no gaps.
- **LAG/LEAD**: read the previous or next row's value, e.g. to compare with the last reading or compute deltas.
- **SUM/AVG OVER**: per-group totals or running totals, without needing `GROUP BY`.

E5. ACID properties

- **Atomicity**: either the whole transaction happens or none of it does. On failure it rolls back (`@Transactional` rolls back on a `RuntimeException`).
- **Consistency**: a transaction moves the database from one valid state to another. Constraints, FKs and triggers still hold afterwards.
- **Isolation**: concurrent transactions don't see each other's uncommitted work. The isolation levels are `READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ` and `SERIALIZABLE`. They prevent dirty reads, non-repeatable reads and phantom reads respectively.
- **Durability**: once a transaction commits, it survives a crash. This is done with write-ahead/redo logs.

Example: a money transfer debits one account and credits another, and both must succeed together.

E6. Stored procedures: what, why, "hierarchy"

What: a precompiled, named block of SQL plus procedural logic, stored in the database. It accepts IN/OUT/INOUT parameters and is invoked with `CALL` or `EXEC`. **Why**: it saves network round-trips because the logic runs next to the data, gives a reusable and centrally owned piece of business logic, offers security (grant `EXECUTE` without granting table access), and benefits from a cached execution plan. **Downsides**: it is harder to version, test and port, and logic gets split between the app and the DB.

Hierarchy / how the DB objects relate:

- **Nested procedures**: a procedure can `CALL` other procedures. For example, `process_order` calls `validate_stock`, then `create_invoice`, then `log_audit`. Nesting depth is limited (SQL Server allows 32).

Transaction handling, i.e. who commits, should normally sit in the outermost procedure.

- **Functions:** return a single value or a table and can be used inside `SELECT / WHERE`. They generally cannot perform DML or commit. Procedures can call functions. A function cannot call a procedure that modifies data.
- **Triggers:** run automatically on INSERT/UPDATE/DELETE. They can call procedures.
- **Views:** saved SELECTs (a virtual table). Procedures and functions can query them.

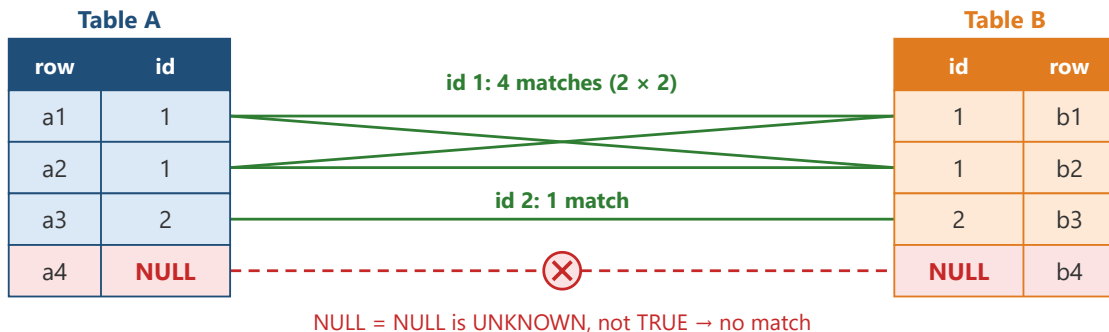
```
-- MySQL
DELIMITER //
CREATE PROCEDURE raise_dept_salary(IN p_dept INT, IN p_pct DECIMAL(5,2))
BEGIN
    UPDATE employee SET salary = salary * (1 + p_pct/100) WHERE dept_id = p_dept;
    CALL log_audit('RAISE', p_dept);      -- nested call
END //
DELIMITER ;
CALL raise_dept_salary(10, 5);
```

From Spring you can call it with `@Procedure` (Spring Data JPA), `SimpleJdbcCall`, or `entityManager.createStoredProcedureQuery(...)`.

E7. Table A and Table B hold the same records: what does INNER JOIN return?

INNER JOIN with Duplicates and NULLs

`SELECT * FROM A INNER JOIN B ON A.id = B.id`



Result: 5 rows

INNER JOIN rule

keeps only row pairs where the ON condition evaluates to TRUE

A.row	A.id	B.id	B.row
a1	1	1	b1
a1	1	1	b2
a2	1	1	b1
a2	1	1	b2
a3	2	2	b3

Duplicates multiply

id 1: $2 \times 2 = 4$ rows
id 2: $1 \times 1 = 1$ row
NULL: 0 rows
total = 5 rows

Figure 25: INNER JOIN with duplicate keys and NULLs

If the join column is unique in both tables, you get every row once, i.e. as many rows as the table has.

Duplicates multiply and **NULLs never match** (`NULL = NULL` evaluates to unknown).

```

A.val: 1, 1, 2, NULL      B.val: 1, 1, 2, NULL
SELECT * FROM A JOIN B ON A.val = B.val;
-> val 1: 2 x 2 = 4 rows
-> val 2: 1 x 1 = 1 row
-> NULL : 0 rows
Total = 5 rows (not 4)

```

For a value that appears m times in A and n times in B, the join produces $m \times n$ rows.

E8. 20 records starting from the 5th row (first and last name)

```

-- MySQL / PostgreSQL
SELECT first_name, last_name FROM users
ORDER BY id
LIMIT 20 OFFSET 4;      -- MySQL also accepts: LIMIT 4, 20

-- Oracle 12c+ / SQL Server 2012+
SELECT first_name, last_name FROM users
ORDER BY id
OFFSET 4 ROWS FETCH NEXT 20 ROWS ONLY;

```

`OFFSET 4` skips rows 1–4, so the result begins at row 5. Always include `ORDER BY`, because without it the row order is not deterministic. For pagination the general formula is `OFFSET (page-1)*size`. Spring Data's `Pageable` produces this SQL for you.

E9. INNER JOIN between two tables

```

SELECT e.id, e.name, e.salary, d.name AS department
FROM employee e
INNER JOIN department d ON e.dept_id = d.id
WHERE e.salary > 50000;

```

Only rows that match in both tables are returned. Employees with no department, or a NULL `dept_id`, are excluded. If you need them, use `LEFT JOIN`.

E10. Highest receiver amount

```

-- (a) Receiver who received the highest TOTAL amount
SELECT receiver_id, SUM(amount) AS total_received
FROM transactions
GROUP BY receiver_id
ORDER BY total_received DESC
LIMIT 1;      -- Oracle/SQL Server: FETCH FIRST 1 ROWS ONLY / TOP 1

-- with the receiver's name, and ties kept
SELECT u.first_name, u.last_name, t.total_received FROM (
  SELECT receiver_id, SUM(amount) AS total_received,
    RANK() OVER (ORDER BY SUM(amount) DESC) AS rnk
  FROM transactions GROUP BY receiver_id
) t JOIN users u ON u.id = t.receiver_id
WHERE t.rnk = 1;

```

```
-- (b) The single largest transaction and who received it
SELECT receiver_id, amount FROM transactions
WHERE amount = (SELECT MAX(amount) FROM transactions);
```

Ask the interviewer which meaning they intend: the highest total, or the single biggest transaction.

E11. Explain Angular

Angular architecture — component, DI, HTTP and routing

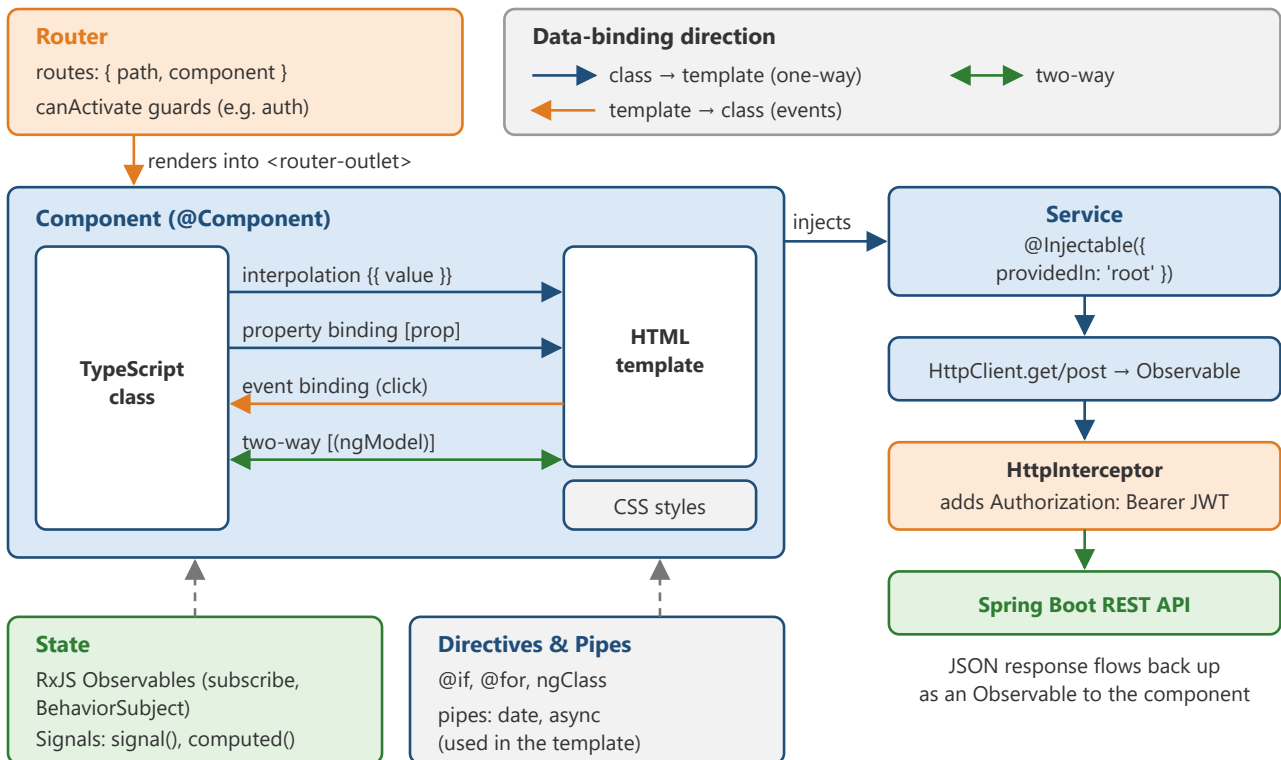


Figure 26: Angular architecture: component bindings, services, HttpClient, interceptor, router, state, directives and pipes

Angular is Google's **component-based, TypeScript framework** for building SPAs. Its main pieces:

- **Components** (template + class + styles) and **standalone components**, which are the default since v17 and don't need an `NgModule`. Older apps use **NgModules**.
- **Data binding**: interpolation `{{ }}`, property binding `[prop]`, event binding `(event)`, and two-way binding `[(ngModel)]`.
- **Dependency injection**: **services** declared with `@Injectable({providedIn: 'root'})` hold shared logic and API calls.
- **RxJS** Observables for async work, e.g. `HttpClient`, `async` pipe, `switchMap`, `debounceTime`.
- **Router**: lazy-loaded routes, guards (`canActivate`), resolvers.
- **Forms**: template-driven and reactive (`FormGroup`, validators).
- **Signals** (v16+): `signal()`, `computed()`, `effect()` give fine-grained reactivity. Together with zoneless change detection they are the newer alternative to `Zone.js`.
- Also: directives, pipes, HTTP interceptors (attach JWTs, handle errors), lifecycle hooks (`ngOnInit`, `ngOnDestroy`), and the CLI for scaffolding and builds.

E12. Your Angular work / roles / how you introduce yourself as an Angular developer (TEMPLATE)

Replace the [placeholders] with your real work. Don't claim anything you can't explain.

"I'm a full-stack developer with about [X] years in Angular, currently on version [Angular version]. At Trinity Mobility I work on the [module, e.g. incident management / traffic dashboard] UI of our Smart City [product name, e.g. integrated command & control platform]. My responsibilities:

- I build reusable components such as [data grids, map views, alert panels, charts using e.g. Leaflet/OpenLayers, ECharts/Chart.js] and lazy-loaded feature modules or standalone components.
- I integrate Spring Boot REST APIs through services, using `HttpClient`, interceptors for [JWT/Keycloak] auth, and global error handling.
- I handle real-time data with [WebSockets/STOMP/SSE] and RxJS to show [live IoT sensor readings / incident updates].
- I build reactive forms with validation for [e.g. incident creation, device configuration], plus role-based route guards.
- For performance I use `OnPush`, `trackBy`, lazy loading and [signals].
- I write unit tests with [Jasmine/Karma or Jest], do code reviews, and fix production UI issues. One thing I'm proud of is [e.g. reducing dashboard load time from X s to Y s by ...]."

E13. Basic JavaScript concepts

- **Types:** the primitives are string, number, bigint, boolean, undefined, null and symbol. Everything else is an object, including arrays and functions. `typeof null === 'object'` is a historical quirk.
- **Closures:** a function remembers the variables of the scope where it was defined, even after that scope has returned. Closures are used for private state, factories and memoisation.
- **this:** set by how a function is called. As an object method it is the object. In a plain call it is `undefined` in strict mode, or `window` otherwise. `call` / `apply` / `bind` set it explicitly. Arrow functions don't have their own `this` and inherit it lexically.
- **Promises / async-await:** a Promise is a future value that is pending, fulfilled or rejected. `async/await` is syntactic sugar over promises; use `try/catch` for errors and `Promise.all` to run work in parallel.
- **Event loop:** JS runs on a single thread with a call stack. When the stack is empty, all **microtasks** run first (promise callbacks), then the next **macrotask** (`setTimeout`, I/O). So `setTimeout(fn, 0)` runs after `Promise.resolve().then(fn)`.
- **== vs ===:** `==` coerces types (`'5' == 5` is true, `null == undefined` is true). `===` compares value and type with no coercion. Default to `===`.

E14. var, let, const and hoisting (TDZ)

JavaScript: var vs let vs const (Scope, Hoisting, TDZ)

var	let	const
Scope function (or global)	Scope block { }	Scope block { }
Hoisting hoisted + set to undefined	Hoisting hoisted; TDZ until declared	Hoisting hoisted; TDZ until declared
Redeclare (same scope) allowed	Redeclare (same scope) SyntaxError	Redeclare (same scope) SyntaxError
Reassign allowed	Reassign allowed	Reassign TypeError; must initialise
<pre>if (true) { var v = 1; } v; // 1 (leaks block)</pre>	<pre>if (true) { let l = 1; } l; // ReferenceError</pre>	<pre>const o = { n: 1 }; o.n = 2; // OK, mutable o = {}; // TypeError</pre>

Temporal Dead Zone (TDZ) inside a block



With var, reading x before its declaration gives undefined instead (no TDZ).

Figure 27: JavaScript var, let, const and hoisting

	Scope	Re-declare	Re-assign	Hoisted
var	function	yes	yes	yes, initialised to undefined
let	block	no	yes	yes, but stays in the TDZ until its declaration line
const	block	no	no (object contents can still change)	yes, but stays in the TDZ

```
console.log(a); // undefined (var is hoisted and initialised)
var a = 1;

console.log(b); // ReferenceError: Cannot access 'b' before initialization (TDZ)
let b = 2;

const c = { x: 1 };
c.x = 2; // OK: the object itself is mutable
// c = {}; // TypeError: Assignment to constant variable

for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3 3 3
for (let j = 0; j < 3; j++) setTimeout(() => console.log(j)); // 0 1 2 (new binding per iteration)

sayHi(); // "hi": function declarations are fully hoisted
function sayHi() { console.log('hi'); }
```

E15. Tell me about yourself (TEMPLATE, ~60–90 s)

Fill in with real facts. Structure: present, then past, then highlights, then why this role.

"Hi, I'm [Name]. I'm a Java full-stack developer with [X] years of experience, currently at **Trinity Mobility**. There I work on [product, e.g. our Smart City integrated command & control platform], which helps [city agencies / police / traffic departments] monitor [IoT sensors, CCTV, traffic, incidents] in real time. My core stack is **Java [version], Spring Boot, microservices, [PostgreSQL/MySQL], and Angular**, along with [Kafka / Redis / Docker / Kubernetes] where applicable. Recently I [key contribution, e.g. built the alert-processing service / designed REST APIs for the incident module / built the live dashboard], which [impact: e.g. reduced response time by X% / handled Y events per second]. Before that I [previous role, project or education in brief]. I enjoy [solving performance problems / owning features end to end], and I'm now looking for [the reason you want this role, e.g. larger-scale distributed systems / product ownership], which is why this position interests me."

E16. Roles and responsibilities (TEMPLATE)

Choose the bullets that are true for you and add specifics.

- I design and develop **Spring Boot microservices and REST APIs** for [module, e.g. incident / asset / alert management].
- I build **Angular** screens and components for [dashboards, maps, reports] and integrate them with backend APIs.
- I write and optimise **SQL**: queries, indexes, [stored procedures], [Flyway/Liquibase] migrations.
- I integrate [IoT/sensor feeds, third-party APIs, GIS, SMS/email gateways] through [Kafka / REST / MQTT].
- I write **unit and integration tests** ([JUnit 5, Mockito, Testcontainers]) and take part in **code reviews**.
- I take part in **Agile** ceremonies (sprint planning, stand-ups, estimation) and update [JIRA] tickets.
- I support **CI/CD and deployments** ([Jenkins/GitLab CI, Docker, K8s]), handle production issues and do RCA.
- I [mentor juniors / coordinate with QA, BA and client teams / write technical documentation].

E17. Explain your project (TEMPLATE)

Use this 5-part structure and fill it with your real project details.

1. **Problem:** "[City/client] needed to [e.g. monitor traffic, incidents and civic services from one place instead of N disconnected systems]."
2. **Solution / product:** "We built [product name], a [web-based command & control / Smart City platform] that ingests data from [CCTV, traffic signals, environment sensors, GPS vehicles, 3rd-party systems] and shows it on [live dashboards / GIS maps] with [alerts, SOP workflows, reports]."
3. **Users:** "[Control-room operators, police/traffic departments, municipal officials, admins]."
4. **Modules:** "[e.g. Device/Asset management, Incident management, Alert & notification engine, GIS map, Reports & analytics, User/role management]."

5. **My part:** "I owned [module or features]. I built [APIs / services / Angular screens], handled [integration X], and improved [performance or reliability item]."
6. **Impact:** "[e.g. incident response time dropped from X to Y, the platform handles Z events/day, it's deployed in N cities]." *Use only numbers you can defend.*

E18. Architecture and codebase structure (TEMPLATE + typical layout)

Describe what your project actually does. The flow below is a typical shape to adapt.

"Our system is [microservices / modular monolith]. The **Angular SPA** calls an **API Gateway** ([Spring Cloud Gateway / NGINX]). The gateway handles [JWT/Keycloak] auth and routes requests to services such as [device-service, incident-service, alert-service, notification-service, report-service]. Services find each other through [Eureka / K8s DNS] and get configuration from [Config Server / ConfigMaps]. IoT and third-party data arrive through [Kafka / MQTT], [Redis] is used for caching, and data is stored in [PostgreSQL/MySQL + MongoDB/time-series DB]. Everything runs on [Docker/Kubernetes] and is deployed via [Jenkins], with [ELK / Prometheus-Grafana] for monitoring."

Typical Spring Boot service layout

```
incident-service/  
└─ src/main/java/com/trinity/incident/  
    │ controller/      REST endpoints (@RestController)  
    │ service/         business logic (interfaces + impl)  
    │ repository/      Spring Data JPA repositories  
    │ entity/          JPA entities  
    │ dto/             request/response DTOs, mapper/ (MapStruct)  
    │ config/          security, Kafka, Swagger, CORS config  
    │ client/          Feign/WebClient clients to other services  
    │ messaging/       Kafka producers/consumers  
    │ exception/       custom exceptions + @RestControllerAdvice  
    └─ util/  
└─ src/main/resources/ application.yml, db/migration (Flyway)  
└─ src/test/java/      unit + integration tests
```

Typical Angular layout

```
src/app/  
└─ core/              singleton services, interceptors, guards, auth  
└─ shared/            reusable components, pipes, directives, models  
└─ features/          incident/, devices/, dashboard/, reports/ (lazy-loaded)  
    │ layout/          header, sidebar, shell  
    └─ app.routes.ts / app.config.ts  
src/environments/     environment.ts, environment.prod.ts
```

E19. Technologies and tools (TEMPLATE: pick what you actually used)

Only list tools you can answer follow-up questions on.

- **Backend:** Java [8/11/17/21], Spring Boot, Spring Security, Spring Data JPA/Hibernate, Spring Cloud (Gateway, Config, Eureka, OpenFeign), Resilience4j
- **Frontend:** Angular [ver], TypeScript, RxJS, Angular Material/PrimeNG, Bootstrap, Leaflet/OpenLayers, Chart.js/ECharts
- **Databases:** PostgreSQL / MySQL / Oracle, MongoDB, Redis, [PostGIS / InfluxDB / Elasticsearch]
- **Messaging:** Kafka / RabbitMQ / MQTT
- **DevOps:** Git (GitHub/GitLab/Bitbucket), Maven/Gradle, Jenkins / GitLab CI, Docker, Kubernetes, Helm, NGINX
- **Quality and testing:** JUnit 5, Mockito, Testcontainers, SonarQube, Jasmine/Karma/Jest, Postman, Swagger/OpenAPI
- **Monitoring:** ELK, Prometheus + Grafana, Zipkin/Jaeger
- **Process:** JIRA, Confluence, Agile/Scrum
- **Other languages and scripting:** Python [scripts / data processing], Shell/Bash, [Node.js]

For "apart from Java/Spring Boot/Angular": "I've also worked with [Kafka for event streaming, Docker/K8s for deployment, Redis for caching, Python for scripts], plus the usual tooling: Git, Jenkins, Postman, Swagger, SonarQube and JIRA."

E20. Coding standards, best practices and precautions

- **Clean code:** meaningful names, small single-purpose methods, SOLID, DRY, no magic numbers, consistent formatting (Checkstyle/Prettier/ESLint).
- **Layering:** Controller, then Service, then Repository. Use DTOs, never expose entities, and use constructor injection.
- **Validation and errors:** Bean Validation (`@Valid`), a global `@RestControllerAdvice` , meaningful HTTP status codes, and no swallowed exceptions.
- **Security:** parameterised queries (no SQL injection), no secrets in code (use Vault/env/K8s secrets), input sanitisation, least privilege, OWASP Top 10, keep dependencies updated.
- **Performance:** avoid N+1 (fetch joins / `@EntityGraph`), paginate, add indexes, cache where it makes sense, use connection pools, and set timeouts on external calls.
- **Transactions and concurrency:** keep `@Transactional` boundaries correct, use optimistic locking (`@Version`), prefer immutable or thread-safe objects.
- **Logging:** SLF4J at the right levels, correlation/trace IDs, never log PII or passwords.
- **Testing and review:** unit tests for business logic, a $[\geq X\%]$ coverage gate in SonarQube, PR reviews, and passing CI before merge.
- **Angular:** `OnPush`, `trackBy` , `unsubscribe ( async pipe / takeUntilDestroyed )` , strict TypeScript, lazy loading, no business logic in templates.
- **Git:** small focused commits, a branch per feature, meaningful messages, and no direct pushes to main.

E21. Challenges faced (STAR template + examples)

STAR: Situation (context), Task (your responsibility), Action (what *you* did, technically), Result (measurable outcome plus what you learned). Keep it to 60–90 s.

The two scenarios below are **examples only**. Replace them with your own real experience.

Example 1: slow dashboard API. S: The [incident dashboard] API took [8 s] under load as data grew. T: I was asked to bring it under [1 s]. A: I profiled it with [logs / Hibernate statistics] and found N+1 queries plus a missing index. I added a fetch join with DTO projections, a composite index on `(status, created_at)`, pagination, and a [Redis] cache for reference data. R: Response time dropped to [600 ms], and the team adopted query reviews.

Example 2: lost or duplicate sensor events. S: During peak hours some [IoT alerts] were duplicated and others were missed. T: I had to make ingestion reliable. A: I made the [Kafka] consumer idempotent with a unique event-ID check, used manual offset commits after a successful DB write, added retries with a dead-letter topic, and added alerting on consumer lag. R: No duplicate alerts after release, and failed events became traceable in the DLT.

E22. How do you handle conflicts? (STAR template)

Core message: listen first, focus on facts and goals rather than people, find common ground, and escalate only when needed.

"**S:** In [sprint/project], a [teammate / QA / frontend dev] and I disagreed about [e.g. API contract design / whether to refactor or patch / an estimate]. **T:** We had to agree quickly to stay on schedule. **A:** I set up a short one-to-one call and listened to their concerns first. I laid out the trade-offs using data ([e.g. performance numbers, a quick PoC]) and suggested [a compromise, e.g. patch now and refactor in the next sprint as a tech-debt ticket]. Where we couldn't agree, we took it to [the tech lead] with both options written down. **R:** We delivered on time, the [decision] was documented, and the working relationship got better. I learned to [discuss trade-offs early and in writing]."

E23. How do you integrate third-party data?

- **Pull (REST):** `WebClient` (reactive) or `OpenFeign / RestClient` calling the vendor APIs. For periodic sync use `@Scheduled` or Quartz/ShedLock in a cluster. Track a `lastSyncedAt` value or cursor to do incremental pulls.
- **Push:** expose **webhook** endpoints. Verify the signature or API key, acknowledge quickly (2xx), and process asynchronously.
- **Streaming:** consume from **Kafka/MQTT** for high-volume IoT and sensor feeds, and scale with consumer groups.
- **Mapping and validation:** use an adapter/anti-corruption layer to translate vendor DTOs into the internal model (`MapStruct`), validate the schema, and reject or quarantine bad records.
- **Resilience:** timeouts, **retries with exponential backoff**, **circuit breaker** (`Resilience4j`), rate limiting, fallbacks, and a dead-letter queue for failures.
- **Idempotency:** dedupe on the vendor event ID or a hash, and use upserts, so retries never create duplicates.
- **Storage:** keep the **raw payload** (for audit and replay) separately from the **normalised** tables used by the app.
- **Security and ops:** keep credentials in a vault or secrets store, use OAuth2 client-credentials where supported, and log or monitor sync failures with alerts.

Template: "In our project we integrate [e.g. traffic signal controllers / weather API / GPS vehicle tracking] using [mechanism]. My part was [building the adapter / scheduler / consumer, plus retry and idempotency handling]."

E24. Microservices in your project / how you handle microservices architecture (TEMPLATE)

See Section D for the detailed concepts (gateway, discovery, config, resilience, Saga, tracing).

"Our platform is split into [N] services by business capability, for example [device, incident, alert, notification, user, report]. Each service owns its database and is deployed independently on [Docker/K8s]. Clients come in through [Spring Cloud Gateway], which handles [JWT] auth and routing. Services talk to each other **synchronously** over REST/[Feign], with timeouts and [Resilience4j] circuit breakers. They also talk **asynchronously** over [Kafka] for events such as `IncidentCreated` and `SensorThresholdBreached`. We use [Config Server/ConfigMaps] for configuration and [Eureka/K8s DNS] for discovery. For observability we use [ELK + Zipkin/Prometheus-Grafana] and propagate correlation IDs. Cross-service consistency is handled with [eventual consistency / Saga / outbox]. I personally [built/maintained X service, its APIs and its Kafka integration]."

E25. "Given a list of strings, write..." (question incomplete)

Note: the source cut this question off (G2Healthcare Q26). **Fill in the full question if you remember it.** Below are 3 likely completions with one-line Java Stream answers.

```
List<String> list = List.of("apple", "bat", "avocado", "cat", "apple", "banana");

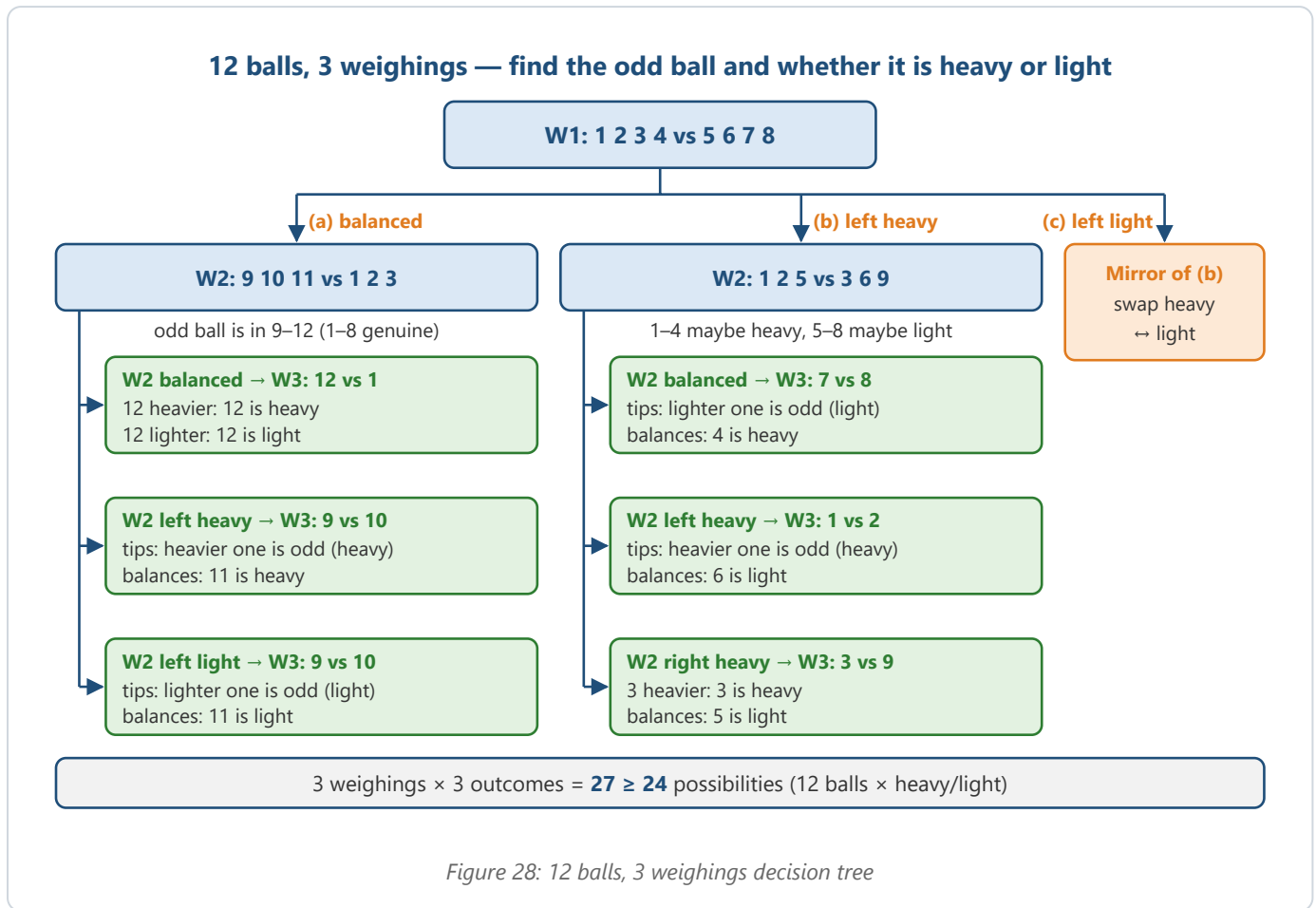
// 1) Group strings by length
Map<Integer, List<String>> byLen = list.stream().collect(Collectors.groupingBy(String::length));

// 2) Find strings starting with a given letter (e.g. "a")
List<String> startsA = list.stream().filter(s -> s.startsWith("a")).toList(); // Java 16+; else
collect(Collectors.toList())

// 3) Count the frequency of each string
Map<String, Long> freq = list.stream().collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()));
```

Other common variants: remove duplicates (`distinct()`), sort by length (`sorted(Comparator.comparing(String::length))`), find the first non-repeated string (`LinkedHashMap` counting, then filter `count == 1`), join with commas (`Collectors.joining(",")`), and uppercase all (`map(String::toUpperCase)`).

E26. Puzzle: 12 balls, one odd (heavier or lighter), find it in 3 weighings.



- **Why 3 weighings are enough:** each weighing has 3 outcomes, so 3 weighings give $3^3 = 27$ outcomes. There are 24 possibilities (12 balls × heavy or light), and $27 \geq 24$.
- Label the balls 1–12. **Weighing 1: 1,2,3,4 vs 5,6,7,8.**
- **If balanced:** the odd ball is in 9–12, and 1–8 are known good. **Weighing 2: 9,10,11 vs 1,2,3.**
 - Balanced: 12 is the odd ball. Weigh 12 vs 1 to find out whether it is heavy or light.
 - 9–11 side heavier: the odd ball is heavy and is one of 9–11. Weigh 9 vs 10. The heavier one is odd; if balanced, it is 11.
 - 9–11 side lighter: same as above, but the lighter one is odd; if balanced, it is 11.
- **If 1,2,3,4 > 5,6,7,8:** the odd ball is one of 1–4 (heavy) or one of 5–8 (light). **Weighing 2: 1,2,5 vs 3,6,9** (9 is known good).
 - Balanced: it is 4 (heavy), 7 (light) or 8 (light). Weigh 7 vs 8. The lighter one is odd; if balanced, it is 4 (heavy).
 - Left heavier: it is 1 (heavy), 2 (heavy) or 6 (light). Weigh 1 vs 2. The heavier one is odd; if balanced, it is 6 (light).
 - Right heavier: it is 3 (heavy) or 5 (light). Weigh 3 vs 9. If 3 is heavier, 3 is the heavy ball; if balanced, 5 is the light ball.
- **If 1,2,3,4 < 5,6,7,8:** use the same steps with heavy and light swapped.

E27. Puzzle: remove exactly half the water from a full cylindrical glass.

- Tilt the glass slowly and let water pour out. Stop when the water surface touches the rim on one side and the very edge of the base on the other side.
 - That diagonal water surface passes through the cylinder's centre. Because a cylinder is symmetric about its centre, the plane cuts it into two equal volumes, so exactly half the water remains.
 - This does not work for a tapered (conical) glass. It is not symmetric about its centre, so the two parts would not be equal.
-

F. Production & Troubleshooting Scenarios

F1. Your application suddenly throws `OutOfMemoryError` in production. How will you identify the root cause and fix it?

1. Read the exact message: `Java heap space`, `Metaspace`, `GC overhead limit exceeded` and `unable to create native thread` each have a different cause.
2. Make sure the JVM writes a dump when it fails: `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps`. If the pod is still alive, take one yourself with `jcmd <pid> GC.heap_dump /dumps/heap.hprof`.
3. Open the dump in Eclipse MAT. Use the Leak Suspects report and the Dominator Tree to find what retains the most memory, for example a static `Map` cache, an unbounded `List`, or `ThreadLocal`s that are never cleared.
4. Compare the trend of Actuator `jvm.memory.used` in Grafana with the deployment timeline.
5. Fix the code, for example with a bounded Caffeine cache, streaming or pagination instead of `findAll()`, or closing resources. Only then size the heap correctly (`-XX:MaxRAMPercentage=75` in Kubernetes).
6. Add an alert on heap usage above 85% after GC.

F2. Multiple users are updating the same data simultaneously and wrong data is getting stored. How will you handle this concurrency issue?

1. Confirm it is a lost update: two transactions read the same version and the last write wins.
2. **Optimistic locking** (the default choice): add a `@Version` column. Hibernate adds `WHERE version=?` to the update and throws `ObjectOptimisticLockingFailureException`. Return 409 to the user or retry.
3. **Pessimistic locking** when contention is high, for example a wallet balance: `@Lock(LockModeType.PESSIMISTIC_WRITE)`, which issues `SELECT ... FOR UPDATE`.
4. Prefer an atomic SQL update to read-modify-write: `UPDATE account SET bal = bal - :amt WHERE id=:id AND bal >= :amt`.
5. Across several instances, use a distributed lock (Redis/Redisson) only when the database cannot enforce the rule.

```
@Entity class Incident { @Id Long id; @Version Long version; String status; }
```

F3. One API is taking 20 seconds response time only in production, but locally it works fine. How will you debug it?

1. Look at the trace (Zipkin/Jaeger, Micrometer Tracing) to see which span takes the time: database, a downstream HTTP call, or the service's own code.
2. Check Actuator `/actuator/metrics/http.server.requests?tag=uri:/api/x` for p95/p99 latency. Also check `hikaricp.connections.pending`, because waiting for a connection is a common cause.
3. Database: production has far more data than local. Run `EXPLAIN ANALYZE` on the query and look for a missing index or a sequential scan. Enable `pg_stat_statements` to find slow queries.

4. Take 3–4 `jstack <pid>` dumps a few seconds apart while the request runs, to see where threads are blocked (socket read, lock, pool wait).
5. Check what differs in production: DNS or network latency, missing timeouts on `RestClient / WebClient`, SSL handshakes, logging set to DEBUG.
6. Fix it (index, timeout, caching, async), then confirm with a load test.

F4. CPU usage suddenly increased to 95% in production server. What steps will you take to investigate?

1. Find the process: `top`, or `kubectl top pod`. Check whether the spike lines up with a deployment, a traffic peak or a batch job.
2. Find the busy threads: `top -H -p <pid>`. Convert the thread ID to hex (`printf '%x' <tid>`) and match it to `nid=0x...` in the `jstack <pid>` output.
3. Take several thread dumps. If the same stack keeps showing up, that is usually the culprit: an infinite loop, regex backtracking, heavy JSON serialization or a busy-wait.
4. Check whether GC is the cause with `jstat -gcutil <pid> 1000`. Frequent full GCs burn CPU.
5. Profile with JFR (`jcmd <pid> JFR.start duration=60s filename=cpu.jfr`) or async-profiler to get a flame graph.
6. Mitigate first (scale out with HPA, roll back), then fix the code and add CPU alerts.

F5. Your application is creating too many threads and server becomes slow. How will you identify thread leak?

1. Watch the Actuator metric `jvm.threads.live` and `jvm.threads.states` over time. Steady growth without a plateau means a leak.
2. Run `jcmd <pid> Thread.print` (or `jstack`) and group the threads by name. A large count of `pool-N-thread-M` threads points to a new `ExecutorService` being created per request.
3. Typical causes: `Executors.newFixedThreadPool()` inside a method and never shut down, `new Thread()` in a loop, an HTTP client created per call, `@Async` running on the default executor.
4. Fix it with one shared, bounded `ThreadPoolTaskExecutor` bean with names that show where the threads come from. Use `shutdown()` in `@PreDestroy`.
5. Java 21 virtual threads are an option, but Java 17 needs bounded pools.

```
@Bean ThreadPoolTaskExecutor reportExecutor() {
    var ex = new ThreadPoolTaskExecutor();
    ex.setCorePoolSize(8); ex.setMaxPoolSize(16); ex.setQueueCapacity(500);
    ex.setThreadNamePrefix("report-"); return ex;
}
```

F6. One scheduled job is processing duplicate records multiple times. How will you prevent duplicate processing?

1. Find out why. Either several pods run the same job, the previous run is still going when the next one starts, or records are not marked as processed atomically.

2. Use a status column and claim rows atomically: `SELECT ... FOR UPDATE SKIP LOCKED`, then set `status='PROCESSING'` in the same transaction.
3. Make the processing idempotent with a unique constraint on a business key or an idempotency key, so a duplicate insert fails safely.
4. For clustered deployments, use ShedLock so only one instance runs the job.
5. Use `fixedDelay` instead of `fixedRate` so runs do not overlap.

```
SELECT * FROM events WHERE status='NEW' ORDER BY id LIMIT 100 FOR UPDATE SKIP LOCKED;
```

F7. Garbage collection time suddenly increased and application became slow. How will you troubleshoot?

1. Enable GC logs: `-Xlog:gc*:file=/logs/gc.log:time,uptime:filecount=5,filesize=20m`. Analyse them with GCEasy or GCViewer, looking at pause times, frequency and full GCs.
2. Run `jstat -gcutil <pid> 1000`. If old gen stays near 100% after each GC, there is a leak or the heap is too small.
3. Take a heap dump (`jcmd <pid> GC.heap_dump`) and use MAT to find the objects that fill old gen.
4. Use JFR's allocation profiling to find code that allocates heavily, such as large temporary collections, string concatenation in loops or loading whole result sets.
5. Tune only after fixing the code: G1 (the default in Java 17) with `-XX:MaxGCPauseMillis=200` and the right `-Xmx / MaxRAMPercentage`. Consider ZGC for large heaps.
6. Check that the container memory limit is not causing throttling or OOMKills.

F8. Large Excel/file processing causes application crash due to memory issue. How will you optimize it?

1. Find the cause: loading the whole file into memory (`XSSFWorkbook`, `Files.readAllBytes`, `MultipartFile.getBytes()`).
2. Read in streaming mode: Apache POI event model / `XSSFReader` (SAX), the `excel-streaming-reader` library, or EasyExcel. For CSV, read line by line with `BufferedReader` / `Files.lines`.
3. Write large Excel files with `SXSSFWorkbook(100)`, which keeps only 100 rows in memory.
4. Process in chunks (for example 1,000 rows) with JDBC batch inserts. Spring Batch with a chunk-oriented step gives restartability.
5. Move it to async: upload to S3/MinIO, return 202 with a job ID, and process in a background worker.
6. Limit the upload size with `spring.servlet.multipart.max-file-size`.

```
try (var wb = new SXSSFWorkbook(100)) { var sh = wb.createSheet(); /* write rows */ wb.write(out);
wb.dispose(); }
```

F9. A production issue occurs randomly and logs are not clear. How will you debug step by step?

1. Collect the facts: timestamps, affected users or endpoints, frequency. Look for a pattern such as a specific pod, a time of day or particular input.
2. Improve observability: add a correlation or trace ID to the MDC (Micrometer Tracing does this automatically), use structured JSON logs, and log the inputs and outcomes at the important decision points.
3. Raise log levels at runtime without redeploying: `POST /actuator/loggers/com.trinity.x {"configuredLevel": "DEBUG"}`.
4. Correlate the logs with metrics in Grafana and traces in Zipkin/Jaeger. Check GC pauses, connection pool waits, and pod restarts (`kubectl describe pod` , `kubectl logs --previous`).
5. Look for intermittent causes: race conditions, timeouts, cache expiry, one bad node.
6. Reproduce it in staging with production-like data and load (JMeter/Gatling). Write a regression test, fix it, and add an alert.

F10. One API works for small data but fails for huge data. What performance improvements will you do?

1. Add pagination: `Pageable` , or keyset pagination for deep pages. Never return unbounded lists.
2. Fix the queries: use projections or DTOs instead of whole entities, remove N+1 queries (`JOIN FETCH` , `@EntityGraph` , `hibernate.default_batch_fetch_size`), add indexes, and run `EXPLAIN ANALYZE` .
3. Stream large exports with `Stream<T>` plus `@Transactional(readOnly=true)` and a fetch size, or `StreamingResponseBody` , so they do not build a list in memory.
4. Move heavy work to async processing (202 plus job status, or Kafka).
5. Compress responses: `server.compression.enabled=true` .
6. Cache hot reads with Redis.
7. Tune timeouts and pool sizes, then confirm with a load test (Gatling).

F11. Two microservices are communicating and one dependent service is down. How will your service handle the failure without impacting users?

1. Set connect and read timeouts on every outbound call so threads do not hang.
2. Add a Resilience4j circuit breaker so the service fails fast once the dependency's error rate crosses a threshold.
3. Add a fallback: cached data (Redis), a default response or a degraded feature. For example, show the last known sensor status.
4. Use a bulkhead (a separate thread pool or semaphore) so one slow dependency cannot use up all request threads.
5. For writes, go asynchronous: publish to Kafka and let the consumer process when the dependency recovers.
6. Have Kubernetes readiness probes remove unhealthy pods, and alert on circuit breaker state changes.

```
@CircuitBreaker(name = "vehicleSvc", fallbackMethod = "cached")
public Vehicle get(String id) { return client.get(id); }
Vehicle cached(String id, Throwable t) { return cache.get(id); }
```


F12. Your API is returning 401 Unauthorized randomly for valid users. How will you debug?

1. Find the pattern: specific pods, near token expiry, after a key rotation, or only when requests go through the gateway.
2. Enable `logging.level.org.springframework.security=DEBUG` to see which filter rejects the request.
3. Decode the JWT (jwt.io) and check `exp`, `iss`, `aud` and `kid`. Check for **clock skew** between nodes, which needs NTP plus an allowed clock skew in the validator.
4. Check JWKS key rotation. A pod with a stale cached key rejects tokens signed with the new `kid`.
5. Check for a session-based setup behind a load balancer without sticky sessions or a shared session store (Spring Session Redis). Use `SessionCreationPolicy.STATELESS`.
6. Check that the token refresh logic on the frontend handles parallel requests, and that proxies are not stripping the `Authorization` header.
7. Check CORS preflight (`OPTIONS`) requests that are blocked by security.

F13. After deployment application is starting very slowly. What things will you check?

1. Read the startup logs with timestamps to see where the time goes. Use Actuator `/actuator/startup` with `BufferingApplicationStartup` for per-bean timings.
2. Check database and network: slow connections, Flyway/Liquibase migrations on large tables, `ddl-auto=update` (should be `validate` / `none` in production), Hikari waiting for the database.
3. Check `@PostConstruct` or `CommandLineRunner` code that loads caches or calls remote services.
4. Check classpath scanning: an overly broad `@ComponentScan` or unused auto-configurations (exclude them).
5. Check container resources: a low CPU limit throttles JIT and startup. Tune `resources.requests.cpu`.
6. Check Kubernetes probes: add a `startupProbe` so the liveness probe does not kill a slow-starting pod.
7. Options to speed it up: `spring.main.lazy-initialization=true` (with care), CDS/AppCDS, GraalVM native image.

F14. One API is receiving too many requests and server becomes overloaded. How will you implement rate limiting?

1. Decide the key (per user, API key, or IP) and the limit, for example 100 requests per minute.
2. Prefer enforcing it at the edge: Spring Cloud Gateway `RequestRateLimiter` with Redis (token bucket, shared across instances), or NGINX/Ingress `limit_req`.
3. Inside the service, use the Resilience4j `@RateLimiter` or Bucket4j with Redis for a distributed limit.
4. Return `429 Too Many Requests` with a `Retry-After` header.
5. Also protect the service with a bulkhead and timeouts, scale with HPA, and cache hot responses.

```
spring.cloud.gateway.routes:
- id: incidents
  uri: lb://incident-service
  predicates: [Path=/api/incidents/**]
  filters:
    - name: RequestRateLimiter
```

```
args: { redis-rate-limiter.replenishRate: 50, redis-rate-limiter.burstCapacity: 100, key-  
resolver: "#{@userKeyResolver}" }
```

F15. Your Spring Boot application works locally but fails in QA/PROD environment. How will you troubleshoot?

1. Read the actual stack trace: `kubectl logs <pod>`, `kubectl logs --previous`, `kubectl describe pod` (OOMKilled, CrashLoopBackOff, failing probes).
2. Check the active profile and the resolved properties with `/actuator/env` and `/actuator/configprops` (secured). Look for a missing env var, ConfigMap or Secret.
3. Check connectivity from inside the pod: database, Kafka, external APIs, DNS, firewall rules or NetworkPolicy, TLS certificates.
4. Compare versions: JDK, base image, dependency versions (`mvn dependency:tree`), and the database version or schema (Flyway history).
5. Check environment-specific behaviour: file paths, timezone or locale, case-sensitive table names in MySQL on Linux, memory limits.
6. Reproduce locally with the same image and config (`docker run` with the QA env vars), then fix it.

F16. A REST API is exposing sensitive data accidentally. What security improvements will you implement?

1. Never return entities. Map to response DTOs (Java `record` s) that contain only the fields the client needs. Use `@JsonIgnore` or `@JsonProperty(access = WRITE_ONLY)` for passwords as a backstop.
2. Mask PII (phone, Aadhaar, email) in responses and logs. Use a Logback masking pattern and never log full request bodies or tokens.
3. Enforce authorization: `@PreAuthorize`, plus ownership checks on the object to prevent IDOR.
4. Use generic error messages. Set `server.error.include-stacktrace=never` and `include-message=never`.
5. Lock down Actuator: `management.endpoints.web.exposure.include=health,info,prometheus` and secure everything else.
6. Use HTTPS only, encrypt sensitive columns at rest, and add security tests and a ZAP scan to CI.

F17. One external API dependency is timing out frequently. How will you implement retry and fallback mechanism?

1. Set explicit timeouts first: a connect timeout of about 2s and a read timeout based on the API's SLA.
2. Retry only idempotent calls and only transient errors (timeouts, 5xx), using exponential backoff with jitter and at most about 3 attempts.
3. Put a circuit breaker around the retry so the service stops calling once the dependency is clearly down.
4. Provide a fallback: a cached response, a default value, or queue the request for later.
5. Monitor the Resilience4j metrics in Prometheus.

```
resilience4j:  
  retry.instances.weatherApi: { maxAttempts: 3, waitDuration: 500ms, enableExponentialBackoff: true,  
    exponentialBackoffMultiplier: 2, retryExceptions: [java.io.IOException,  
    java.util.concurrent.TimeoutException] }
```

```
circuitbreaker.instances.weatherApi: { slidingWindowSize: 20, failureRateThreshold: 50,
waitDurationInOpenState: 30s }
timelimiter.instances.weatherApi: { timeoutDuration: 3s }
```

Use it as `@Retry(name="weatherApi") @CircuitBreaker(name="weatherApi", fallbackMethod="fallback")`.

F18. Application configuration differs between DEV, QA, and PROD causing issues. How will you manage environment-specific configs?

1. Put common settings in `application.yml` and only the differences in `application-{dev,qa,prod}.yml`. Activate the profile with `SPRING_PROFILES_ACTIVE`.
2. Externalize configuration: Kubernetes ConfigMaps for settings and Secrets (or Vault) for credentials. Never commit secrets.
3. Centralize with Spring Cloud Config Server backed by Git so changes are versioned and reviewed, or use Helm values per environment.
4. Bind configuration to typed `@ConfigurationProperties` records with `@Validated`, so a missing or wrong value fails at startup.
5. Use the same image in every environment and change only the configuration.
6. Check the effective configuration with `/actuator/env` and compare environments as part of the release checklist.

F19. Your API is throwing exceptions but frontend only sees generic error. How will you implement global exception handling?

1. Create a `@RestControllerAdvice` with an `@ExceptionHandler` for each exception type: validation, not found, conflict, and a generic fallback.
2. Return a consistent body. Spring Boot 3's `ProblemDetail` (RFC 7807) gives the right status code, an error code, a message and the trace ID.
3. Map business exceptions to 4xx and unexpected ones to 500 with a safe message. Log the full stack trace on the server side only.
4. Set `spring.mvc.problemdetails.enabled=true` for the built-in exceptions.
5. The frontend displays `detail` and reads the error code.

```
@RestControllerAdvice
class GlobalHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    ProblemDetail notFound(ResourceNotFoundException ex) {
        var pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setProperty("code", "RESOURCE_NOT_FOUND"); return pd;
    }
}
```

F20. A scheduled task is running multiple times in clustered deployment. How will you prevent duplicate execution?

1. Cause: every pod runs its own `@Scheduled` methods.

2. Use **ShedLock**: a lock row in the database (or Redis) means only one instance runs each scheduled execution.
3. Set `lockAtMostFor` above the worst-case runtime, and `lockAtLeastFor` to absorb clock differences between nodes.
4. Alternatives: Quartz in clustered JDBC mode, a Kubernetes `CronJob`, or leader election (Spring Integration / Kubernetes leader).
5. Still make the job idempotent as a safety net.

```
@EnableSchedulerLock(defaultLockAtMostFor = "10m")
// ...
@Scheduled(cron = "0 */5 * * * *")
@SchedulerLock(name = "syncSensors", lockAtMostFor = "4m", lockAtLeastFor = "30s")
public void syncSensors() { ... }
```

You also need a `LockProvider` bean, for example `JdbcTemplateLockProvider`, and a `shedlock` table.

F21. Your application suddenly becomes slow because of database queries. How will you identify the problematic query?

1. Find the slow requests in APM or traces (Micrometer Tracing / Zipkin), then the slow database spans inside them.
2. On the database: PostgreSQL `pg_stat_statements` (order by `total_exec_time` or `mean_exec_time`), `pg_stat_activity` for running queries and locks. MySQL: the slow query log with `long_query_time=1`, then `SHOW PROCESSLIST`.
3. In the application, log slow statements: `spring.jpa.properties.hibernate.session.events.log.LOG_QUERIES_SLOWER_THAN_MS=500`, or use `datasource-proxy` / `p6spy`. Enable Hibernate statistics to count queries per request and catch N+1.
4. Run `EXPLAIN (ANALYZE, BUFFERS)` on the query and look for sequential scans, bad join order or row estimates that are far off.
5. Fix it: an index, a rewritten query, `JOIN FETCH`, a projection. Then `ANALYZE` the table and confirm the improvement.

F22. Hibernate lazy loading exception occurs only in production. How will you fix it?

1. Cause: a lazy association is accessed after the session has closed, for example during Jackson serialization or in async code. It shows up only in production because `spring.jpa.open-in-view` behaves differently there, or a code path is not covered locally.
2. Fetch what the use case needs inside the transaction: `JOIN FETCH` in JPQL or `@EntityGraph(attributePaths="sensors")`.
3. Map to DTOs inside the `@Transactional` service method and never return entities to the controller.
4. Keep `spring.jpa.open-in-view=false` in all environments so the problem shows up early.
5. Do not use `FetchType.EAGER` everywhere or `enable_lazy_load_no_trans`, because they hide N+1 problems.

```
@EntityGraph(attributePaths = {"sensors"})
Optional<Junction> findWithSensorsById(Long id);
```

F23. Database connections are exhausted and new users cannot access application. How will you debug connection leak?

1. Confirm it from the metrics: `hikaricp.connections.active` is pinned at max, `pending` is above 0, and the logs show `Connection is not available, request timed out after 30000ms`.
2. Enable leak detection: `spring.datasource.hikari.leak-detection-threshold=20000`. Hikari logs the stack trace of the code that borrowed a connection and did not return it.
3. Check `pg_stat_activity` for connections that are `idle in transaction` and long-running queries.
4. Typical causes: manual JDBC without try-with-resources, long `@Transactional` methods that make remote calls, `Stream` results that are never closed, slow queries holding connections.
5. Fix it: try-with-resources, short transactions, set `maximum-pool-size` and `connection-timeout` correctly. Make sure the total across all pods stays under the database's `max_connections`, or use PgBouncer.

F24. One query works fast initially but becomes slow after data grows to millions of records. What optimizations will you do?

1. Run `EXPLAIN ANALYZE` and look for sequential scans, sorts that spill to disk and stale statistics. Run `ANALYZE`, and check autovacuum.
2. Add the right indexes. Composite indexes follow the column order of the WHERE clause and then ORDER BY. Use covering indexes (`INCLUDE`) and partial indexes (`WHERE status='OPEN'`).
3. Make the conditions sargable: no functions on indexed columns, no leading `%LIKE`, no implicit type casts.
4. Select only the columns you need and use keyset pagination instead of a large `OFFSET`.
5. Partition large time-series tables by month (sensor or event data), and archive old data.
6. Cache results or use read replicas and materialized views for reporting queries.

```
CREATE INDEX CONCURRENTLY idx_incident_city_status_created ON incident (city_id, status, created_at DESC);
```

F25. Duplicate records are inserted during concurrent transactions. How will you avoid it?

1. Cause: a check-then-insert race. Both transactions see "not exists" and both insert.
2. The database is the real guard. Add a **unique constraint** on the business key, for example `(vehicle_no, event_time)` or an idempotency key.
3. Use an upsert: PostgreSQL `INSERT ... ON CONFLICT (key) DO NOTHING/UPDATE`, MySQL `INSERT ... ON DUPLICATE KEY UPDATE`.
4. Catch `DataIntegrityViolationException` and treat it as "already exists".
5. For API retries, have the client send an `Idempotency-Key` header and store the processed keys.
6. For Kafka consumers, use idempotent processing with a processed-message table.

```
ALTER TABLE challan ADD CONSTRAINT uk_challan UNIQUE (vehicle_no, violation_time);  
INSERT INTO challan(...) VALUES (...) ON CONFLICT (vehicle_no, violation_time) DO NOTHING;
```

F26. A transaction partially updates data and application crashes in middle. How will you maintain consistency?

1. Within one database, put all related writes in one `@Transactional` service method. The database rolls back everything if the process crashes before commit.
2. Watch for the common pitfalls. By default only unchecked exceptions trigger rollback, so use `rollbackFor = Exception.class` for checked ones. Self-invocation bypasses the proxy. Do not catch and swallow exceptions inside the method.
3. Keep remote calls and Kafka publishes out of the database transaction. Use the **transactional outbox**: write the event to an outbox table in the same transaction, then publish it with a relay (Debezium CDC or a poller).
4. Across services, use a Saga with compensating actions.
5. For batch jobs, use Spring Batch chunk commits plus restartable job metadata, so a restart resumes from the last committed chunk.
6. Make operations idempotent so retries are safe.

F27. Deadlock occurs frequently in database during peak traffic. How will you analyze and fix it?

1. Get the deadlock details. PostgreSQL writes both statements to the server log (`deadlock detected`). Set `log_lock_waits=on`. MySQL: `SHOW ENGINE INNODB STATUS (LATEST DETECTED DEADLOCK)`, or `innodb_print_all_deadlocks=ON`.
2. Find the pattern. It is usually two transactions locking the same rows in opposite order, or missing indexes that cause wide range or gap locks.
3. Fix it by locking in a consistent order (for example sort the IDs before updating), keeping transactions short, and doing no remote calls inside them.
4. Add indexes on the WHERE and FK columns so fewer rows get locked.
5. Use optimistic locking (`@Version`) where it fits, or `SELECT ... FOR UPDATE SKIP LOCKED` for queue-like tables.
6. Retry deadlock victims (`CannotAcquireLockException` / `DeadlockLoserDataAccessException`) with Spring Retry and backoff.

F28. Pagination API becomes slow for huge tables. How will you optimize database performance?

1. Cause: `OFFSET 500000 LIMIT 20` still scans and discards 500k rows, and Spring Data `Page` also runs a `COUNT(*)` on every request.
2. Use **keyset (seek) pagination**: `WHERE (created_at, id) < (:lastTs, :lastId) ORDER BY created_at DESC, id DESC LIMIT 20`, backed by a matching composite index.
3. Return `slice` instead of `Page` to avoid the count query, or cache or estimate the total (PostgreSQL `reltuples`).
4. Select only the needed columns through a DTO projection, and do not fetch collections with paging (Hibernate HHH90003004 warns about in-memory pagination).
5. Use cursor-based API responses (`nextCursor`) for infinite scroll.

6. For filtered searches, add suitable indexes or move the search to Elasticsearch/OpenSearch.

F29. Users report inconsistent data between UI and database. How will you debug end-to-end?

1. Reproduce with a specific record and request. Compare the actual network response in browser DevTools with the database row.
2. Follow the request through each layer using the trace ID: UI state or cache, then CDN or browser cache (`Cache-Control`), then the API gateway, then the service, then Redis, then the database.
3. Check caching. A stale Redis or Caffeine entry means a missing `@CacheEvict` / `@CachePut` on update, or a TTL that is too long.
4. Check read replicas for replication lag, which shows up as read-after-write inconsistency.
5. Check the transaction and mapping layer: uncommitted reads, DTO mapping bugs, timezone conversion (store UTC, use `OffsetDateTime`), Kafka consumer lag for event-driven updates.
6. Fix the root cause and add a cache-invalidation or contract test.

F30. Batch insert operation is taking too much time. How will you improve performance?

1. Enable JDBC batching: `hibernate.jdbc.batch_size=500` , `order_inserts=true` , `order_updates=true` .
2. Do not use `GenerationType.IDENTITY` , because it disables insert batching. Use `SEQUENCE` with `allocationSize=50` , and a pooled optimizer.
3. Call `flush()` and `clear()` on the EntityManager every batch so the persistence context does not grow without limit.
4. For pure bulk loads, use `JdbcTemplate.batchUpdate` . Add `rewriteBatchedInserts=true` (PostgreSQL) or `rewriteBatchedStatements=true` (MySQL) to the JDBC URL. Use PostgreSQL `copy` for millions of rows.
5. Use one transaction per chunk, not per row, and not one huge transaction. Process chunks in parallel if ordering does not matter.
6. Drop or defer non-essential indexes and triggers during very large loads.

```
spring.jpa.properties.hibernate:
  jdbc.batch_size: 500
  order_inserts: true
  order_updates: true
```

F31. One microservice is down and another service continuously retries causing system overload. How will you handle it?

1. This is a retry storm. Limit retries to about 3 attempts with **exponential backoff and jitter** instead of immediate retries.
2. Put a **circuit breaker** in front of the retry so calls stop entirely while the dependency is down, then probe again in `HALF_OPEN` state.
3. Make sure retries are not stacked at several layers (client, gateway, service mesh). Retry at one layer only.
4. Use a bulkhead and timeouts so waiting threads cannot exhaust the pool.

5. For asynchronous work, use Kafka retry topics or a DLT (`@RetryableTopic` with backoff) instead of synchronous loops.
6. Return fallbacks or 503 quickly, and alert on circuit breaker state.

```
@RetryableTopic(attempts = "4", backoff = @Backoff(delay = 1000, multiplier = 2.0))
@KafkaListener(topics = "incident-events")
public void consume(IncidentEvent e) { ... }
```

F32. Service-to-service communication is failing intermittently in production. How will you debug network or application issue?

1. Classify the errors from logs and traces: connection refused or reset, timeout, 5xx, `UnknownHostException`. Network errors point to infrastructure, while 5xx or slow responses point to the application.
2. Check Kubernetes: `kubectl get endpoints <svc>`, pod restarts, readiness probe failures, OOMKills. Pods being removed during rolling deploys need graceful shutdown (`server.shutdown=graceful`) plus a `preStop` sleep.
3. Test from inside a pod with `curl` / `nslookup`. Check DNS, NetworkPolicies, Istio sidecar logs.
4. Check connection pool problems: stale keep-alive connections closed by a load balancer, which needs an idle timeout shorter than the LB's. Check pool exhaustion in the HTTP client.
5. Check the callee: thread pool saturation, GC pauses, database waits.
6. Fix with tuned timeouts, safe retries, circuit breakers and graceful shutdown.

F33. One microservice deployment breaks communication with another service because of API contract change. How will you avoid such issues?

1. Make only **backward-compatible** changes: add optional fields, do not rename or remove fields, set `FAIL_ON_UNKNOWN_PROPERTIES=false` on consumers (Spring Boot's default).
2. **Version** breaking changes (`/api/v2/...` or a header) and run both versions until consumers have migrated, then deprecate the old one.
3. Add **contract testing**: Spring Cloud Contract or Pact (consumer-driven) in CI, so a provider build fails if it breaks a consumer.
4. Keep an OpenAPI spec as the source of truth and diff it in CI (openapi-diff).
5. For Kafka events, use a Schema Registry (Avro/JSON Schema) with `BACKWARD` compatibility.
6. Use canary or blue-green deployments with quick rollback.

F34. Distributed transaction failed between payment service and order service. How will you maintain consistency?

1. Avoid 2PC/XA across microservices. Use the **Saga pattern** with eventual consistency.
2. Choreography: Order creates `PENDING` and emits `OrderCreated`. Payment charges and emits `PaymentCompleted` or `PaymentFailed`. Order then moves to `CONFIRMED`, or `CANCELLED` as the **compensating action**. Orchestration (a central orchestrator, Temporal/Camunda) suits more complex flows.
3. Use a **transactional outbox** in each service so a database update and its event are never lost or sent half-way.

4. Make consumers **idempotent** (a processed-event table, unique payment reference) because Kafka delivers at least once.
5. Handle timeouts: a scheduler finds orders stuck in `PENDING` and queries Payment or compensates.
6. Send failed messages to a DLT, and reconcile with the payment gateway.

F35. Users are seeing old data because services are not synchronized properly. How will you solve it?

1. Find where the staleness comes from: a cache TTL, Kafka consumer lag (`kafka-consumer-groups.sh --describe`), read replica lag, or a missed event.
2. Cache: evict or update on write (`@CacheEvict` / `@CachePut`), shorten TTLs, publish invalidation events (Redis pub/sub or Kafka) so all instances evict their local Caffeine caches.
3. Events: use the outbox pattern so no update is missed. Use partition keys (entity ID) to keep per-entity ordering. Use idempotent, version-checked consumers that ignore events older than the stored `version`.
4. Reduce consumer lag by scaling consumers up to the partition count and tuning batch size.
5. For read-your-writes, read from the primary after a write, or return the updated resource in the response.
6. Run a periodic reconciliation job, and tell users when data is eventually consistent (a "last updated" timestamp).

F36. API gateway is routing traffic incorrectly after deployment. How will you troubleshoot?

1. Confirm the symptom: which paths go to the wrong place, and whether it's all traffic or only some. Check gateway access logs for the matched route ID and the upstream URI.
2. Diff the route config between the previous and current release (Spring Cloud Gateway `predicates / filters , order`). A broader `Path=/api/**` placed before a specific route wins first.
3. Hit `/actuator/gateway/routes` to see the routes actually loaded at runtime, not just what's in Git.
4. Check service discovery. `lb://traffic-service` resolves through Eureka or a K8s Service, so confirm with `kubectl get endpoints` that the new pods are registered and old ones are gone.
5. Check `StripPrefix` / `RewritePath` filters. Wrong prefix handling often shows up as 404s from the upstream.
6. Fix the order or predicate, roll it out with a canary, and add a contract test for each route.

```
- id: incidents
  uri: lb://incident-service
  order: 0
  predicates: [ "Path=/api/incidents/**" ]
  filters: [ "StripPrefix=1" ]
```

F37. One microservice scales properly but another becomes bottleneck under high traffic. How will you identify issue?

1. Compare RED metrics (rate, errors, duration) per service in Grafana. The bottleneck shows rising p99 latency while its upstream services wait on it.

2. Use the distributed trace waterfall (Micrometer Tracing to Tempo/Zipkin) to find the span where most of the time is spent.
3. Check resource saturation: CPU throttling (`container_cpu_cfs_throttled_seconds_total`), HPA status (`kubectl get hpa`), and whether the pods have hit `maxReplicas` .
4. Check the pools inside the service: Hikari `hikaricp_connections_pending` , Tomcat busy threads, HTTP client pool size. Adding pods doesn't help if they all share one saturated DB.
5. Take a thread dump (`jcmd <pid> Thread.print`) to look for BLOCKED threads, locks, or synchronous calls.
6. Fix the actual constraint: caching, async work, a read replica, bigger pools, or HPA on a custom metric.

F38. Logs are distributed across multiple services and difficult to trace. How will you implement centralized logging and tracing?

1. Log structured JSON to stdout (logstash-logback-encoder or Spring Boot 3.4 `logging.structured.format.console=ecs`).
2. Ship the logs with Fluent Bit (DaemonSet) to Elasticsearch/OpenSearch or Loki, and view them in Kibana or Grafana.
3. Add Micrometer Tracing with the OpenTelemetry bridge. It propagates W3C `traceparent` across RestClient/WebClient/Kafka and puts `traceId` / `spanId` into the MDC.
4. Export spans over OTLP to Tempo or Jaeger, and link logs to traces using the `traceId`.
5. Follow a standard: same field names, no PII, and a correlation ID returned in a response header.

```
management:
  tracing.sampling.probability: 0.1
  otel.tracing.endpoint: http://otel-collector:4318/v1/traces
logging.pattern.level: "%5p [%${spring.application.name},%X{traceId},%X{spanId}]"
```

F39. A dependent service becomes slow and entire system response time increases. What resiliency patterns will you implement?

1. **Timeouts** on every outbound call (connect plus read). Never rely on the default infinite wait.
2. **Circuit breaker** (Resilience4j). It opens on slow-call or failure rate and fails fast.
3. **Bulkhead**: a separate thread pool or semaphore per dependency, so one slow service can't use up all Tomcat threads.
4. **Retry with exponential backoff and jitter**, only for idempotent calls and limited to 2–3 attempts.
5. **Fallback**: cached or default data, or a degraded response.
6. Move work that doesn't need to be synchronous to Kafka.

```
@CircuitBreaker(name = "gis", fallbackMethod = "cachedLocation")
@Bulkhead(name = "gis")
@TimeLimiter(name = "gis")
public CompletableFuture<Location> locate(String id) {
    return CompletableFuture.supplyAsync(() -> gisClient.get(id));
}
```

Config: `slowCallDurationThreshold: 2s` , `failureRateThreshold: 50` , `waitDurationInOpenState: 10s` .

F40. Your microservice is healthy but Kubernetes/load balancer marks it unhealthy. What will you check?

1. Run `kubectl describe pod` and read the Events for "Readiness probe failed", including the HTTP status or timeout.
2. Probe path and port: is it `/actuator/health/readiness` on the management port (if `management.server.port` is separate)? Is a context path missing?
3. Timing: `initialDelaySeconds` or a `startupProbe` that is too short for a slow Spring boot, or a `timeoutSeconds: 1` probe on a busy JVM.
4. Health groups: a downstream indicator (Redis, mail, DB) included in readiness makes the pod show DOWN while the app itself works. Keep liveness limited to `livenessState`.
5. Security: Spring Security returning 401 on actuator endpoints.
6. For the LB: the target group health-check path, port, and expected codes, plus NetworkPolicy and security groups.

```
management.endpoint.health.probes.enabled: true
management.endpoint.health.group.readiness.include: readinessState,db
```

F41. Kafka consumers are processing same message multiple times. How will you make processing idempotent?

Kafka gives at-least-once delivery by default, so duplicates will happen. Make the consumer idempotent:

1. Give every event a unique `eventId` (UUID) in a header or the payload.
2. Keep a `processed_events(event_id PK)` table and insert into it **in the same DB transaction** as the business update. A duplicate key means skip.
3. Or use natural idempotency: an upsert (`INSERT ... ON CONFLICT DO UPDATE`), a status transition guard (`WHERE status='PENDING'`), or a version check.
4. Reduce how often duplicates happen: commit offsets after processing (`AckMode.RECORD / MANUAL`), and tune `max.poll.interval.ms` so rebalances don't replay batches.
5. For Kafka-to-Kafka flows, use transactions (`transactional.id` , `isolation.level=read_committed`).

```
@Transactional
public void handle(Event e) {
    if (processedRepo.insertIfAbsent(e.id()) == 0) return; // ON CONFLICT DO NOTHING
    service.apply(e);
}
```

F42. Consumer lag is increasing continuously in production. How will you investigate?

1. Run `kafka-consumer-groups.sh --bootstrap-server ... --describe --group alerts-cg`. Check whether lag is on all partitions (throughput) or one partition (hot key or stuck consumer).
2. Compare the producer rate with the consumer rate in Grafana (`kafka_consumer_fetch_manager_records_lag_max` , or Burrow).

3. Look for rebalance storms in the logs ("Revoking partitions"). These are usually caused by processing that exceeds `max.poll.interval.ms`.
4. Profile per-record processing time: slow DB calls, external API calls, retries blocking the thread.
5. Check the consumer count against the partition count. Consumers beyond the number of partitions sit idle.
6. Fix it: add partitions and consumers, raise `concurrency`, batch DB writes, tune `max.poll.records`, or move slow work out of the listener.

F43. Messages are not reaching consumer even though producer sends successfully. What all things will you verify?

1. Confirm the producer write really succeeded: check the `send()` future/callback for `RecordMetadata` (partition and offset). Fire-and-forget hides errors.
2. Confirm the topic name matches exactly, including environment prefixes, and that `auto.create.topics` didn't create a typo topic.
3. Run `kafka-console-consumer --topic t --from-beginning` to check the data is actually on the broker.
4. Consumer group: run `kafka-consumer-groups --describe`. Are partitions assigned? Is a different `group.id` already ahead? `auto.offset.reset=latest` skips older messages.
5. Deserialization: a schema or type mismatch can make the error handler skip records silently. Check logs and the DLT.
6. Transactions: a `read_committed` consumer won't see aborted or uncommitted producer transactions.
7. ACLs, SSL/SASL config, and a listener `filter` or `RecordFilterStrategy` dropping records.

F44. Kafka broker goes down during high traffic. How will your system handle it?

The goal is that this is a non-event if we designed for it:

1. Topics use `replication.factor=3` and `min.insync.replicas=2`. When a broker dies, the controller elects new partition leaders from the ISR.
2. Producers use `acks=all` and `enable.idempotence=true` (the default in Kafka 3.x). They retry until `delivery.timeout.ms` without creating duplicates or reordering.
3. Clients get a bootstrap list of several brokers and refresh metadata automatically.
4. If the producer still fails, write to an outbox table or local buffer and alert. Don't drop data.
5. Consumers rebalance, reconnect to the new leaders, and resume from committed offsets.
6. Monitor `UnderReplicatedPartitions` and `OfflinePartitionsCount`, and keep `unclean.leader.election.enable=false`.

F45. Events are processed out of order causing wrong business logic. How will you maintain ordering?

1. Kafka only guarantees order **within a partition**, so use the entity ID (e.g. `incidentId`) as the message key. All events for that entity go to the same partition.
2. On the producer, `enable.idempotence=true` keeps order during retries (allowed with `max.in.flight.requests.per.connection ≤ 5`).

3. On the consumer, don't hand records to an unordered thread pool. Use container `concurrency` (one thread per partition) or key-ordered parallelism (Confluent Parallel Consumer).
4. Be careful with retry topics and DLTs: non-blocking retries break ordering. For strictly ordered flows, use blocking retries.
5. Defend against stale events anyway: store a `version` or event timestamp and ignore anything older than the current state.

```
kafkaTemplate.send("incident-events", incident.id(), event);
```

F46. One failed message blocks entire consumer processing. How will you handle poison messages?

1. Classify the error as transient (DB down) or permanent (bad JSON, validation failure).
2. Wrap deserialization in `ErrorHandlingDeserializer` so a bad payload doesn't loop forever.
3. Configure `DefaultErrorHandler` with a bounded backoff, then `DeadLetterPublishingRecoverer` to send the record to `<topic>.DLT` and move on.
4. Mark permanent exceptions as not retryable so they go straight to the DLT.
5. Alert on DLT volume, and give ops a tool to inspect and replay messages after the fix.

```
@Bean
DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> t) {
    var h = new DefaultErrorHandler(new DeadLetterPublishingRecoverer(t),
                                   new FixedBackOff(1000L, 3));
    h.addNotRetryableExceptions(ValidationException.class, DeserializationException.class);
    return h;
}
```

F47. Retry mechanism causes duplicate event processing. How will you avoid duplicates?

1. Producer side: `enable.idempotence=true` and `acks=all`. Broker-side sequence numbers remove duplicates caused by network retries.
2. Consumer side: an idempotent handler. Store `eventId` in a processed table in the same transaction as the update, or use upserts and state guards.
3. Make side effects idempotent too. Pass an idempotency key to the payment, SMS, or notification provider.
4. Retry only the failing step, not the whole flow. Don't re-publish events from a retry loop.
5. For consume-transform-produce, use Kafka transactions (`@Transactional` with a `KafkaTransactionManager`, consumers on `read_committed`) for exactly-once within Kafka.
6. Make sure the retry happens before the offset commit and stays under `max.poll.interval.ms`, or a rebalance will replay the batch.

F48. High volume event traffic slows down consumers. How will you scale Kafka consumers?

1. The partition count limits parallelism. Increase partitions (this changes the key-to-partition mapping, so plan for that) and run replicas up to the partition count.

2. Raise `ConcurrentKafkaListenerContainerFactory.setConcurrency(n)` per pod.
3. Autoscale pods on lag with KEDA's Kafka scaler instead of CPU.
4. Make each record cheaper: batch listener with `saveAll` and JDBC batching, cache lookups, move slow I/O off the listener.
5. Tune fetching: `max.poll.records`, `fetch.min.bytes`, `fetch.max.wait.ms`.
6. Split topics by priority (critical alerts vs telemetry) so bulk traffic doesn't starve important events.

```
spring.kafka.listener: { type: batch, concurrency: 6 }  
spring.kafka.consumer.max-poll-records: 500
```

F49. Offsets are committed incorrectly and messages are lost. How will you troubleshoot?

1. Check the config. `enable.auto.commit=true` commits on a timer even if processing failed. Spring Kafka sets it to false and uses container AckMode by default, but confirm nobody overrode it.
2. Look for async handoff. If the listener submits to an executor and returns, the offset gets committed before the work is done.
3. Look for swallowed exceptions. A `try/catch` that logs and continues means the record counts as processed.
4. Check `auto.offset.reset=latest` on a new or expired group, and `offsets.retention.minutes`.
5. Compare `kafka-consumer-groups --describe` offsets with DB records to find the gap. Replay with `--reset-offsets --to-datetime ... --execute` (group stopped), relying on idempotent consumers.
6. Fix with `AckMode.RECORD` or `MANUAL` plus `ack.acknowledge()` after processing, and a DLT for failures.

F50. Different services consume same event differently causing inconsistency. How will you design event contracts?

1. Treat events as public APIs. Put the schema (Avro/Protobuf/JSON Schema) in a Schema Registry with `BACKWARD` or `FULL` compatibility enforced.
2. Use a standard envelope: `eventId`, `eventType`, `version`, `occurredAt`, `source`, `correlationId`, and `payload` (CloudEvents style).
3. Document field meaning precisely: units, time zone (UTC ISO-8601), enums, nullability, and what "status=CLOSED" means.
4. Evolve by adding only optional fields. Breaking changes get a new version or topic, with both running in parallel during migration.
5. Add consumer-driven contract tests (Spring Cloud Contract or Pact) in CI.
6. Give each event type one owning producer, and have consumers treat the event as fact. Don't re-derive state differently.

F51. Payment updated successfully but notification service failed. How will you ensure eventual consistency?

1. Never dual-write ("save to DB, then call Kafka/HTTP"). Use the **transactional outbox**: in the same DB transaction as the payment, insert a `PaymentCompleted` row into an `outbox` table.

2. A relay (Debezium CDC, or a `@Scheduled` poller with `SELECT ... FOR UPDATE SKIP LOCKED`) publishes outbox rows to Kafka and marks them sent.
3. The notification service consumes with retries and backoff, uses a DLT, and dedupes on `eventId`.
4. Notification failure must not roll back the payment. It's a separate step that gets retried.
5. Add a reconciliation job and alerts for outbox rows stuck older than N minutes.

```
@Transactional
public void complete(Payment p) {
    paymentRepo.save(p.markPaid());
    outboxRepo.save(Outbox.of("PaymentCompleted", p.id(), toJson(p)));
}
```

F52. Distributed cache contains stale data and users see outdated information. How will you handle cache invalidation?

1. Use cache-aside: on write, **update the DB first, then evict the key** (`@CacheEvict`). Don't update the cache in place, because concurrent writes cause races.
2. Always set a TTL (`spring.cache.redis.time-to-live`) as a safety net.
3. For multi-instance local caches (Caffeine L1), broadcast invalidation through Redis Pub/Sub or a Kafka event, or drop L1 for volatile data.
4. To close the read-after-write race, use a delayed second delete or versioned keys (`user:42:v7`).
5. For cross-service data, invalidate from a CDC or Kafka event when the source changes.
6. Monitor hit ratio and staleness, and use short TTLs for frequently changing data like live traffic status.

```
@CacheEvict(cacheNames = "sensor", key = "#s.id")
@Transactional
public Sensor update(Sensor s) { return repo.save(s); }
```

F53. One server has updated data but another server still shows old data. How will you solve replication delay issue?

1. Measure the lag: Postgres `SELECT now() - pg_last_xact_replay_timestamp();` on the replica, or `pg_stat_replication`. MySQL: `SHOW REPLICA STATUS` → `Seconds_Behind_Source`.
2. Read-your-writes: send reads that follow a write (same user or session) to the primary for N seconds, or use `@Transactional(readOnly=false)` routing through `AbstractRoutingDataSource`.
3. Keep critical reads (payment status, balances) on the primary. Use replicas for reports and dashboards.
4. If needed, use synchronous replication (`synchronous_commit=remote_apply`) for strict cases. Writes get slower.
5. Fix the cause of the lag: long-running queries on the replica, heavy write bursts, network or IO. Alert when lag exceeds a threshold.
6. If the issue is app-level caches, invalidate them through events (see F52).

F54. Network partition occurs between services in distributed system. How will system behave?

By CAP, during a partition we choose consistency or availability for each use case:

1. Callers fail fast. Timeouts and circuit breakers open, and fallbacks serve cached or degraded data (availability).
2. For strongly consistent operations (payments, unique allocation), reject or queue the request instead of guessing.
3. Async flows keep working: producers buffer or write to the outbox, Kafka holds events, and consumers catch up after the partition heals.
4. Clustered stores: Kafka keeps acks=all writes only on the side with ISR quorum. `min.insync.replicas` stops writes on the minority side. Redis Sentinel/Cluster or etcd use quorum to avoid split-brain.
5. After healing: idempotent replay, reconciliation jobs, and conflict resolution (version or last-write-wins where acceptable).
6. Test it with chaos tools (Chaos Mesh, Toxiproxy).

F55. Multiple services update same resource causing inconsistency. How will distributed locking help?

1. A distributed lock ensures only one instance performs the critical section at a time across pods, e.g. assigning one ambulance to one incident.
2. Use Redisson `RLock` with a wait time and lease time (or its watchdog), always released in `finally`.
3. Locks can expire during GC pauses, so also protect the DB with **optimistic locking** (`@Version`) or a fencing token. Treat the lock as an optimization and the DB constraint as the guarantee.
4. Simpler options when everything is in one DB: `SELECT ... FOR UPDATE`, Postgres advisory locks, or a conditional update (`UPDATE ... WHERE status='FREE'`).
5. ShedLock for scheduled jobs.

```
RLock lock = redisson.getLock("vehicle:" + id);
if (lock.tryLock(2, 10, TimeUnit.SECONDS)) {
    try { assignVehicle(id); } finally { lock.unlock(); }
}
```

F56. One region/server goes down completely. How will high availability architecture handle it?

1. No single points of failure: at least 2 replicas per service, spread across zones (`topologySpreadConstraints` /pod anti-affinity), and a `PodDisruptionBudget`.
2. The load balancer or ingress health checks remove dead nodes, and K8s reschedules pods onto healthy nodes.
3. Data tier: Postgres primary plus standby with automatic failover (Patroni, RDS Multi-AZ), Redis Sentinel/Cluster, and Kafka RF=3 across racks/AZs.
4. Multi-region: active-passive with DNS failover (Route 53 health checks) and async replication, or active-active for stateless tiers. MirrorMaker 2 for Kafka.

5. Define RPO and RTO, keep backups in another region, and run DR drills regularly.
6. Keep services stateless (sessions in Redis or JWT) so any instance can serve any request.

F57. Traffic suddenly increases 10x during sale/event. How will distributed system scale?

1. Before the event: load test (JMeter/Gatling/k6) at 10x, pre-scale pods and nodes, warm caches, and raise DB and connection limits.
2. HPA on CPU or RPS with a reasonable `maxReplicas`, and Cluster Autoscaler or Karpenter for nodes.
3. Offload reads: CDN for static content, Redis for hot data, read replicas.
4. Absorb writes asynchronously: accept the request, push it to Kafka, process at a steady rate (queue-based load leveling).
5. Protect the core: rate limiting at the gateway (`RequestRateLimiter` with Redis), bulkheads, and circuit breakers.
6. Degrade gracefully by turning off non-critical features with feature flags. Watch dashboards live with an on-call bridge.

```
filters:
- name: RequestRateLimiter
  args: { redis-rate-limiter.replenishRate: 500, redis-rate-limiter.burstCapacity: 1000 }
```

F58. Data synchronization failed between microservices and database. How will you recover safely?

1. Stop the bleeding: pause the failing consumer or sync job so it doesn't make the divergence worse.
2. Find the scope: compare source and target by ID and `updated_at` /version, with counts and checksums per time window.
3. Recover from a reliable source: the outbox table, Kafka retention (reset consumer offsets to before the failure), or a CDC replay.
4. Make sure replay is idempotent (upserts, eventId dedupe, version checks), so reprocessing a window twice is harmless.
5. Replay into staging first or in small batches, then verify counts and business totals.
6. Add prevention: an outbox for dual writes, a DLT, a periodic reconciliation job, and alerts on drift.

F59. A service continuously retries failed requests and creates cascading failure. How will you stop it?

This is a retry storm. Retries multiply load on a service that's already struggling.

1. Immediate fix: open the circuit or disable retries with a config or feature flag, and scale or rate limit the victim service.
2. Limit retries to 2–3 with **exponential backoff and jitter**, and only for idempotent requests on transient errors (503, timeouts). Never retry 4xx.
3. Retry at **one layer only**, not in the gateway, client, and Kafka all at once ($3 \times 3 \times 3 = 27x$).
4. Put a circuit breaker in front of retries (Resilience4j order: Retry wraps CircuitBreaker), so an open circuit stops calls.

5. Respect `Retry-After` and set a timeout budget per request.

```
resilience4j.retry.instances.gis:
  maxAttempts: 3
  waitDuration: 200ms
  enableExponentialBackoff: true
  enableRandomizedWait: true
```

F60. Distributed tracing is missing for some requests. How will you debug cross-service issues?

1. Check sampling first: `management.tracing.sampling.probability` below 1.0 drops traces by design. Use tail sampling in the OTel Collector to keep errors and slow requests.
2. Find where propagation breaks. Use the Spring-managed `RestClient.Builder` / `WebClient.Builder` / `RestTemplateBuilder` (a `new RestTemplate()` isn't instrumented).
3. Async code: `@Async`, `CompletableFuture`, and custom executors lose context unless you wrap them with `ContextPropagatingTaskDecorator` or `context-propagation`.
4. Kafka: enable `spring.kafka.template.observation-enabled=true` and `spring.kafka.listener.observation-enabled=true`.
5. Proxies or gateways stripping the `traceparent` header. Legacy services using B3: align `management.tracing.propagation.type`.
6. In the meantime, correlate by a correlation ID in logs, timestamps, and gateway access logs.

F61. Production is down and users cannot login. What will be your immediate action plan?

1. Declare an incident, start a war room, set the severity, and post a status update to stakeholders. Then post updates every 15–30 minutes.
2. Find the scope: all users or some, which app, since when, and whether it matches a recent deploy or config or cert change.
3. Check the login path from end to end: gateway or ingress → auth service (Keycloak or our auth) → DB or LDAP → Redis sessions. Use `kubectl get pods`, `kubectl logs`, and the dashboards.
4. Common causes: expired cert or JWKS key rotation, DB connection pool exhausted, Redis down, a bad secret or config, or a clock skew breaking JWT `exp` / `nbf`.
5. Mitigate first: roll back (`kubectl rollout undo`), restart, scale, or fail over. Find the root cause after service is restored.
6. Verify recovery with a synthetic login, then write a blameless postmortem with action items.

F62. After deployment users report major issue. How will you decide rollback or hotfix?

1. Assess impact quickly: severity, number of users, data corruption risk, and whether it's getting worse.
2. **Default to rollback** if the impact is high and rollback is safe. It's the fastest known-good state (`kubectl rollout undo deployment/x`, Helm rollback, or flipping the feature flag off).

3. Check what blocks rollback: irreversible DB migrations (Flyway changes without backward compatibility), events already published in a new schema, or external contracts. This is why we use expand/contract migrations.
4. Choose a **hotfix** when the fix is small, well understood, quick to test, and rollback would cause more harm.
5. Either way: communicate, verify with metrics and a smoke test, then do an RCA and add a test or canary gate so it doesn't happen again.

F63. Logs are huge and issue is not reproducible locally. How will you debug production issue?

1. Narrow it down: get the exact time window, user, and request ID, then filter in Kibana or Loki by `traceId`, service, and level instead of using `grep`.
2. Open the trace for a failing request to see which service or span errored.
3. Compare failing and successful requests: payload, tenant, data volume, headers, pod or node.
4. Change log levels at runtime without a redeploy: `POST /actuator/loggers/com.trinity.incident {"configuredLevel": "DEBUG"}` for a short window.
5. Inspect the JVM live: thread dump, heap histogram (`jcmd GC.class_histogram`), or Arthas `watch / trace` on a method.
6. Reproduce in staging with production-like data (masked) and similar load. Add targeted logs or metrics if needed.

F64. One API fails only during peak traffic hours. How will you identify bottleneck?

1. Correlate the error spike with load: RPS, p95/p99, 5xx vs timeouts in Grafana for that endpoint only.
2. Check pool saturation: Hikari `connections.pending / timeout`, Tomcat `threads.busy` vs `max`, HTTP client pool, Redis connections.
3. Database: `pg_stat_activity` for waiting or locked queries, slow query log, `EXPLAIN ANALYZE` of the endpoint's query under real data volume, and lock contention.
4. JVM: GC pauses (`jvm_gc_pause`), heap, CPU throttling. Take thread dumps at peak.
5. Downstream dependencies: trace spans show whether an external call slows down or hits rate limits.
6. Reproduce with a k6/Gatling test at peak RPS, fix the constraint (index, pool size, cache, async), and retest.

F65. Application works fine from Postman but frontend users face errors. What will you check?

1. Browser DevTools first: the Console and Network tabs show the actual status and error.
2. **CORS**: Postman ignores CORS but browsers enforce it. Check the preflight `OPTIONS` response, `Access-Control-Allow-Origin / -Headers`, and credentials settings. Configure `CorsConfigurationSource` in Spring Security.
3. Auth differences: the token is missing or expired in the browser, cookies are blocked (`SameSite`, `Secure`), or there's a CSRF token for cookie-based sessions.
4. Request differences: `Content-Type`, payload shape, encoding, and header size.

5. Environment: the frontend points at a different base URL, a proxy or CDN caches old responses, or there's mixed content on HTTP vs HTTPS.
6. Client-side: a JS parsing error, timeout settings, or behavior specific to one browser.

F66. Monitoring alerts show high memory usage but application still runs. How will you analyze future risk?

1. Separate JVM heap from container memory. Look at `jvm_memory_used_bytes{area="heap"}` after GC compared with pod RSS. Off-heap (metaspace, threads, direct buffers) can get the pod OOMKilled even when heap looks fine.
2. Watch the trend: if post-GC heap stays flat, it's normal caching. If it keeps rising, it's a leak.
3. Check GC behavior: rising GC time or frequency and longer pauses show the risk is getting worse.
4. Take evidence: `jcmd <pid> GC.class_histogram` twice to compare, a heap dump (`GC.heap_dump`) analyzed in Eclipse MAT, and Native Memory Tracking for off-heap.
5. Check config: `-XX:MaxRAMPercentage=75`, container limits, and unbounded caches or collections.
6. Set alerts on post-GC heap > 80% and on growth rate, and plan the fix before it turns into an OOM.

F67. SSL certificate expired in production. What impact can happen and how will you resolve urgently?

Impact: browsers block users, mobile apps and API clients fail with `PKIX path / SSLHandshakeException`, service-to-service mTLS breaks, and webhooks and partner integrations stop.

1. Confirm: `openssl s_client -connect api.city.gov:443 -servername api.city.gov | openssl x509 -noout -dates`.
2. Find where TLS terminates: LB, ingress, gateway, or the app keystore.
3. Renew or reissue (cert-manager or Let's Encrypt, or the CA), then update the K8s TLS secret: `kubectl create secret tls api-tls --cert=... --key=... --dry-run=client -o yaml | kubectl apply -f -`.
4. Reload or restart the components that don't hot-reload certificates, and verify the full chain including intermediates.
5. Prevent recurrence: cert-manager auto-renewal, and an expiry alert at 30 and 7 days (blackbox exporter `probe_ssl_earliest_cert_expiry`).

F68. A batch job failed in middle after processing lakhs of records. How will you restart safely without duplicates?

1. Use Spring Batch's restartability. `JobRepository` stores the `ExecutionContext` and commit count per chunk, so restarting the **same JobInstance** (same identifying parameters) resumes from the last committed chunk.
2. Use chunk-oriented processing (e.g. 500 per chunk). Each chunk's writes and checkpoint commit in one transaction.
3. Use readers that save their position (`JdbcPagingItemReader` with a sort key, `saveState=true`), or a processed flag or status column.
4. Make writers idempotent (upsert or `ON CONFLICT`), so a partially written chunk is safe to redo.

5. Use `faultTolerant().skip(...).skipLimit(...)` and `retry` for bad records, and log skipped items.
6. Fix the root cause, restart with `JobOperator.restart(executionId)`, then reconcile counts.

F69. Server restarted automatically at midnight. How will you investigate root cause?

1. Find out what restarted: the pod, the container, or the node. `kubectl describe pod` → `Last State: Terminated, Reason: OOMKilled / Error, Exit Code` (137 = SIGKILL/OOM, 143 = SIGTERM).
2. `kubectl logs <pod> --previous` for the last lines before the crash, and `kubectl get events --sort-by=.lastTimestamp`.
3. Check whether it's scheduled: cron jobs, a midnight batch or report causing a memory spike, log rotation, backups, a CronJob doing `rollout restart`, or cluster autoscaler/node upgrade windows.
4. Liveness probe failures during heavy midnight load can make kubelet kill the container.
5. VM or bare metal: `journalctl -b -1`, `last reboot`, `dmesg | grep -i oom`, and cloud maintenance events.
6. Correlate metrics (memory, CPU, GC) around midnight, and enable `-XX:+HeapDumpOnOutOfMemoryError`.

F70. Business team says reports show wrong data in production. How will you validate and fix issue?

1. Get specifics: which report, filters, expected vs actual values, and since when. Get a sample record.
 2. Recompute the number directly from the source tables with SQL and compare it to the report output at each layer: source DB → ETL/view → API → UI.
 3. Usual causes: time zone or date boundaries (UTC vs IST), duplicate rows from joins, NULL handling, soft-deleted records counted, replica lag, stale cache or materialized view, or a failed or partial ETL run.
 4. Check recent changes: deploys, query changes, or upstream data issues.
 5. Fix the logic, add a test with known data, then backfill or recompute the affected period (`REFRESH MATERIALIZED VIEW`, rerun the job idempotently).
 6. Tell the business which reports were affected and for what dates, and add data quality checks (row counts, totals reconciliation).
-